

复杂产品系统行为建模与仿真设计及实现

余士严

2021 年 6 月

中图分类号：TP391.9

UDC 分类号：621

复杂产品系统行为建模与仿真设计及实现

作者姓名	<u>余士严</u>
学院名称	<u>机械与车辆学院</u>
指导教师	<u>王国新 副教授</u>
答辩委员会主席	<u>马树元 教授</u>
申请学位	<u>工程硕士</u>
学科专业	<u>机械工程</u>
学位授予单位	<u>北京理工大学</u>
论文答辩日期	<u>2021 年 6 月</u>

Design and Implementation of Behavior Modeling and Simulation for Complex Product System

Candidate Name:	<u>SHE Shiyan</u>
School Or Department:	<u>School of Mechanical Engineering</u>
Faculty Mentor:	<u>Associate Prof. WANG Guoxin</u>
Chair, Thesis Committee:	<u>Prof. MA Shuyuan</u>
Degree Applied:	<u>Master of Engineering</u>
Major:	<u>Mechanical Engineering</u>
Degree By:	<u>Beijing Institute of Technology</u>
The Date Of Defence:	<u>June, 2021</u>

复杂产品系统行为建模与仿真设计及实现

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签 名： 日期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名： 日期：

导师签名： 日期：

摘要

基于模型的系统工程（Model-Based Systems Engineering, MBSE）是一种持续贯穿系统整个研发生命周期的标准化建模方法。在系统的早期设计阶段，基于行为模型对系统进行仿真，能够验证系统行为逻辑的正确性和合理性。此外，随着产品复杂度逐渐提高，在早期设计阶段进行仿真验证，会避免因设计缺陷造成产品设计周期长，设计成本大等问题。同时，设计人员也必须面对复杂的各类系统，其中包括混合系统。因此在 MBSE 中进行混合系统行为仿真具有重要意义。目前在 MBSE 中进行系统行为仿真，主要存在两方面问题。一方面，用于系统行为仿真的工具平台依赖性强，与建模工具没有很好地结合，导致建模与仿真过程繁琐。另一方面，由于复杂产品全生命周期研发过程往往是一个多学科交叉开发的过程。进行系统行为建模时，领域工程师采用不同领域专用建模语言（UML, SysML, UPDM 等）建立各自领域的系统行为模型。且针对于不同行为模型，采用的仿真工具往往也不一致。缺少一种语言对这些建模语言的系统行为模型进行统一描述，以便于后续对这些行为模型进行仿真验证。为此本文对复杂产品系统行为建模与仿真设计及实现进行了相关研究。

（1）基于元元模型的系统行为建模方法研究。通过对系统行为模型的分析，采用 GOPRR-E 元建模方法实现对不同领域建模语言（SysML 和 UML）描述的系统行为模型的统一表达和构建。

（2）支持混合自动机的系统行为描述研究。通过混合自动机理论建立混合系统行为的描述机制，并与 GOPRR-E 元建模方法进行结合，解决混合自动机支持 SysML 和 UML 系统行为模型的统一描述问题。

（3）系统行为建模和仿真语言设计。在上述研究的基础上，提出了系统行为建模与仿真语言。在多架构建模语言 KARMA 语言的基础上设计系统行为建模与仿真语言的具体语法和抽象语法，使其具备描述系统行为的能力。

（4）系统行为仿真实现。构建采用系统行为建模与仿真语言描述的行为模型的半自动化仿真流程，并在建模工具 MetaGraph 中实现。该流程主要由仿真编译器和仿真求解引擎构成，编译器对可执行的系统行为模型进行编译，仿真求解引擎对仿真编译内容进行仿真。并在仿真流程的基础上，提出了多仿真文件合并和复合状态分解方法，

解决了多状态机联合仿真和复合状态仿真编译问题。

(5)在上述理论研究的基础上,通过四种不同场景分别对复杂产品系统中状态机图、活动图、序列图和多状态机图联合仿真进行仿真实现,验证了本文研究内容的可行性和有效性。

关键词: MBSE; 混合系统; 系统行为仿真; 混合自动机; KARMA 语言

Abstract

Model-Based Systems Engineering (MBSE) is a standardized modeling approach that continues throughout the entire R&D life cycle of the system. In the early design stage of the system, simulation of the system based on the behavior model can verify the correctness and rationality of the system behavior logic. In addition, with the gradual increase in product complexity, simulation verification in the early design stage will avoid problems such as long product design cycles and high design costs due to design defects. At the same time, designers must also face various complex systems, including hybrid systems. Therefore, it is of great significance to conduct mixed system behavior simulation in MBSE. At present, there are two main problems in the system behavior simulation in MBSE. On the one hand, the tool platform used for system behavior simulation is highly dependent and is not well integrated with modeling tools, resulting in cumbersome modeling and simulation processes. On the other hand, because the R&D process of complex products throughout the life cycle is often a multi-disciplinary cross-development process. When modeling system behavior, domain engineers use different domain-specific modeling languages (UML, SysML, UPDM, etc.) to establish system behavior models in their respective domains. And for different behavior models, the simulation tools used are often inconsistent. There is a lack of a language to uniformly describe the system behavior models of these modeling languages in order to facilitate subsequent simulation and verification of these behavior models. For this reason, this paper conducts related research on the design and implementation of complex product system behavior modeling and simulation.

(1) Research on system behavior modeling method based on meta-meta model. Through the analysis of the system behavior model, the GOPRR-E meta-modeling method is adopted to realize the unified expression and construction of the system behavior model described by different domain modeling languages (SysML and UML).

(2) Research on system behavior description supporting hybrid automata. The hybrid automata theory establishes the description mechanism of hybrid system behavior and

combines it with the GOPRRR-E meta-modeling method to solve the unified description problem of hybrid automata supporting SysML and UML system behavior models.

(3) Design of system behavior modeling and simulation language. Based on the above research, a system behavior modeling and simulation language is proposed. Design the concrete grammar and abstract grammar of the system behavior modeling and simulation language based on the multi-architecture modeling language KARMA language, so that it has the ability to describe the system behavior.

(4) Realization of system behavior simulation. Construct a semi-automated simulation process using the behavior model described by the system behavior modeling and simulation language, and implement it in the modeling tool MetaGraph. The process is mainly composed of a simulation compiler and a simulation solution engine. The compiler compiles the executable system behavior model, and the simulation solution engine simulates the content of the simulation compilation. And on the basis of the simulation process, a method of combining multiple simulation files and decomposing composite states is proposed, which solves the problems of multi-state machine co-simulation and composite-state simulation compilation.

(5) On the basis of the above theoretical research, the co-simulation of state machine diagram, activity diagram, sequence diagram, and multi-state machine diagram in complex product system is carried out through four different scenarios respectively, which verifies the feasibility and effectiveness of the research content in this paper.

Key Words: MBSE; hybrid system; system behavior simulation; hybrid automata; KARMA language

目录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 课题来源	2
1.3 国内外研究现状	2
1.3.1 MBSE 研究现状	2
1.3.2 MBSE 建模语言研究现状	5
1.3.3 MBSE 系统行为仿真研究现状	8
1.3.4 研究现状分析总结	10
1.4 研究内容及组织结构	10
1.5 本章小结	11
第 2 章 基于元元模型的系统行为建模方法研究	13
2.1 引言	13
2.2 系统行为定义及介绍	13
2.3 系统行为的参考模型及模型描述	13
2.3.1 描述系统行为参考模型	13
2.3.2 系统行为模型描述	15
2.4 基于 GOPPRR-E 元元模型的元建模方法	16
2.4.1 GOPPRR-E 元元模型介绍	16
2.4.2 基于 GOPPRR-E 方法的模型构建流程	20
2.5 描述系统行为的模型及元模型	21
2.6 本章小结	26
第 3 章 支持混合自动机的系统行为描述研究	27
3.1 引言	27
3.2 系统行为表现形式	27
3.2.1 系统行为的机制	27

3.2.2 采用混合自动机对混合系统进行描述	28
3.3 混合自动机对混合系统行为描述	31
3.3.1 混合自动机对混合系统行为描述的符号表达	31
3.3.2 混合自动机的运行过程	31
3.4 基于 GOPRR-E 与混合自动机理论的系统行为描述方法	32
3.4.1 混合自动机描述系统行为的模型	32
3.4.1 混合自动机描述系统行为的元模型	33
3.5 本章小结	34
第 4 章 系统行为建模与仿真语言设计	35
4.1 引言	35
4.2 KARMA 语言介绍	35
4.3 系统行为建模与仿真语言抽象语法	36
4.4 系统行为建模与仿真语言具体语法	37
4.4.1 系统行为建模与仿真语言关键字	37
4.4.2 语言具体语法设计	37
4.5 本章小结	45
第 5 章 系统行为仿真实现研究	47
5.1 引言	47
5.2 MetaGraph 工具介绍	47
5.3 系统行为仿真的半自动化流程设计	49
5.3.1 系统行为仿真引擎介绍及工具集成	49
5.3.2 仿真文件编译生成	50
5.3.3 系统行为建模与仿真流程设计	51
5.4 仿真编译文件映射关系研究	52
5.4.1 多系统行为模型联合仿真	52
5.4.2 复合状态仿真	53
5.5 本章小结	54
第 6 章 案例实现及验证	55
6.1 引言	55

6.2 多场景（状态机图、活动图、序列图）仿真案例	55
6.2.1 无人车行驶状态机图仿真案例	55
6.2.2 无人车行驶活动图仿真案例	60
6.2.3 机舱影音播放序列图仿真案例	65
6.3 探测拦截-多状态机图仿真案例	72
6.3.1 案例背景	72
6.3.2 案例流程	73
6.3.3 模型建立	74
6.3.4 案例系统行为仿真流程	76
6.4 本章小结	78
第 7 章 总结与展望	79
7.1 工作总结	79
7.2 未来展望	81
参考文献	83
攻读学位期间发表论文与研究成果清单	89
参与科研项目	89
发表论文	89
学科竞赛	89
附录 A	91
附录 B	95
致谢	121

本文图目录

图 1.1 MBSE 远景规划图	4
图 1.2 SysML 和 UML2.0 的关系	6
图 1.3 SysML 的分类及构成	6
图 1.4 论文组织架构	11
图 2.1 数据流行为模型	14
图 2.2 控制流行为模型	14
图 2.3 状态机行为模型	15
图 2.4 MOF 的层次结构(a) M3、M2、M1 三层之间的关系(b).....	16
图 2.5 GOPPRR-E 元元模型.....	17
图 2.6 关系描述	18
图 2.7 GOPPRR 中属性划分	19
图 2.8 基于 GOPPRR-E 的模型构建过程.....	20
图 2.9 系统行为模型统一描述	26
图 3.1 有限状态机	29
图 3.2 水位监测器混合自动机	29
图 3.3 混合自动机模型描述系统行为模型	32
图 3.4 混合自动机描述系统行为元模型	33
图 4.1 系统行为建模与仿真语言抽象语法	36
图 4.2 系统行为建模与仿真语言数据类型	37
图 5.1 MetaGraph 工具启动页面	48
图 5.2 仿真执行启动界面	49
图 5.3 系统行为建模与仿真语言执行过程	50
图 5.4 仿真文件编译界面	51
图 5.5 系统行为建模与仿真流程	51
图 5.6 多个.sim 文件编译并合并	52
图 5.7 联合仿真文件生成选择界面	53
图 5.8 复合状态	53

图 5.9 复合状态分解	54
图 6.1 无人车检测、图像收集、行驶过程	56
图 6.2 无人车行驶状态机图	56
图 6.3 无人车状态机图仿真结果	60
图 6.4 无人车检测、避障、巡航行驶过程	61
图 6.5 无人车探测过程活动图	61
图 6.6 无人车探测活动图仿真结果	65
图 6.7 影音播放系统与乘客、乘务员之间的交互	66
图 6.8 交互场景-音频视频娱乐方案序列图	67
图 6.9 交互场景-音频视频娱乐方案序列图仿真结果	71
图 6.10 探测拦截示意图	72
图 6.11 探测拦截流程图	73
图 6.12 地面雷达站状态机图	74
图 6.13 我方船舰、预警机、战机状态机图	75
图 6.14 地方船舰和战机状态机图	76
图 6.15 建模、仿真流程	77
图 6.16 探测拦截系统系统行为仿真结果	77
图 6.17 探测拦截系统系统行为 SVG 显示	78

本文表目录

表 2.1 GOPPRR-E 元元模型.....	17
表 2.2 GOPPRR-E 元元模型连接规则.....	19
表 2.3 SysML 状态机图元模型	21
表 2.4 SysML 活动图元模型	22
表 2.5 SysML 序列图元模型	23
表 2.6 UML 状态图元模型	23
表 2.7 UML 活动图元模型	24
表 2.8 UML 序列图元模型	25
表 4.1 系统行为建模与仿真语言关键字	37
表 4.2 逻辑运算规则	44
表 6.1 影音播放系统音乐、视频统计表	65
表 6.2 仿真功能实现总结	78

第1章 绪论

1.1 研究背景与意义

1.1.1 研究背景

近年来，复杂产品系统呈现出功能高度集成、各学科复杂耦合、设计信息庞杂等发展趋势。具有高研制成本，技术密集和客户化定制等特点的重大装备产品、系统、网络、控制中心、软件程序包、建筑群或服务方案，称为复杂产品系统^[1]。传统的基于文档的系统工程难以保证复杂产品系统开发过程中数据的一致性、且难以准确地描述需求分析、仿真等活动过程，已经无法满足复杂系统的开发需求。从而导致其研发过程面临新的问题：需求更新困难、系统层设计验证不足、理解不一致性等。因此，基于模型的系统工程（Model-based System Engineering, MBSE）作为问题的有效解决方案被提出，现已成为系统工程的发展方向^[2]。

采用 MBSE 方法在系统设计早期阶段进行系统行为模型仿真验证，可以帮助发现设计缺陷，从而降低设计成本、缩短研发周期，提高系统的可靠性。对系统行为模型进行仿真验证需要明确系统的类型（连续系统、离散系统、混合系统）。在复杂产品系统中，混合系统包含连续系统和离散系统的特性。混合系统是一种变量连续变化与离散事件发生并存且相互之间存在信息交互的动态系统，其广泛存在于工业生产控制、电力控制、军事应用等诸多领域之中。混合系统作为计算机科学与控制科学的交叉领域，既有描述连续变化的模拟量，也有表征离散事件的数字量，二者间存在复杂的相互作用，构成了复杂的动态变化模型，能够较好地表征实际生产生活中诸多动态过程^[3]。因此，在 MBSE 方法应用中，针对混合系统进行系统行为仿真验证更具有意义。大部分混合系统行为建模基础采用的是混合自动机（Hybrid Automata）^[4]，该模型的基本思想是，以离散行为为基础形成状态机。通过向状态机中的每个状态赋予函数表示的连续行为，从而将离散和连续结合起来，表示混合行为。

1.1.2 研究意义

本课题的研究力图设计一种对复杂产品系统的行为模型进行统一描述的语言。由

于复杂产品全生命周期研发过程往往是一个多学科交叉开发的过程。进行系统行为描述及建模时，领域工程师采用不同领域专用建模语言（UML，SysML，UPDM 等）建立各自领域内的系统行为模型，且针对不同的行为模型，采用的仿真工具也不一致。缺少一种语言对这些建模语言的系统行为模型进行统一描述，以便于后续对这些行为模型进行仿真验证。并且用于系统行为仿真的工具对平台依赖性强，与建模工具没有很好地结合，设计模型到仿真模型的信息传递低效，导致建模与仿真过程繁琐。

针对以上问题，本文进行复杂产品系统混合行为建模与仿真语言设计及实现研究，其意义在于：

（1）设计可对系统行为模型进行统一描述的语言——系统行为建模与仿真语言。该语言结合 MBSE 中 GOPPRR-E 元建模方法和混合自动机理论，对多架构建模语言 KARMA 语言进行语法和语义扩充，使其具备描述系统行为的能力。

（2）系统行为建模仿真过程方便、灵活，且容易理解。一方面，将系统行为仿真工具与建模工具结合，设计模型建立到仿真的半自动化流程，增加设计模型到仿真模型的信息传递效率。另一方面，用户在使用系统行为建模与仿真语言进行系统行为描述过程时，只需要在元模型层增加的四个模块（Automaton 图、Group 对象、State 对象、Transition 关系）中添加对应描述内容，使得建模仿真过程更为方便快捷，且容易理解。除此之外，在系统行为模型的半自动化仿真流程的基础上，提出了多仿真文件合并和状态分解方法，解决了多状态机联合仿真和复合状态仿真编译问题。

1.2 课题来源

本课题来源于国家重点研发计划：“复杂产品设计制造服务一体化系统建模理论”，本课题的研究内容是项目研究中的关键组成部分。

1.3 国内外研究现状

本文分别对 MBSE、建模语言研究现状、系统行为仿真三个方面的国内外研究现状及存在问题进行了梳理。

1.3.1 MBSE 研究现状

2007 年，国际系统工程协会（International Council on Systems Engineering, INCOSE）

在《系统工程 2020 年愿景》^[5]中，正式给出了“基于模型的系统工程（MBSE）”的定义：MBSE 是一种应用建模方法的正式范式，用于支持系统需求、设计、分析、校验和确认活动，这些活动从概念设计阶段开始，贯穿整个开发过程及后续的生命周期阶段。MBSE 以模型代替文档，模型相比文本、有着图形可视化、无歧义、表达简便、可靠性高、容易传播交流和支持模型交换、集成等诸多优点。MBSE 是由模型驱动的，从需求阶段开始，通过模型的不断演化和迭代，实现了产品的系统设计，并在系统工程层面上保证了设计方案的正确性和优化。SysML（System Modeling Language）的出现，基于模型的现代系统工程方法就逐步替代了传统的基于文本的系统工程方法。

采用标准系统建模语言（System Modeling Language, SysML）构建系统设计模型，可作为可追踪的信息集成框架。支持从概念设计的需求阶段开始，通过模型的不断演化、迭代支持系统设计、分析、验证和确认等活动，并贯穿整个研发过程和系统的后续生命周期阶段^[6]。这种新的系统研发模式具有以下优势：集成表达产品研发早期需求、功能、结构和行为四方面的信息，为研发过程的需求管理和追踪提供便利；通过持续动态仿真环节验证发现概念设计方案存在的不合理之处并及时修正；系统设计模型具有唯一性，为研发的不同参与者提供了直观的、统一的、无二义性的设计信息交流平台^[7]。

1.3.1.1 MBSE 国外研究现状

国际系统工程学会（INCOSE）于 2007 年正式对 MBSE（Model-Based Systems Engineering）进行定义，同时发布了《系统工程 2020 年愿景》，制定了 MBSE 的发展规划，如图 1.1 所示^[8,9]。根据规划，2020 年前后要形成较为成熟和规范的 MBSE 理论。2025 年将 MBSE 方法推广为主流系统工程方法^[10]。

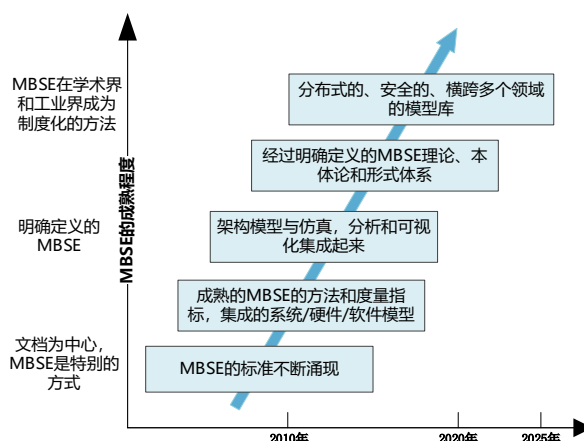


图 1.1 MBSE 远景规划图

目前 MBSE 在各个领域均得到广泛应用，主要在航空航天、国防武器系统研制、汽车行业等领域。NASA 的喷气推进实验室（JPL）在实际的工程中采用 MBSE 的方式来研制航天系统，主要的工作集中在获取系统架构相关的信息并通过基于模型的方式使用系统建模语言（SysML）进行设计。为了支持系统的建模，开发了一种基于严格框架下特定领域的语言，并进行飞行系统研制的设计信息形式化分析以及设计工具的集成^[11]。自 MBSE 被正式确立以来，NASA 喷气推进实验室即制定了长期的应用发展战略，目前已在约 20 个开发任务的全生命周期中应用该思想^[12]。NASA 所属的兰利研究中心主要是在 MISSE-X 项目早期，即项目需求分析及系统规范设计阶段，采用商业公司开发的专业软件展开尝试，取得良好效果，随后该方法被迅速推广至该中心的其他项目^[13, 14]。瑞典的林雪平大学在其 Skeldar 无人直升机的项目中采用 SysML 系统建模语言来对无人系统的总体进行设计工作，它们研究了 MBSE 的系统工程方法和工具在工程实践中的应用，并总结了在 Skeldar 设计中的 MBSE 的实践经验^[15]。此外，欧洲航天局一直以来致力于系统全生命周期数据一致性和基于模型的验证等工作，在例如虚拟航天器设计、数据映射编辑器和空间系统数据库等典型项目中应用了 MBSE。著名的洛克希德·马丁公司等也都在 MBSE 的应用和推广方面给予了极大助力^[16]。

1.3.1.2 MBSE 国内研究现状

国内 MBSE 虽然起步较晚，但由于近些年来对国外先进思想、技术的大量引入，MBSE 越来越受到各领域研究机构的重视。其中最为典型的是中航工业成都飞机设计

研究所,该所协同中航工业信息技术中心于2014年率先启动了MBSE的试点项目^[17]。该所与IBM构建战略合作关系,使用IBM Rational Doors搭建需求管理平台,采用Harmony-SE方法论,基于IBM Rational Rhapsody建模工具,实现了需求、设计、建模、仿真、测试等的集成开发,取得成功尝试^[18]。上海飞机设计研究院采用基于模型的系统工程方法来进行民用飞机的系统顶层设计^[19]。西北机电工程研究所的曹雷等在雷达系统设计中,用基于模型驱动的设计方法,在Rhapsody中应用UML2.0标准建模语言建立了系统开发过程中各个阶段的模型,根据模型设计并实现了雷达仿真系统^[20]。

此外,各高校也相继开展对MBSE的实践应用。哈尔滨工业大学胡涛副教授,带领学生黄昌彬选用MBSE思想为理论指导,使用系统建模工具Rhapsody对导弹测试内容进行顶层建模,在Eclipse环境下,使用GEF(Graphical Editor Framework)图形化编辑框架进行流程设计模块的搭建,以Activiti引擎为仿真模块后台,完成了导弹测试流程设计仿真软件的开发^[21]。南京航空航天大学的曹云峰教授,运用MBSE理论,以Rhapsody为开发环境,对无人机飞控管理系统展开顶层建模,并与Matlab、Petri网等结合展开联合仿真^[22, 23]。电子科技大学的黎业飞副教授将MBSE引入北斗接收机的建模仿真开发中,使用SysML(Systems Modeling Language)在Rhapsody建模环境中建立了完整的北斗接收机系统模型,并对其进行了动态仿真^[24]。

1.3.2 MBSE 建模语言研究现状

在实践MBSE时,通常需要建模语言来创建系统的需求、结构、行为和约束模型。然而建模语言具有多种类型,不同设计领域的工程师和设计人员都有可用的建模语言,不同领域的工程师和设计师根据其领域的特点选择建模语言。建模语言分两大类,第一类是图形建模语言(例如:SysML、UML、UPDM、BPMN等);第二类是文本建模语言(例如:Modelica等)。

(1) UML

统一建模语言UML(Unified Modelling Language)自1997年被对象管理组织OMG(Object Management Group)定为标准以来,已经在各界广泛应用。UML本身也在不断地改进,目前最新的版本是UML2.5。UML是软件工程领域的标准建模语言,支持模型化和软件系统开发的图形化表示,主要为了解决不同建模语言之间表达方式不同

的问题^[25]。UML 定义了九种不同类型的图，从不同侧面对系统进行描述，便于对系统进行分析 and 设计。但是在 UML 是软件工程的建模语言，在 MBSE 领域适应性不高。

(2) SysML

SysML (Systems Modeling Language) 是国际系统工程学会 INCOSE (the International Council on Systems Engineering) 和对象管理组织 OMG (Object Management Group) 在 UML2.0 的基础上推出的一种标准系统建模语言。SysML 在 UML 的基础上开发，消除了不同建模语言在表达和术语上的不一致性^[26]。SysML 和 UML2.0 的关系如图 1.2 所示。SysML 的分类及构成如图 1.3 所示。使用 SysML 建模语言进行系统建模不仅方便直观，而且可重用，以确保设计过程的一致性，减少模型之间的不确定性。SysML 能够以图形化多视图建模的方式支持各种复杂系统的详细描述、分析、设计和验证。

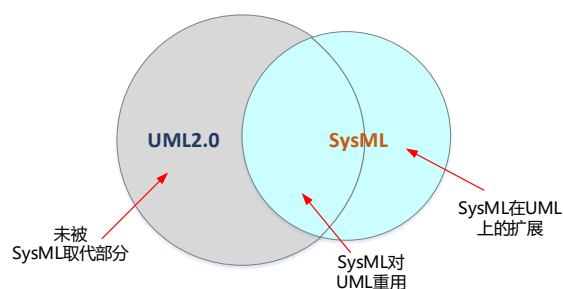


图 1.2 SysML 和 UML2.0 的关系

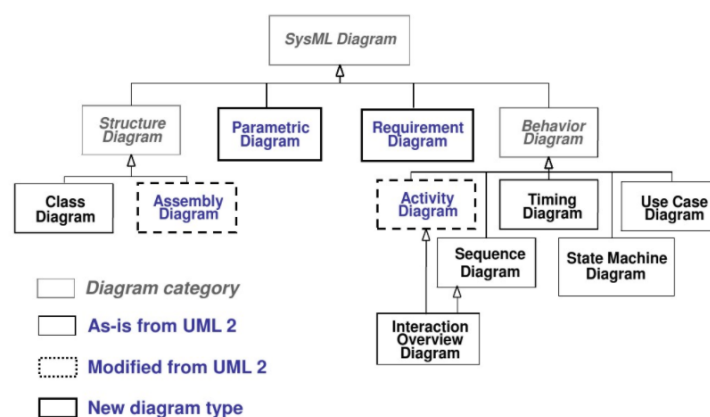


图 1.3 SysML 的分类及构成

使用 SysML 对系统进行设计的目的在于提高开发人员和设计人员之间的信息交互能力，建立统一的系统架构。它可以支持系统工程应用的多领域系统，包含硬件、

软件、信息等系统的需求分析、系统设计、功能描述及系统验证等^[27]。

(3) BPMN

BPMN(Business Process Modeling Notation)是 BPMI(Business Process Management Initiative)业务流程管理倡议组织开发的一套标准化的业务流程建模语言^[28]。它定义了业务流程图,可以用于设计业务流程操作的图形化模型,该模型定义了业务包含的活动及这些活动操作顺序的流程控制器,为复杂系统提供流程建模的支持。BPMN 可以提供整个系统中全部利益相关者都可以理解的一套语义,但是适用范围仅限在业务流程建模中。

(4) DSML

DSML(Domain-Specific Modeling Language)领域特定建模语言是针对特定的系统开发的建模语言,具有很强的针对性,可以很好地满足特定领域的建模需求,DSML 的种类较多。法国 Thales 公司的建模软件 Capella 中应用的建模语言就是 DSML 中的一种类型^[29]。该语言通过对需求、功能、组件等组件及其拓扑关系的模型化表达,对系统的概念和关系进行描述。

(5) UPDM

武器装备体系统一建模平台 UPDM(Unified Platform for Defense Modeling),是目前世界上最先进的体系工程和武器平台论证工具^[30]。支持作战能力与武器装备体系论证的最新标准,并为武器装备体系模型验证提供手段^[31]。

(6) AADL

架构分析和设计语言 AADL(Architecture Analysis and Design language)是一种可扩展的标准架构描述语言^[32]。与 SysML 和 UML 等其他建模语言相比,AADL 为标准的软硬件组件分类提供了强大的语义,这样常用分析方法和良好定义的系统集成就可以根据这些规范实施。

(7) IDEF0

IDEF0(Integrated DEFinition language 0)是在 SADT(Structured Analysis and Design Technique)基础上进一步形式化形成的 MBSE 建模语言,可以形式化地表示数据流,系统控制等^[33]。但是这种表示方法也缺少统一标准,在对大型系统进行建模分析时易出现交互问题。

(8) Modelica

Modelica 是一种开放的，多领域统一建模规范，适用于复杂物理系统架构描述的文本化语言，具有对系统的动态行为进行有效的仿真的能力^[34]。Modelica 通常应用于工程领域，不但能够表达系统的物理行为，还能统一表达子系统间的能量、信号交换及在子系统间的其他的连续交互行为。

从上述建模语言可以看出，在不断地迭代过程中，MBSE 可以满足通用领域的标准化建模，并在系统的全生命周期中为利益相关者提供互相认可的语义。但是在不同领域下的建模语言不尽相同，在一定程度上造成了资源的浪费，因此如何实现不同领域的快速建模成为一个急需解决的问题。

1.3.3 MBSE 系统行为仿真研究现状

1.3.3.1 系统行为仿真语言和工具研究现状

目前系统行为仿真语言和工具的发展趋势逐渐向多领域方向发展。应用广泛的是基于通用仿真语言 Modelica 的多领域统一建模和仿真工具，例如国外的 Dymola、AMESim、MapleSim，以及国内的 MWorks^[35]，这些工具多数属于商业软件。Dymola 作为支持通用仿真语言 Modelica 的软件，可以进行多领域系统仿真，例如机、电、热混合系统。Dymola 建模和仿真的基础就是 Modelica 建模语言，里面存储了大量的文件形式的 Modelica 模型，也就是涉及特定物理系统领域有关组件模型的 Modelica 库，Dymola 支持状态化的创建和仿真。

基于仿真建模语言 Simulink 的 Matlab/Simulink 仿真工具目前也在应用。Stateflow 是一种图形化的设计开发工具，是有限状态机的图形实现工具，主要用于 Simulink 中控制和检测逻辑关系。它和 Simulink 同时使用，在 Simulink 中绘制 Stateflow 状态图。Stateflow 模型一般是嵌在 Simulink 模型下运行，且由事件驱动，使得 Simulink 更具有事件驱动控制能力。

基于混合自动机仿真执行求解语言的混合自动机求解引擎^[36]具有支持可伸缩向量图形 (Scalable Vector Graphics, SVG) 可视化仿真、混合自动机行为仿真等特点^[37]。该求解引擎目前应用于实际工业环境，例如使用基于状态的监督控制综合来提高患者通信控制系统的可扩展性^[38]，行李处理监控控制器的设计和实时仿真^[39]，飞利浦 MPI 扫描仪监控控制器的设计^[40]。

1.3.3.2 系统行为仿真转换研究现状

由于 MBSE 中的系统设计模型无法直接执行仿真验证,因此需要将其与系统仿真模型 (Modelica 或 Simulink 模型等) 进行集成转换以弥补不足。在设计和仿真集成方向的研究方面,Johnson 等^[41]基于 SysML 扩展机制建立与 Modelica 等价的模型元素,采用基于 TGG (Triple Graph Grammars) 的转换方法建立 2 种语言元模型的映射关系,实现自动转换。

Schamai 等通过扩展 SysML 并定义新的方程图和仿真图以构建可执行的系统仿真模型^[42]。

Cao 等以 Simulink/MapleSim 作为目标仿真平台,通过建立面向不同平台的仿真补充模型,将 SysML 系统混合行为模型自动转换到 Simulink/MapleSim 中进行系统行为仿真^[43]。但是 Matlab/Simulink 的仿真偏向于控制系统仿真,并不是一个完整的系统工程仿真平台。

北京航空航天大学刘继红、李新光等^[44]在基于元模型体系的模型转换思路基础上,提出一种基于 SysML 的系统设计-仿真模型可视化转换方法。针对具体的 SysML 系统设计模型,通过对转换规则与转换活动元模型的实例化层次建模确定 SysML 元模型与 Modelica 目标模型之间的转换关系,执行动态转换活动实现转换。该方法并没有对 UML/SysML 进行扩展,而是通过对 Modelica 进行元模型建模,并与 SysML 元模型建立映射关系。但是这种映射关系是粗粒度的对应关系,许多 Modelica 的基础语义都没有表达,无法满足复杂机电系统的映射转换需求。

W. Schamai^[45]提出了一种对 UML 状态机图进行 Modelica 转换的正式方法,将状态机图作为离散和连续行为混合建模的载体,通过在状态转换中添加注释来与连续行为结合。但是纯文本的描述方式处理方式僵硬,缺乏形式化的模型表示。另外,该方法没有解决状态机图与结构模型之间的参数对应问题,使得状态机图游离于结构模型之外。

张政^[46]以 Modelica 元模型为基准,分析和构造了面向 SysML 和 Modelica 集成的 SysML 扩展——M-Design。通过抽象模型映射的转换方程,构造了平台的设计和仿真模型映射规则,利用仿真工具 AMESim 对仿真建模和集成进行实现。

通过对上述研究工作的分析,总结出如下三个设计和仿真集成研究的现有问题:一是选择的仿真语言和平台不具备多领域的统一性,使得一些领域的仿真语义无法表

达；二是对于仿真语言的某些语义构造缺乏形式化表达，使得设计语言的表达比较僵硬；三是采用的转换实现方法有局限性，难以维护和扩展。

1.3.4 研究现状分析总结

根据国内外研究现状可知，学者们在 MBSE 的应用、建模语言、系统行为仿真等方面展开了广泛而深入的研究，已经取得了很多有价值的研究成果，但依旧存在一些缺点或不足，具体表现在以下几方面：

(1) 由于 MBSE 中的系统设计模型无法直接执行仿真验证，需要将系统设计模型与系统仿真模型（Modelica 或 Simulink 模型等）进行集成转换来进行仿真验证。目前一些建模工具（MagicDraw、Rhapsody 等）通过 Matlab/Simulink 或支持 Modelica 的 Dymola、AMESim 等仿真工具进行自动机行为仿真，需要将系统行为模型转化为仿真工具环境下的模型，缺少从建模到仿真的自动化流程，转化过程复杂，并且设计模型到仿真模型的信息传递低效。上述研究能够实现特定系统领域内设计模型向仿真模型的转换过程，通过仿真来验证系统的性能需求，但存在平台依赖性强，适用性低等方面的不足。

(2) 由于复杂产品全生命周期研发过程往往是一个多学科交叉开发过程，对系统行为进行建模时，领域工程师采用不同的建模语言建立各自特定域（UML，SysML，BPMN，UPDM 等）行为模型，且对于不同的行为模型，采用的仿真工具也不一致。缺少一种语言对这些建模语言的行为模型进行统一描述，以便后续的仿真。

1.4 研究内容及组织结构

本文从基于模型的系统工程领域中的混合系统行为描述及仿真方法出发，发现用于系统行为仿真的工具平台依赖性强，与建模工具没有很好地结合。此外，由于复杂产品全生命周期研发过程往往是一个多学科交叉开发过程，进行系统行为描述及建模时，领域工程师采用 MBSE 方法需要建立各自特定域（UML，SysML，UPDM 等）行为模型，缺少一种语言对这些建模语言的系统行为模型进行统一描述，以便于后续的仿真验证。为此本文提出了复杂产品系统混合行为建模与仿真语言设计及实现进行了相关研究。在 GOPRR-E 元建模方法的基础上，结合系统行为参考模型以及可描述系统行为的元模型和模型，以及混合自动机系统行为描述的过程，扩充多架构建模语言

KARMA 语言语法和语义,设计系统行为建模与仿真语言,具备描述系统行为的能力。并在 MetaGraph 工具上设计开发系统行为仿真流程(即系统行为建模与仿真语言实现过程)。

为了完成此过程,本文在第一章阐述了课题的研究背景和意义,并对 MBSE 研究现状、MBSE 建模语言研究现状和 MBSE 系统行为仿真研究现状及存在问题进行了总结。本文第二章给出了基于元元模型的系统行为建模方法,实现对系统行为模型统一描述和构建。本文第三章中支持混合自动机的系统行为描述研究,实现对混合系统行为的统一描述。本文第四章根据前两章的内容,以及在 KARMA 语言的基础上,扩充 KARMA 语言的具体语法和语义,设计系统行为建模与仿真语言,具备描述系统行为的能力。本文第五章为实现系统行为仿真,构建了一个支持系统行为建模与仿真语言的建模和仿真流程,并在建模工具 MetaGraph 中实现。本文第六章通过设计四个不同场景案例,验证该研究的可行性。本文第七章为总结与展望。本文的组织架构如图 1.4 所示。

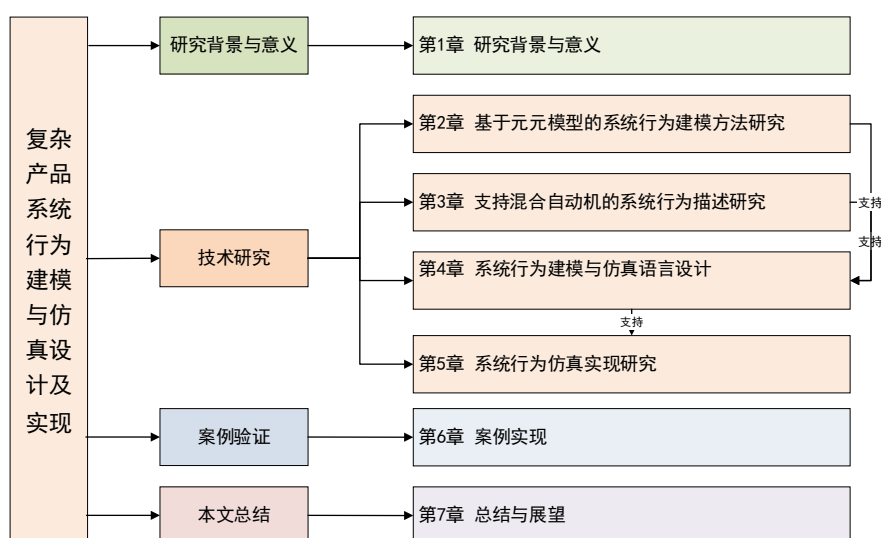


图 1.4 论文组织架构

1.5 本章小结

本章提出了相关研究的研究背景、研究意义以及国内外研究现状。确定了本文的研究内容,并提出了论文组织架构。

第2章 基于元模型的系统行为建模方法研究

2.1 引言

复杂产品系统行为是指系统中对状态变量随时间变化的特性或对外界事件的感应。系统行为的表达主要有三个参考模型，即数据流行为模型、控制流行为模型、状态机行为模型。因此，在 MBSE 领域中，对系统行为的描述可参考以上三个模型。

然而，在使用基于模型的系统工程方法进行建模的过程中，针对不同建模语言可以构建不同行为模型，例如通过 SysML (Systems Modeling Language) 可构建状态机图、活动图、序列图三种行为模型来描述系统行为；通过 UPDM (Unified Profile for DoDAF and MoDAF) 可构建状态转换模型来描述系统行为。因此，缺少一种方法对系统行为模型统一描述构建。

针对上述问题，本章基于 M3-M0 的设计框架，在 GOPRR-E (GOPRR-Extension) 建模方法的基础上对系统行为的参考模型进行分析，总结分析出 SysML 和 UML 建模语言中对可用于构建描述系统行为的模型，并描述系统行为的模型基本元素，可实现对系统行为模型统一描述和构建。

2.2 系统行为定义及介绍

系统行为是系统中的状态变量随时间变化的特性，一般指的是对象之间的关系交互情况。同时，系统行为表达系统的动作与反应，这些活动指的是外界可见的、可测试的系统活动。系统中对象的行为，指的是它们的活动、交互和状态^[47]。

2.3 系统行为的参考模型及模型描述

2.3.1 描述系统行为参考模型

系统行为参考模型指的是可以准确描述系统行为的模型，体现系统中对象与对象或环境之间的交互过程。

系统行为参考模型提供了有关系统功能的可执行描述。描述系统行为的参考模型主要分为三类：数据流行为模型 (Data Flow Behavior Model)、控制流行为模型 (Control Flow Behavior Model)、状态机行为模型 (State Machine Behavior Model)。

2.3.1.1 数据流行为模型

数据流行为模型侧重于描述输入和输出之间的功能依赖，当其他步骤提供数据输入时，数据流会向下一步骤执行。该模型不强调步骤的顺序，因为步骤的所有输入到达的时间取决于各种输入步骤完成所需的时间。数据流行为模型如图 2.1 所示。

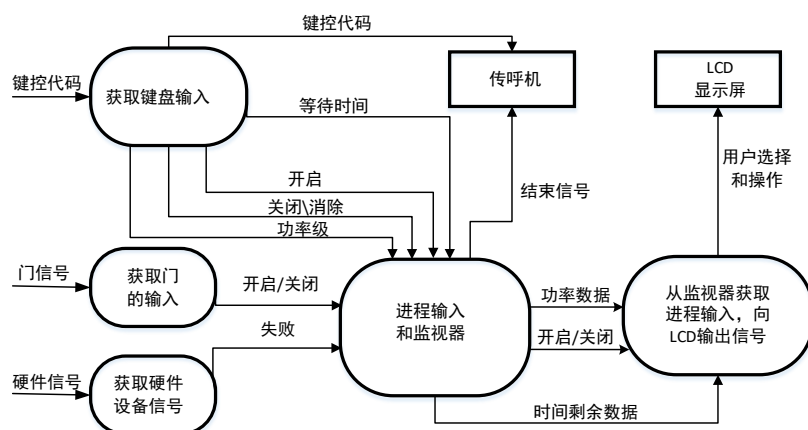


图 2.1 数据流行为模型

2.3.1.2 控制流行为模型

控制流行为模型适用于描述具有精确顺序和步骤的系统行为。控制流行为模型需要一个步骤完成后开始另一个步骤，控制流行为模型如图 2.2 所示。

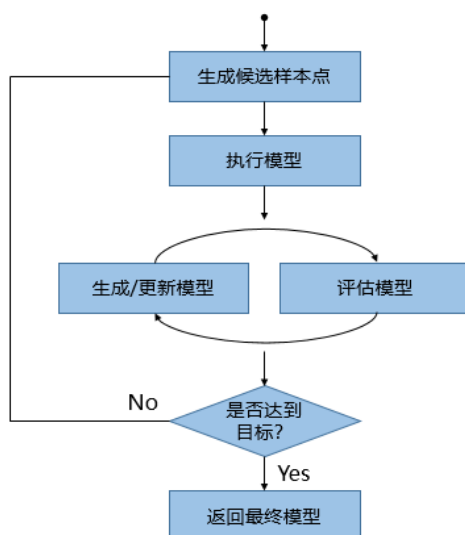


图 2.2 控制流行为模型

状态机行为模型适用于描述对象状态明确、状态之间的转换通过事件的发生进行的系统行为。状态机行为模型如图 2.3 所示。

图 2.3 状态机行为模型

根据 2.3.1 小节中关于系统行为参考模型的分类以及描述，本小节主要通过状态机行为模型、控制流行为模型、数据流行为模型这三种参考模型总结出在 SysML、UML 中可用于描述系统行为的模型。

- 状态机图通过系统中有限状态进行离散行为建模。与活动图和序列图不同的是，状态机图关注的是系统中结构如何根据随时间发生的事件改变状态。
- 活动图用于协调各种系统行为的输入、输出、序列以及条件，说明随着时间的推移，行为和事件的发生序列。活动图可以表示各种各样的活动，描述复杂的行为。
- 序列图用于描述参与者与其相关组件之间的控制流。它表达了系统的一种动态行为。

态视图，这种视图可体现随着时间推移而发生的行为和事件的序列。序列图是对行为的精确说明，因此更适合用于在详细设计阶段。

在 UML 中，行为图是用于描述系统的动态模型和组成对象间的交互关系。其行为模型主要包括状态图、活动图以及序列图。与 SysML 中的行为图类似，状态图、活动图、序列图分别对应于系统行为参考模型中的状态机行为模型、数据流行为模型、控制流行为模型。

- 状态图用于描述类的对象所有可能的状态以及事件发生时状态的转换条件。
- 活动图用于描述满足用例要求所要进行的活动以及活动间的约束关系。
- 序列图用于显示参与者与系统的对象之间交互的一系列顺序步骤，强调消息是如何在对象之间被发送和接收。

2.4 基于 GOPPRR-E 元元模型的元建模方法

2.4.1 GOPPRR-E 元元模型介绍

为了元建模过程的规范化，OMG 制订了元对象机制 (Meta-Object Facility, MOF)，MOF 的层次结构如图 2.4(a)所示，其中 M1、M2、M3 之间的关系如图 2.4 (b)所示。根据目前大多数的建模语言的形式化可以将模型的元素提取，作为元元模型，分别为图(Graph)、对象(Object)、点(Point)、属性(Rproperty)、角色(Role)、关系(Relationship)，即 GOPPRR 方法^[48]。该方法可以构建特定领域的元模型，开发需要的特定域建模语言，并在此基础上支持建立模型，实现系统的形式化表达。

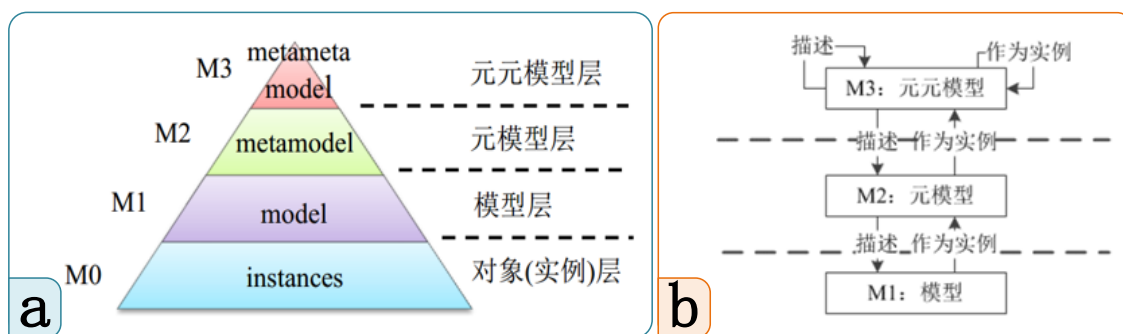


图 2.4 MOF 的层次结构(a) M3、M2、M1 三层之间的关系(b)

由于这些元元模型之间存在一定的引用、关联、绑定等规则和约束，这些规则和

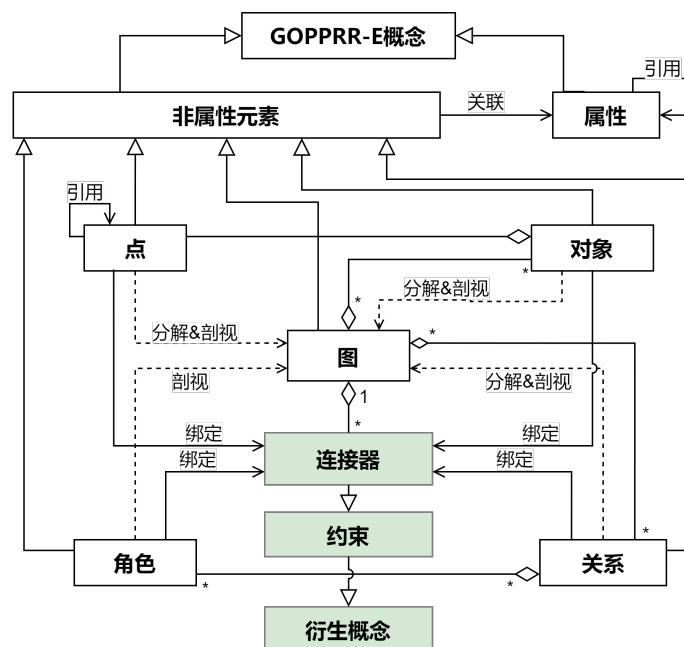


图 2.5 GOPPRR-E 元元模型

表 2.1 GOPPRR-E 元元模型

17

元元模型	名称	描述
Property	属性	(1) 描述其余五种类型元素的性质，不能单独存在 (2) 属性可以具有不同的数据类型（字符串，文本，数字，布尔值等） (3) 允许属性类型之间的引用，可以继承属性的数据类型，如“出生日期”，“日期”等属性，共用同一数据类型
Relationship	关系	表示对象之间的关联，通过角色和对象相连
Role	角色	描述对象或对象上的点如何与关系相连接，表示对象类型在关联中的角色

GOPRRR-E 具有六种底层元元模型：图（Graph）、对象（Object）、点（Point）、属性（Property）、关系（Relationship）、角色（Role）。GOPRRR-E 方法中的扩展（Extension），用于描述元元模型之间的规则和约束。以下将对这七部分内容分别进行介绍。

（1）图（Graph）

该图描述的并不是一个“图形”，而是一个元类。每个图是由一组对象（Object）和绑定（Binding）组成的。在绑定中存储着如何连接图中的对象（Object）、点（Port）、角色（Role）和关系（Relationship）的信息。

（2）对象（Object）

与图类似，此处的对象不是通常意义上的“对象实例”，而是指“对象类型”。对象是“图”中最常见的元素，如一个状态、一个按钮等。

（3）关系（Relationship）

关系是表示两个对象之间是如何连接的，每个联系通过“角色”与“对象”相连接。关系的表现形式如图 2.6 所示。

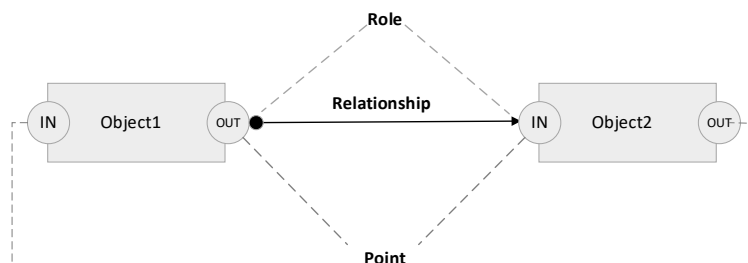


图 2.6 关系描述

（4）属性（Property）

属性是用来描述或修饰 GOPPRR (Graph、Object、Point、Role 和 Relationship)。GOPPRR 按概念划分可以分属性部分和非属性部分。GOPPRR 属性划分如图 2.7 所示。

属性部分：属性部分不能单独存在，而必须依附在非属性部分，它描述了非属性构造块所具有的属性，每个属性包含两个字段：属性名称和属性类型。Property 属于属性部分。非属性部分：每个非属性构造块都有自己的属性。Graph、Object、Point、Role 和 Relationship 都属于非属性部分。

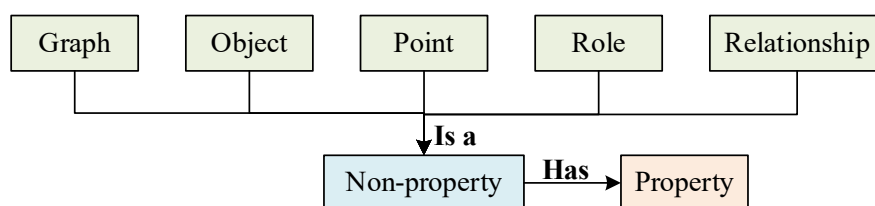


图 2.7 GOPPRR 中属性划分

(5) 点 (Point)：通常附属到对象上，连接对象和角色表示连接的端口。

(6) 角色 (Role)：在关系的两端，连接对象，表示对象以何种方式或身份进行连接。

(7) 扩展 (Extension)：元元模型之间存在一定的规则和约束，具体的关联关系见表 2.2。

表 2.2 GOPPRR-E 元元模型连接规则

关联规则	描述
包含	描述图类型和对象、关系、角色类型之间的组合关系
属性连接	描述图、对象、关系、角色、点五种类型元素和属性类型的映射关系
剖视	描述一个对象可以实现到一个或多个图的映射
分解	描述一个对象可以分解成一个图，这个图和对象之间的关系为一一映射
绑定	(1) 在有点参与的建模中，用于描述对象上的点和关系两端角色的连接关系（如 SysML 的内部模块图） (2) 在无点参与的建模中，用于描述对象和关系两端的角色之间的连接关系（如 SysML 的需求图）

关联规则	描述
	(3) 每个对象的端点应关联至少一个角色
关联	描述对象类型和点类型，关系类型和角色类型之间的连接关系

2.4.2 基于 GOPPRR-E 方法的模型构建流程

根据 MOF 框架以及 GOPPRR-E 元建模方法，模型的描述和构建过程如图 2.8 所示。在 MOF 框架中，每一层的元素严格对应上层模型元素，即上层模型是下层模型的抽象，下层模型是上层模型的实例，整体的建模过程以逐层递进的方式进行。

元元模型作为最基本的元素，其不包含任何与领域相关的信息。首先通过元元模型实例化为元模型。元模型一般是特定领域相关的，是特定领域概念的抽象描述。领域元模型受到领域相关标准、规范等的限制。因此，在元建模过程中，需要参考领域概念、标准、规范等，通过元元模型实例化形成元模型。模型的构建过程称为建模，即建模的产物是模型。建模是元模型实例化的过程。特定域模型是通过建模语言进行描述，例如 UML，SysML 语言都属于特定域建模语言。最后通过特定域建模语言构建的多种视图用于描述某个真实系统的视角。M0 层是模型的实例，用于描述利益相关人视角。例如需求分析师关注的系统需求视角，系统设计师关注的系统架构视角。

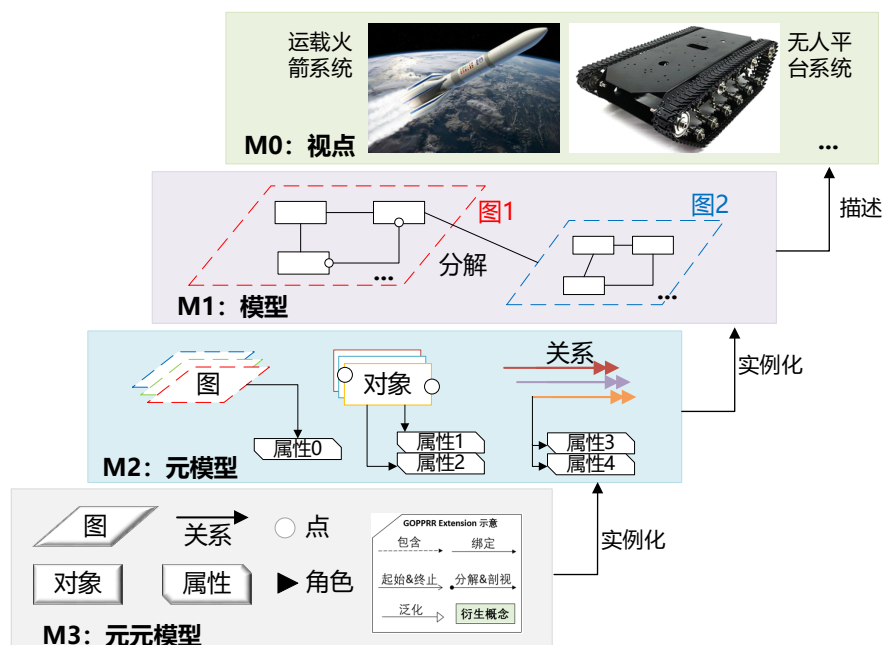


图 2.8 基于 GOPPRR-E 的模型构建过程

2.5 描述系统行为的模型及元模型

在 2.3.2 小节中，总结出在 SysML 建模语言中，描述系统行为的模型包括状态机图、活动图、序列图。在 UML 中，描述系统行为的模型包括状态图、活动图以及序列图。为了模型建立以及第 4 章节系统行为建模与仿真语言描述的便捷性，本小节主要分析在 SysML 以及 UML 中行为模型中的基本功能单元。

SysML 状态机图元模型见表 2.3。其中“开始”、“结束”、“状态”作为基本功能单元可以用于描述系统行为中某一对象或事件发生的起始，终止以及中间状态情况。“转换”关系是一种基本功能单元，可以用于描述状态之间的转换关系，代表的是从一种状态向另一种状态的改变。

表 2.3 SysML 状态机图元模型

状态机图	图标	说明
选择		对输入的状态进行判断
注释		对图或者对象进行说明
点		对状态进行汇集，转向后续的状态，或表示状态转换的标志点
开始		表示一个状态的起始点
状态		描述一种状态
结束		整个状态机的结束点
终止		其中一个对象流的状态的终结，对整个状态机图的结束无影响
转换		状态之间的转换，连接起始点和结束点
依赖		表示一个状态依赖于另一个状态
连接注释		对图或者对象进行注释说明

SysML 活动图元模型列表见表 2.4。其中“开始”、“结束”可以用于描述系统行为中某一对象或事件发生的起始，终止情况。“动作”表示某种类型的处理或者转换情况，它会在系统操作过程中活动被执行的时候发生。“对象流”关系可以用于表示事件、能量或者数据的传递过程。“控制流”用于表示一系列动作之间的序列约束。

表 2.4 SysML 活动图元模型

活动图	图标	说明
动作		活动中的节点，系统操作执行的活动
选择		对流入的事件进行判断选择
阻流器		对其中一个流程的动作进行终止
注释		对一个图或者一个对象进行说明
包		对具有某特定属性的对象进行划分，表示其属于一个整体或类型
划分		对活动的阶段进行划分
发送动作		从其输入对象的操作，并将其传输到目标对象，进行状态转换或执行活动
开始		活动的起始点
结束		活动的结束点
节点		表示某个动作之间的关系，包括合并节点、决定节点、分支节点、集合节点
时间事件		等待时间动作，可以表示到特定时刻或者时长之后进行后续进程
栓		一种特殊类型的对象节点，把栓附加到动作上，表示动作的输入或输出
控制流		连接活动图中的对象，控制流的到达可以启动等待它的动作
连接注释		对图或者对象进行说明
对象流		表示在系统操作过程中活动执行时，从一个对象向另一个节点或对象流动

SysML 序列图元模型列表见表 2.5。其中“生命线对象”可以用于描述系统行为中某一对象。“执行说明”表示对象之间信息交互的执行状况。“消息”表示发送生命线和接收生命线之间的通信。

表 2.5 SysML 序列图元模型

序列图	图标	说明
操作符		表示操作符类型
注释		对图或者对象进行说明
外部角色		序列图中的角色
生命线对象		表示交互活动中参与者的单一实例，会与其他生命线交换数据
执行说明		附加到生命线上，表示事件
创建消息		生成消息
传递消息		将消息在生命线中进行传递
发送消息		表示消息发送
关联		连接角色、消息、生命线等
删除消息		对传递的消息进行删除
依赖		把消息附加到生命线上
回复消息		对消息进行回复传递

UML 状态图元模型列表见表 2.6。其中“开始”、“结束”分别表示系统行为的初态和终态。“状态”用于描述系统行为中某一对象在事件发生之间某时刻所处的情形。

“转换”表示两个状态之间的关系，它表明当某个事件发生时，对象从当前状态转换到后来的状态。

表 2.6 UML 状态图元模型

状态图	图标	说明
选择		对输入的状态进行判断
注释		对图或者对象进行说明
包		对具有某特定属性的对象进行划分，表示其属于一个整体或类型
点		表示状态转换的标志点
开始		表示一个状态的起始点
状态		描述一种状态

状态图	图标	说明
结束		整个状态机的结束点
终止		表示其中一个对象流的状态终结
转换		状态之间的转换，连接起始点和结束点
附属		将注释文本附属到对象上提供说明
连接注释		对图或者对象进行注释说明

UML 活动图元模型列表见表 2.7。其中“开始”、“结束”分别表示系统行为的初态和终态。“活动”用于描述系统行为中某一对象的活动。“对象流”关系可以用于表示事件、能量或者数据的传递过程。“控制流”用于表示一系列动作之间的序列约束。

表 2.7 UML 活动图元模型

活动图	图标	说明
活动		系统操作执行的活动
选择		对流入的事件进行判断选择
阻流器		对其中一个流程的动作进行终止，对整个活动不造成影响
注释		对图或者对象进行说明
包		对具有某特定属性的对象进行划分，表示其属于一个整体或类型
划分		对活动的阶段进行划分
发送动作		从其输入对象的操作，并将其传输到目标对象，进行状态转换或执行活动
开始		活动的起始点
结束		活动的结束点
控制流		连接活动图中的对象，控制流的到达可以启动等待它的动作
附属		表示一个对象依赖另一个对象或将注释文本附属到对象上提供说明
连接注释		对图或者对象进行说明

活动图	图标	说明
对象流		表示在系统操作过程中活动执行时，从一个对象向另一个节点或对象流动

UML 序列图元模型列表见表 2.8。其中“生命线对象”可以用于描述系统行为中某一对象。“激活”是对象操作的执行。“消息”是对象之间的一种通信机制。由发送对象向另一个或其它几个接收对象发送信号，或由一个对象（发送者或调用者）调用另一个对象（接收者）的操作。

表 2.8 UML 序列图元模型

序列图	图标	说明
激活		在生命线上表示对象操作的执行
注释		对图或者对象进行说明
外部角色		序列图中的角色
生命线对象		表示序列的顺序，为一条直线有顺序关系
框架		用矩形框框处表示一个层次
传递消息		将消息在生命线中进行传递
发送消息		表示消息发送
回复消息		对消息进行回复传递

系统行为指的是系统中对象和对象之间的交互，通过对 SysML 和 UML 中行为模型中的基本功能单元——“对象”和“关系”进行抽取和抽象表达，实现对系统行为模型的统一描述，如图 2.9 所示。

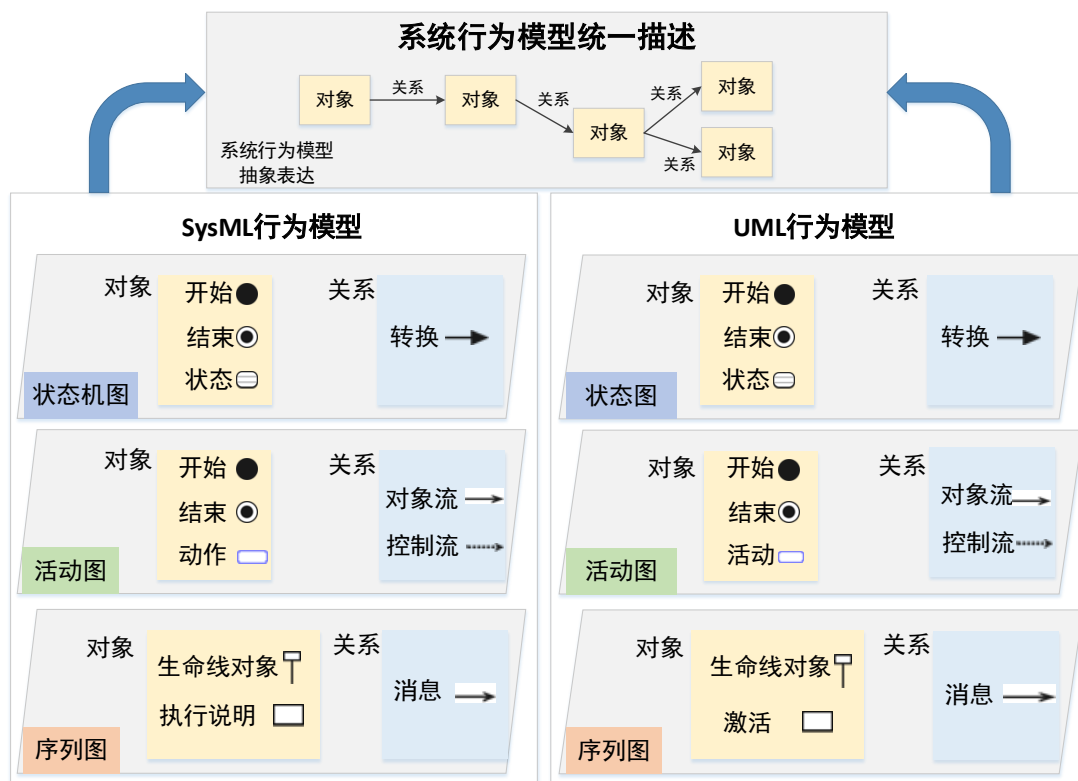


图 2.9 系统行为模型统一描述

2.6 本章小结

本章针对在 MBSE 领域中,采用不同建模语言构建不同行为模型时,缺少对系统行为模型统一描述构建的问题,采用 GOPRR-E 元建模方法对系统行为模型进行统一描述构建,主要是对 SysML 和 UML 建模语言中可用于构建描述系统行为的模型进行统一描述构建。首先通过系统行为定义以及可描述系统行为的三个参考模型(数据流行为模型、控制流行为模型、状态机行为模型)为出发点,分析出在 SysML 和 UML 建模语言中可描述系统行为的模型。然后基于 GOPRR-E 元建模方法,对上一步总结出的模型进行分析,尤其是描述系统行为的模型基本元素,可实现对系统行为模型统一描述和构建。

第3章 支持混合自动机的系统行为描述研究

3.1 引言

随着现在复杂产品系统的复杂度不断提高, 以及进行系统设计的自动化程度、集成度不断提高, 产品设计人员在设计过程中必须面对复杂的各类系统。复杂产品系统是由许多相互作用的组件组成的, 且这些组件之间的总体行为是非线性的^[49]。在复杂产品系统设计早期阶段进行系统行为仿真验证可以避免因设计失误造成的设计成本大、设计周期长的问题。并且系统行为描述是进行系统行为仿真验证的前提。因此, 进行复杂产品系统行为仿真验证, 首先需要对系统行为进行确切地描述。

由于复杂产品系统主要可以分为: 连续系统、离散系统、混合系统。因此, 在对系统行为进行描述时, 需要判别复杂产品系统的类型。复杂产品系统中的混合系统是由连续变量动态系统和离散事件动态系统相互混合、相互作用而形成的统一的动态系统^[50]。混合系统的主要特征是离散控制信号和连续过程变量之间相互作用。对于混合系统的分析通常是围绕着系统中物理元素的连续状态和离散事件之间的复杂关系展开研究。因此, 对混合系统进行系统行为描述更符合复杂产品系统的特点。

3.2 系统行为表现形式

在复杂产品系统中, 系统行为的主要表现形式有三种: 离散型系统行为、连续型系统行为、混合型系统行为。每种系统行为都有不同的描述方式。

3.2.1 系统行为的机制

3.2.1.1 离散型系统行为描述

离散系统的变量演变是由事件驱动的, 而时间只是隐含在其中, 处理该类事件时并不需要考虑事件发生的时间以及事件与事件之间的动态演化。此类系统也称为离散事件系统。离散型系统行为一般通过逻辑、代数等形式模型来进行描述, 例如有限自动机、Petri 网等。Petri 网^[51, 52]因具有良好的描述与分析离散事件动态系统行为的特性, 被广泛地应用于各领域的复杂系统建模与行为分析中。

3.2.1.2 连续型系统行为描述

连续系统是指变量跟随时间进行演化的系统，也称为时间演化系统^[53]。例如在车辆前进系统中，速度、路程等都是随时间连续变化的物理量。在连续系统中，连续型系统行为可以用微分方程或差分方程来描述。

3.2.1.3 混合型系统行为描述

混合系统是一种包含连续变量系统和离散事件系统的复杂动态系统。此类系统既包含随时间进行演化的变量，又包含按事件进行演化的变量^[54]。在该系统中使用单一的微分方程、差分方程或有限自动机来描述已经远远不够。在对混合型系统行为描述时，最有效的方法之一就是连续型系统的行为描述与离散型系统的行为描述很好的结合。这就需要将描述连续型系统的行为模型嵌入到离散型系统的行为模型中。目前最主要的描述混合系统行为模型为混合自动机模型。

3.2.2 采用混合自动机对混合系统进行描述

3.2.2.1 混合自动机理论介绍

混合自动机最早是由美国 R.Alur 等^[55]在 20 世纪 90 年代提出，并给出了一些有关理论介绍。混合自动机是一种具有离散跳跃转换和连续变量的有限状态机。

有限状态机（Finite State Machine, FSM）又称确定性有限状态机（Deterministic Finite Automata, DFA）^[56]。有限状态机表示有限个状态以及在这些状态之间的转移和动作等行为的数学模型。图 3.1 描述的是一个状态机图，其中 S1 为起始状态，表示状态机的开始；S6 为最终状态，表示状态机的结束；S2、S3、S4、S5 为中间状态；字母 A、B、C 等表示触发状态转换的事件，例如当事件 N 发生时，状态 S2 会转换为状态 S6，即到达最终状态，该状态机结束。

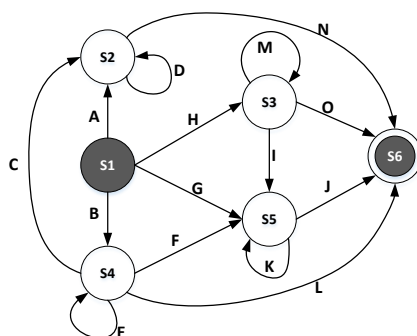


图 3.1 有限状态机

混合自动机是混合系统的一种形式化模型，也是目前混合系统建模中最常用的一种模型^[57]。即描述一组离散和连续变量在时间上进行演化的动态系统。

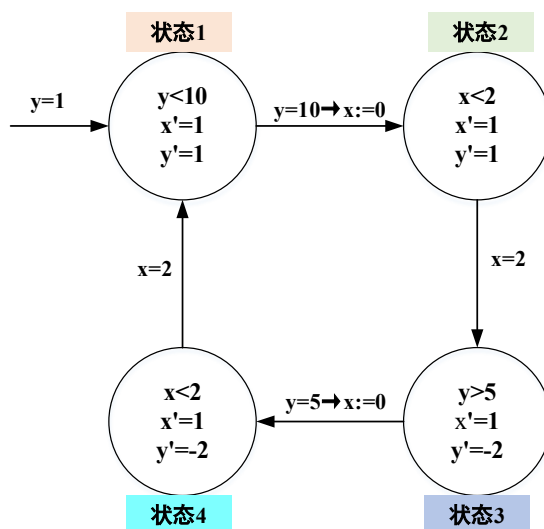


图 3.2 水位监测器混合自动机

图 3.2 描述的是一个水位监测器混合自动机，水箱中的水位是通过一个监测器进行控制。监测器可以不断的监测水位并可以控制水泵的开启和关闭。水位随时间的变化是分段线性函数。当水泵关闭时，水位（变量 y 表示）每秒下降 2 分米；当水泵开启时，水位每秒上升 1 分米。假设初始水位是 1 分米，此刻水泵开启。要求水位保持在 1 到 12 分米之间。由于水位检测器发出信号改变水泵的状态到改变生效的时间，有 2 秒的延迟。因此，水位检测器必须在水位下降到 1 分米之前就发出信号打开水泵，在水位达到 12 分米之前必须发出信号关闭水泵。

当水位分别超过 5 分米和 10 分米时，水位监测器会发出信号。图 3.2 描述的混合自动机有四个状态，在状态 1 和状态 2 的位置，水泵是开启的；在状态 3 和状态 4 的位置，水泵是处于关闭状态。时钟 x 用于指定延迟，无论自动控制是在状态 2 还是状态 4 的位置，开启和关闭水泵的时间都在 x 秒前被发送。

在状态 1 位置， $y < 10$ 表示水位 y 在状态 1 位置，保持在 10 分米以下； $x' = 1$ 表示时钟 x 每秒增加 1； $y' = 1$ 表示水泵在该状态处于开启状态，水位 y 每秒增加 1 分米。在状态 2 位置， $x < 2$ 表示时钟 x 保持在 2 秒以内； $x' = 1$ 表示时钟 x 每秒增加 1； $y' = 1$ 表示水泵在该状态处于开启状态，水位 y 每秒增加 1 分米。在状态 3 位置， $y > 5$ 表示水位 y 在状态 3 位置，至少保持在 5 分米以上； $x' = 1$ 表示时钟 x 每秒增加 1； $y' = -2$ 表示水泵在该状态处于关闭状态，水位 y 每秒下降 2 分米。在状态 4 位置， $x < 2$ 表示时钟 x 保持在 2 秒以内； $x' = 1$ 表示时钟 x 每秒增加 1； $y' = -2$ 表示水泵在该状态处于关闭状态，水位 y 每秒下降 2 分米。

状态 1 和状态 2 之间的转换关系表示，当水位 y 等于 10 分米时，转换关系触发，且时钟 x 的值变为 0。状态 3 和状态 4 之间的转换关系表示，当水位 y 等于 5 分米时，转换关系触发，且时钟 x 的值变为 0。

3.2.2.2 采用混合自动机对混合系统行为进行描述的优势

存在多种可以模拟不同类型行为的自动机形式。离散事件系统的离散行为可以通过确定性有限自动机^[58]（DFA, Deterministic Finite Automaton）进行表示。具有重要的时间信息的离散行为可以通过时间自动机^[59]（Timed Automaton）进行表示。连续动态系统通常用微分方程表示。由于混合系统包含这些行为，因此，进行混合系统建模最合适的建模形式之一就是混合自动机（Hybrid Automaton）。混合自动机本质上代表了时间自动机，能够通过一组微分方程加入到自动机的状态中来描述连续动态系统^[60]。

混合自动机模型是有限状态自动机的推广。它将描述连续动态行为的微分方程嵌入到传统的离散状态机模型中，从而使得自动机模型兼备描述连续行为的能力。在混合自动机中，状态可以看作是在一组微分方程控制下，一组连续变量的连续变化过程，而将状态的转移、转换看作是由事件驱动的结果。

3.3 混合自动机对混合系统行为描述

3.3.1 混合自动机对混合系统行为描述的符号表达

混合自动机 $H^{[61]}$ 可表示为:

$$H = (\text{Loc}, \text{Lab}, \text{Edg}, X, \text{Init}, \text{Inv}, \text{Flow}, \text{Jump}) \quad (3.1)$$

其中, Loc 表示有限离散状态的集合, $\text{Loc} = \{l_1, l_2, \dots, l_m\}$ 。

Lab 表示一组有限的同步标签集合, 也称为字母表, 它可以用来协调几个状态机之间的动态变化。

Edg 表示离散状态之间相互转换的集合, 将每组离散状态之间的迁移作为一条边, $\text{Edg} \subseteq \text{Loc} \times \text{Lab} \times \text{Loc}$ 。

X 表示有限变量的集合, 且 $X = \{x_1, x_2, \dots, x_n\}$, 其中 X 划分为非控制变量 U 和控制变量 Y 。混合自动机 H 的一个状态 s 由一个位置 l 和一个变量值 x 组成, $s = (l, x)$ 。

Init 表示初始状态的集合。 $\text{Init} \subseteq \text{Inv}$, 混合自动机 H 的所有行为都必须从初始状态开始。

Inv 表示不变量集合, 为每一个离散状态 s 分配一个不变条件。当混合自动机处于状态 s 时, 变量的值随着可能的轨迹进行变化, 但是其满足的条件不变。

Flow 表示流动关系, 其中 $\text{Flow}(l) \subseteq \mathbb{R}^X \times \mathbb{R}^X$, 给出每个状态 (l, x) 的导数 $\dot{x}$ 集合。例如使用一个微分方程:

$$\dot{x} = f(x) \quad (3.2)$$

给定一个位置 l , 持续时间的轨迹 $\delta \geq 0$, 是连续可微的函数 $\xi: [0, \delta] \rightarrow \mathbb{R}^X$, 对于所有的 $t \in [0, \delta]$, $(\dot{\xi}(t), \xi(t)) \in \text{Flow}(l)$ 。如果所有的 $t \in [0, \delta]$, $\xi(t) \in \text{Inv}(l)$, 其轨迹满足不变。

Jump (跳跃关系) 表示一个赋值函数, 为每个控制状态之间的迁移赋予一个触发条件的连续变量取值条件。其中, $\text{Jump}(e) \subseteq \mathbb{R}^X \times \mathbb{R}^{X'}$ 为每个转换 $e \in \text{Edg}$ 定义 x 的后继集合 x' 。跳跃关系通常由保护集 $\mathcal{G} \subseteq \mathbb{R}^X$ 和赋值 (或重置) $x' = r(x)$ 定义为:

$$\text{Jump}(e) = \{(x, x') | x \in \mathcal{G} \wedge x' = r(x)\} \quad (3.3)$$

3.3.2 混合自动机的运行过程

混合自动机的运行: 从一个初始状态开始, 随着时间的推移, 状态随着微分方程

和跳跃关系（Jump）进行演化。特殊事件，也称为非受控分配，可对非受控变量进行任意更改的环境进行模拟。

$$(l_0, x_0) \xrightarrow{\delta_0, \xi_0} (l_0, \xi_0(\delta_0)) \xrightarrow{\alpha_0} (l_1, x_1) \xrightarrow{\delta_1, \xi_1} (l_1, \xi_1(\delta_1)) \dots \xrightarrow{\alpha_{N-1}} (l_N, x_N) \quad (3.4)$$

其中， $\alpha_i \in Lab \cup \{\tau\}$ ，满足 $i = 0, \dots, N-1$ ；

- 第一个状态是混合自动机的初始状态， $(l_0, x_0) \in \text{Init}$ 。
- 轨迹：在位置 l_i ， ξ_i 提供一个持续时间 δ_i 的轨迹，满足不变式。
- Jump（跳跃关系）：如果 $\alpha_i \in Lab$ ，存在一个转换 $(l_i, \alpha_i, l_{i+1}) \in Edg$ 和跳跃关系 $\text{Jump}(e)$ ， $(\xi_i(\delta_i), x_{i+1}) \in \text{Jump}(e)$ 并且 $x_{i+1} \in \text{Inv}(l_{i+1})$
- 非受控分配：如果 $\alpha_i = \tau$ ，那么 $l_i = l_{i+1}$ 且 $\xi_i(\delta_i) \downarrow_Y = x_{i+1} \downarrow_Y$ 。这表示可能对非受控变量 $U = X \setminus Y$ 执行的任意分配。

如果对于 i ，存在状态 $(l_i, x_i) = (l, x)$ ，则状态 (l, x) 是可达的。

3.4 基于 GOPPRR-E 与混合自动机理论的系统行为描述方法

3.4.1 混合自动机描述系统行为的模型

在第 2 章节中，总结出可以描述系统行为的模型包括：SysML 状态机图、活动图、序列图以及 UML 状态图、活动图、序列图。由于混合自动机模型是对混合系统行为进行描述。因此，可以将上述每个行为图都可以理解是一个混合自动机模型 H ，如图 3.3 所示。

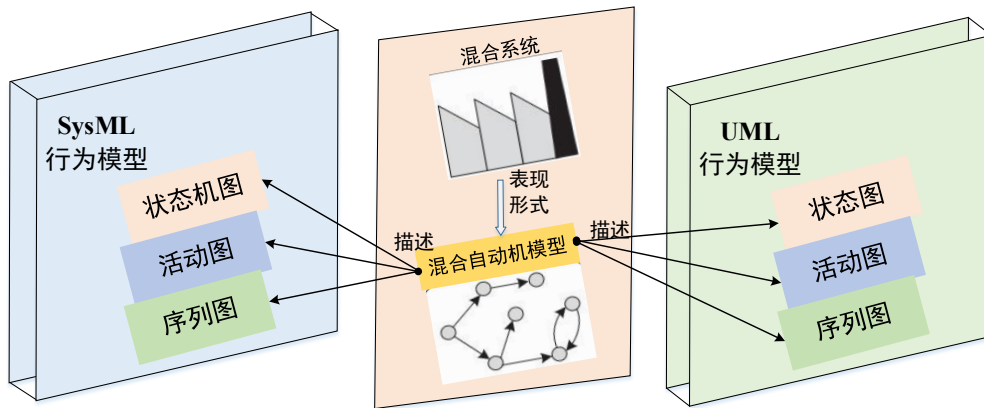


图 3.3 混合自动机模型描述系统行为模型

3.4.1 混合自动机描述系统行为的元模型

在 GOPPRR-E 元模型设计方法, G 表示图 (Graph), 是图中对象、关系、角色类型及其绑定关系的集合; O 表示对象 (Object), 是图的主要元素, 可以用属性类型描述对象的性质; R 关系 (Relationship) 表示对象之间的关联, 通过角色和对象相连接。图、对象、关系都具有自身的属性类型。

在混合自动机H中, Loc, Init, X, Inv对应于 GOPPRR-E 中对象 (Object) 的实例化对象, 即对象元模型。Edg, Flow, Jump, Lab对应于 GOPPRR-E 中关系 (Relationship) 的实例化, 即关系元模型。其关系如图 3.4 所示。

Loc表示混合自动机的位置集合; Init表示混合自动机初始位置; X和Inv表示混合自动机中与位置相关的变量集合和不变量集合。在系统行为模型统一描述过程中, 上述与对象有关的元素可映射为系统行为模型中的对象元模型。

Edg表示混合自动机的转换关系集合; Flow表示混合自动机中流动关系; Jump表示混合自动机中跳跃关系; Lab表示混合自动机中用于转换关系的同步标签集合。在系统行为模型统一描述过程中, 上述与关系有关的元素可映射为系统行为模型中的关系元模型。

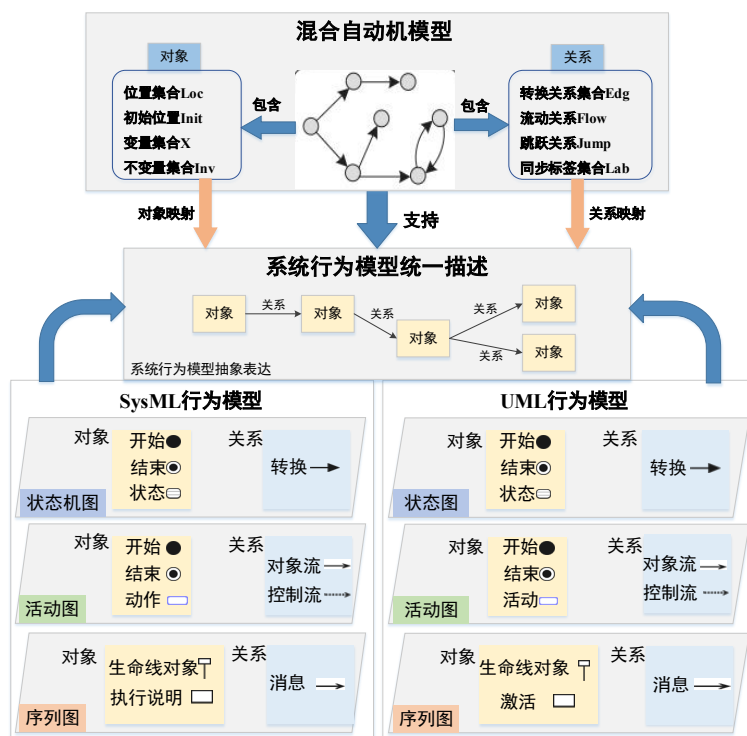


图 3.4 混合自动机描述系统行为元模型

3.5 本章小结

本章首先阐述了在混合系统中系统行为的机制，通过离散型系统行为、连续型系统行为、混合型系统行为的描述确定采用混合自动机模型可以对混合系统更好地表示和描述。然后，分别通过案例介绍混合自动机理论以及相关概念有限状态机，并详细介绍混合自动机描述系统行为的符号表达过程。最后，将混合自动机与 GOPRR-E 元建模方法相结合，分析其元素之间的对应关系，使得混合自动机支持 SysML 和 UML 系统行为模型的统一构建。

第4章 系统行为建模与仿真语言设计

4.1 引言

基于模型的系统工程是一种支持复杂系统开发的新兴技术,应用于系统工程(SE)。采用传统的 MBSE 方法,使用不同的语言(例如 SysML、UPDM、BPMN 等)描述模型。但这些语言往往使用不同的工具,从而导致各种系统工程产品视图(如需求、体系结构等)集成困难。KARMA 语言设计的主要目标是基于 GOPRR-E 元建模方法,采用统一的底层语言对不同领域的系统架构模型进行形式化表达。

为了在设计早期阶段实现对系统行为模型的仿真验证,以及对系统行为模型的统一描述,故在 KARMA 语言的基础上扩充 KARMA 语言的语法和语义,设计出系统行为建模与仿真语言,具备描述系统行为的能力。本章节以第 2、3 章节研究内容为基础,结合计算机编程语言以及混合自动机执行求解语言,研究可描述系统行为的系统行为建模与仿真语言具体语法和抽象语法设计,旨在提出面向 MBSE 领域工程师的、能够实现系统行为建模与仿真语言对系统行为的统一描述,便于后期的系统行为仿真。同时减轻系统工程师学习多种仿真语言的负担,提高研发效率。

4.2 KARMA 语言介绍

KARMA (Kombination of ARchitecture Model specificAtion) 语言^[62]是一种更高抽象层次,以面向对象方法为核心的文本式建模语言,支持描述各种系统架构视角。KARMA 语言采用 GOPRR-E 元建模方法,实现对元元模型、元模型和模型的统一描述。KARMA 语言支持多架构建模,指不同架构的建模语言,如 SysML、UML、UPDM 等。KARMA 语言是从元元模型层的描述开始,实现对元模型、模型的统一描述,具有以下几方面特点:

- 统一各建模语言的语法
- 支持各建模语言语义表达
- 统一的开发环境
- 统一的存储格式

4.3 系统行为建模与仿真语言抽象语法

本文设计的系统行为建模与仿真语言是在 KARMA 语言基础上进行扩展，是 KARMA 语言的一部分内容。KARMA 语言是一种以 GOPPRR-E 元建模方法为基础面向对象的文本式统一系统建模语言，能够形式化 GOPPRR 元元模型、元模型以及模型，从而实现对多个领域、多种特定域建模语言的统一描述。KARMA 语言分为多个模块，其中模型形式化模式是语言的基础，实现模型底层数据的统一描述，并提供元模型及模型元素的相关操作接口（API），便于后续仿真文件的编译。本章节的系统行为建模与仿真语言在 KARMA 语言基础上增加描述系统行为的功能。描述系统行为的 Automaton 图语句、Group 对象语句、State 对象语句、Transition 关系语句都以系统行为建模与仿真语言的形式描述。

系统行为建模与仿真语言按照类别可以分为四个部分：Automaton 图语句、Group 对象语句、State 对象语句、Transition 关系语句，如图 4.1 所示。其中“0..*”表示包含 0 个或多个；“0,1”表示包含 0 个或 1 个。

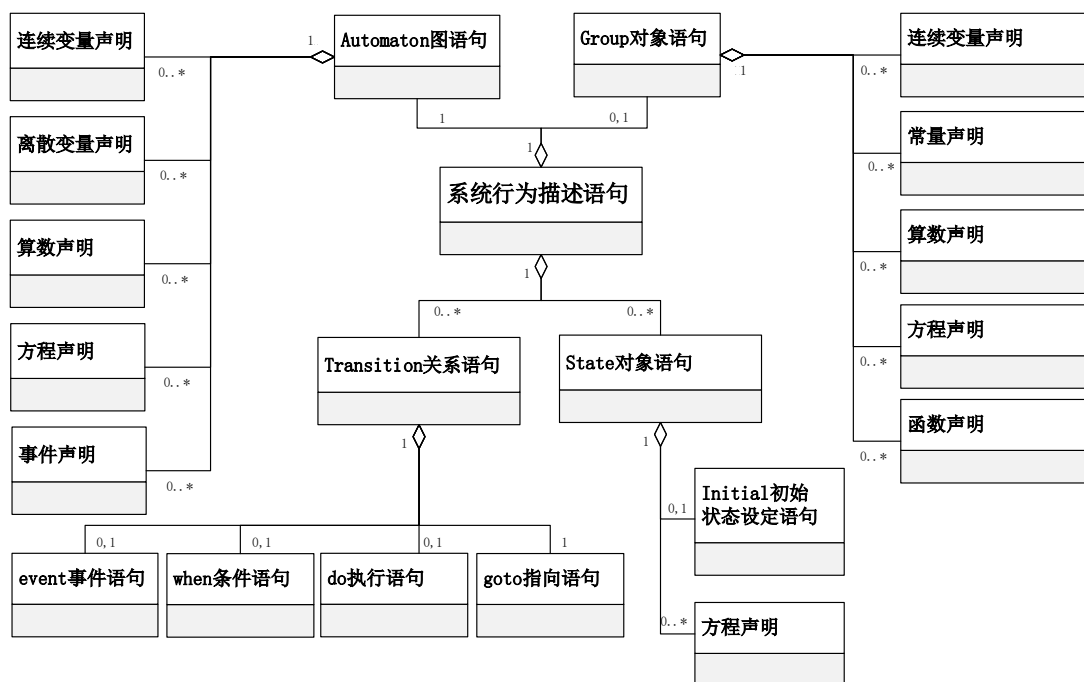


图 4.1 系统行为建模与仿真语言抽象语法

系统行为建模与仿真语言是一种强类型语言，定义变量时需要指定变量类型。作为 KARMA 语言的扩展，系统行为建模与仿真语言继承 KARMA 语言的数据类型，

并有相应扩展，数据类型如图 4.2 所示。

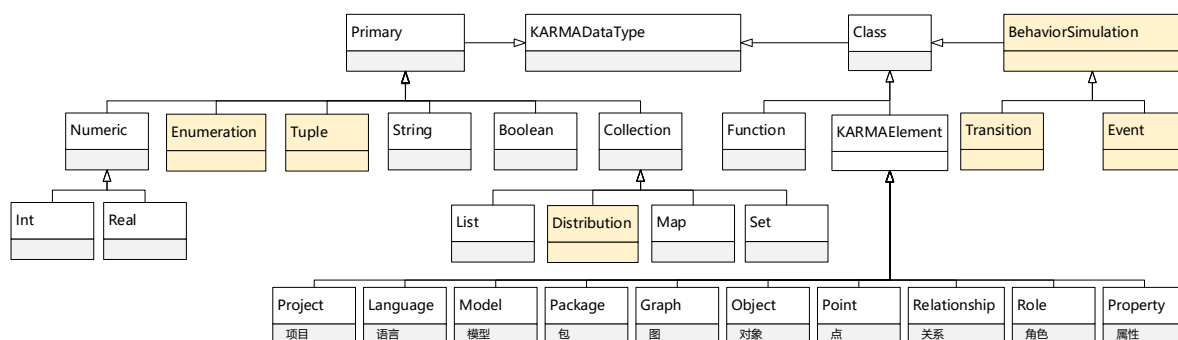


图 4.2 系统行为建模与仿真语言数据类型

4.4 系统行为建模与仿真语言具体语法

上一小节展示了系统行为建模与仿真语言抽象语法，对应完整的系统行为建模与仿真语言具体语法文件见附录 A，下文对该语言中关键概念的语句语法展开介绍。

4.4.1 系统行为建模与仿真语言关键字

如表 4.1 所示的系统行为建模与仿真语言关键字作为保留字，不能作为变量名使用。

表 4.1 系统行为建模与仿真语言关键字

关键字				
Group	State	func	algebraic	equation
continuous	constant	discrete	transition	return
event	when	do	goto	end
if	ifelse	else	enum	

4.4.2 语言具体语法设计

4.4.2.1 变量定义和赋值

系统行为建模与仿真语言的变量声明和赋值继承自 KARMA 语言，在 KARMA 语言支持的数据类型上增加 *Distribution* 类型、*Enumeration* 类型、*Tuple* 类型。支持的

数据基本数据类型有基本数据类型 *Int*、*Real*、*Boolean*、*String*，集合数据类型 *List*、*Map*、*Set*、*Distribution*。

(1) 基本变量声明和赋值的具体语法如下，*type* 表示类型，*varName* 表示变量名，*value* 表示变量值。

```
1. type varName;
2. type varName = value;
```

例如基本数据类型变量声明赋值如下：

```
1. Int i;
2. Int j = 0;
3. Real c = 2.0;
4. Boolean b = false;
5. String st = "hello, world";
```

(2) 连续变量表示该变量跟随时间连续变化，由关键字 *continuous* 表示。连续变量可以在 Automaton 图语句和 Group 对象语句中定义。连续变量声明和赋值的具体语法如下，*type* 表示类型（*Boolean* 类型和 *String* 类型除外）；*varName* 表示变量名；*value* 表示变量值；*unit* 表示单位，用于 *value* 所表示的基本物理量。*unit* 为可选项，根据需求添加。

```
1. continuous type varName; //连续变量声明
2. continuous type varName (unit = "单位")=value; //连续变量声明赋值（带有单位）
3. continuous type varName=value; //连续变量声明赋值（不带单位）
4.
5. continuous Real m; //连续变量初始化
6. continuous Real m (unit = "kg") =1300; //连续变量初始化（带有单位）
7. continuous Real m =1300; //连续变量初始化（不带单位）
```

连续变量声明赋值后，必须声明连续变量的导数，即连续变量跟随时间单位的变化情况。在下列示例中，连续变量 *t* 声明初始值为 0，其导数为 1，意味着每经过一个时间单位，*t* 的值会增加。连续变量始终具有实数值，与离散变量类似，如果未指定其初始值，则为 0。

```

1. continuous type varName (unit = "单位")=value der 变化率; //带有变化率的连续变量声明赋值 (带有单位)
2. continuous type varName= value der 变化率; //带有变化率的连续变量声明赋值 (不带单位)
3.
4. continuous Real t=0 der 1; //带有变化率的连续变量初始化
5. continuous Real t (unit = "cm") =0 der 3; //带有连续变量初始化 (带有单位)

```

也可以使用方程 (equation) 分别指定导数, 与上述第四行表述的意思一致。

```

1. continuous Real t=0; //声明指定连续变量初始化值
2. equation t'=3; //使用方程 equation 指定导数

```

(3) 常量通常用于固定不变的数据, 由关键字 *constant* 表示。常量可以在 Automaton 图语句和 Group 对象语句中定义。常量声明和赋值和连续变量的声明和赋值类似, 常量的具体语法如下, *type* 表示类型; *varName* 表示变量名; *value* 表示变量值; *unit* 表示单位, 用于 *value* 所表示的基本物理量。*unit* 为可选的, 也可以不添加, 根据需求而定。

```

1. constant type varName; //常量声明
2. constant type varName (unit = "单位")=value; //常量声明赋值 (带有单位)
3. constant type varName=value; //常量声明赋值 (不带单位)
4.
5. constant Real t; //常量初始化
6. constant Real t (unit = "kg") =1300; //常量初始化 (带有单位)
7. constant Real t=1300; //常量初始化 (不带单位)

```

(4) 离散变量表示该变量值可以按一定顺序一一列举, 由关键字 *discrete* 表示。由于离散变量用于 Transition 关系语句中的执行 do 语句, 显示的为其分配新值来更改值, 且 Transition 关系语句是包含在 KARMA 语言的 Model 部分。并且 Automaton 图语句也是存放于 KARMA 语言的 Model 部分。所以离散变量的定义只能在 Automaton 图语句中。

离散变量声明和赋值的具体语法如下, *type* 表示类型 (String 类型除外); *varName* 表示变量名; *value* 表示变量值; 离散变量除了定义 Int 类型和 Real 类型, 还可以定义

Boolean 类型，故在离散变量声明和赋值中，不用定义 *unit* 表示单位。

```
1. discrete type varName=value; //离散变量声明赋值
2.
3. discrete Real m=5; //离散变量初始化
4. discrete Boolean MH=true; //离散变量初始化
```

离散变量还用于集合类型变量声明赋值以及枚举 (Enumeration)、元组 (Tuple)。集合类型主要有 List、Set、Map、Distribution。

列表 (List) 是相同数据类型的元素组成的有序集合；集合 (Set) 是由相同数据类型元素组成的、无序的，不能存储重复元素，且每个元素的值都是唯一的；字典 (Map) 是由键值对组成的无序结合，冒号前为键，冒号后为值。Map 中键值对的顺序是无序的，如下列示例 3 所示。分布 (Distribution) 可用于创建具有特定参数的分布，用于初始化离散变量。分布分为离散分布、连续分布、常数分布：离散分布只能表示有限数量的可能值，uniform 函数分布产生有限数量，范围为左闭右开。下列示例 4 表示投掷一枚正常骰子可能会导致六个结果：1、2、3、4、5、6，且每个可能性均等。连续分布返回连续范围内的值，使用连续近似正态分布，下列示例 5 表示 pro 取值的均值为 2.46，方差为 0.0004。常数分布的目的是减少试验结果的随机性，下列示例 6 表示 D 的每次随机取值都为 3。

```
1. discrete List(Int) x = List{7, 8, 3}; //离散列表初始化
2. discrete Set(Int) x = Set{7, 8, 3}; //离散集合初始化
3. discrete Map(String,Int) age = Map{"eve":13 ,"john":20}; //声明并初始化一个 map，键为字符串类型，值为整数类型
4. discrete Distribution(Boolean) Value= Distribution{uniform(1,7)}; //离散变量离散分布实例化
5. discrete Distribution(Boolean) Pro= Distribution{normal(2.46,0.0004)}; //离散变量连续分布实例化
6. discrete Distribution(Int) D= Distribution{constant(3)}; //离散变量常数分布实例化
```

枚举 (Enumeration) 表示相关实体的集合，由关键字 “enum” 表示，例如产品类型、不同的颜色等等。下列示例 1 和 2 表示红绿灯集合。元组 (Tuple) 用来保存一系列数据，将相关字段保留在一个变量中。元组可由多个字段组成，字段的数据类型可

能不同。下列示例 4 和 5 表示一个名为 `box` 的元组变量，由整数类型的 `nr` 和实数类型的 `weight` 两个字段组成。

```
1. enum TrafficColor = {"RED","ORANGE","Green"}; //枚举实例化
2. discrete TrafficColor light = Red; //离散变量实例化
3.
4. discrete Tuple(Int nr;Real weight) box; //离散变量元组声明
5. discrete Tuple(Int nr;Real weight) box = Tuple(5;2.7); //离散变量元组初始化
```

(5) 算数变量是其值由其自身定义确定的变量，可在 Automaton 图语句和 Group 对象语句中定义，等号右边通常表示一个表达式或函数。与连续变量不同，不能为算数变量分配新值，算数变量的值随等号右边表达式或函数的计算结果而改变。算数变量声明和赋值的具体语法如下，*type* 表示类型（String 类型除外）；*varName* 表示变量名；*value* 表示变量值；由于代数变量中可定义 Boolean 类型，所以不在算数变量中添加单位 *unit*。下列示例 3 和 4 表示设定 `Q` 为 `sqrt()` 开方函数的结果，方程 `equation` 表示给定 `V` 一个初值。

```
1. algebraic Type varName =表达式\函数; //算数变量定义
2.
3. algebraic Boolean Q =sqrt(V); //算数变量初始化
4. equation V =25; //V 设定值
```

4.4.2.2 函数

Group 对象语句内部可自定义函数，封装某些操作流程，使用关键字 *func* 标识。使用函数时，通过函数名进行调用。函数可以带有参数和返回值，通过 *return* 关键字来返回值，当返回类型为 `void` 时，则不需要返回值，其语法如下：

```
1. func [returnType] funcName (parameters){
2.   funcBody
3.   return variableName
4. }
```

其中的 *parameters* 包括参数名和参数类型，一个简单的函数示例如下所示，在函数体 *funcbody* 中，可以使用多种 KARMA 支持的操作语句（赋值语句、if 语句、while

循环语句等)。

```
1. func Int signum(Real x){  
2.     if(x>1)  
3.         return 1;  
4.     elseif(x<1)  
5.         return -1;  
6.     else  
7.         return 0;  
8.     end  
9. }
```

4.4.2.3 Automaton 图语句

在一个可以用于描述系统行为的图模型中，该模型可认为是一种 Automaton 类型的模型，即混合自动机中所有描述系统行为及其变化的语句来源于该模型。Automaton 图语句是一个代码片段，存放于由 *Model* 标识一个图实例中。连续变量、方程 (Equation)、离散变量、常量、算数变量、事件 (event) 定义都位于代码片段中，其语法如下：

```
1. partial Model AC instanceof sysml_StateMachineDiagram//名称为 AC 的 SysML  
   状态机实例化  
2. discrete Boolean DC=false; //离散变量 DC 的初始值  
3. continuous Real t = 5.0 ; //连续变量 t 的初始值  
4. equation t'=3; //连续变量 t 的变化率为 3  
5. event open; //定义事件名称为 open  
6. //KARMA 语言其他代码  
7. end AC
```

4.4.2.4 Group 对象语句

Group 对象语句是一个代码片段，存放于由 *Group* 标识一个对象实例中。连续变量、方程 (equation)、常量、算数变量、函数定义都位于该代码片段中，其语法如下：

```
1. Group Object object_Action1 refer object_Action; //Group 对象语句开始  
2. //KARMA 语言其它代码  
3. func Real FN(Boolean accle, Boolean back, Real V, Real m){//函数定义
```

```

4.    Real a=0.0;
5.    Real P=160000;
6.    Real F=P/(V+0.001);
7.    Real k=3;
8.    Real Fn=k*V*V;
9.    Real F1=3500;
10.   if(accle && V<20)
11.     a =(F-Fn)/m;
12.   elseif(back)
13.     if(V>0)
14.       a =-(Fn+F1)/m;
15.     elseif(V=0 || V>=20)
16.       a =0;
17.     end
18.   else a =0;
19.   end
20.   return a; }
21.   continuous Real m=1600;
22.   continuous Real Distance=0;
23.   continuous Real V=0;
24.   algebraic Real a=Fightera(AC.Car1Acc,AC.Car1Brake,V,m);
25.   equation m'=0;
26.   equation V'=a;
27.   equation Distance'=V;
28.   end object_Action1; //Group 对象语句结束

```

4.4.2.5 State 对象语句

State 对象语句是一个代码片段，存放于由 *State* 标识一个对象实例中。初始位置（initial）定义、方程（equation）位于该代码片段中，属性值的更新通常使用 *equation* 关键字，用于修改连续变量的导数（变化率）值。

其语法如下：

```

1. State object_State1 refer object_State
2.   initial;
3.   equation m'=0;
4.   //KARMA 语言其它代码
5.   end object_State1;

```

4.4.2.6 Transition 关系语句

Transition 关系语句用于描述系统行为中对象与对象之间关系的转换。涉及跳转的条件 (*when*)、事件 (*event*)、执行操作 (*do*) 以及明确跳转 (*goto*) 的下一状态，格式为：

```
1. transition{
2.     event 事件名; //事件（可选）
3.     when (条件); //需满足的条件
4.     do 属性名=值; //属性值更新（可选），以逗号分隔多个属性
5.     goto 状态名; //明确跳转的下一状态
6. };
```

其中，当 (*when*) 满足条件且触发了事件 (*event*) 就执行变量值更新并跳转到下一状态。事件是可选项，当在 Automaton 图语句中没有定义时可省略不写；*when* 后跟的是状态跳转的前置条件，只有满足了条件才能够进行状态跳转。当有多个条件时可使用关键字“与 (&&)”或者“或 (||)”连接。对象之间的逻辑运算规则见表 4.2；*do* 执行的是变量值的更新操作，在状态跳转之前执行。当有多个变量值需要更新时，使用逗号 (,) 分隔。

表 4.2 逻辑运算规则

A	B	A&&B	!A	A B
false	false	false	true	false
false	true	false	true	true
true	false	false	false	true
true	true	true	false	true

Transition 关系语句是一个代码片段，存放于由 *Relationship* 标识一个关系实例中。初始化为：

```
1. Relationship relationship_Transition10 refer relationship_Transition
2. //KARMA 语言其它代码
3. transition{
4.     event open;
5.     when (Car1.Distance<500);
```

```
6.      do Car2BS=true;  
7.      goto Car2ShouldB;  
8. };  
9. end relationship_Transition10;
```

event 用于定义事件的名称。

初始化为:

```
1. event open;
```

4.5 本章小结

本章在 KARMA 语言的基础上, 扩展 KARMA 语言语法和语义, 设计系统行为建模与仿真语言, 以实现对系统行为的统一描述。KARMA 语言基于 GOPRR-E 元建模方法, 具有高度抽象的描述能力, 因此 KARMA 语言支持系统行为的统一形式化描述。本章设计出系统行为建模与仿真语言的抽象语法和具体语法, 并详细展开介绍, 同时方便用户对该语言进行学习。

第5章 系统行为仿真实现研究

5.1 引言

基于模型的系统工程是一种在系统开发过程中持续贯穿全生命周期的标准化建模方法。通过形式化的建模手段，以概念建模的形式支持系统需求分析，系统分析与设计以及后续的仿真和验证^[63]。由于仿真是系统验证的重要手段，在 MBSE 实施过程中，针对于复杂混合产品系统，进行仿真验证，可以帮助设计人员在设计早期发现系统设计存在的缺陷，避免将其带入到后续详细设计过程。从而可以有效的降低产品系统开发成本、提高产品系统设计质量。

混合自动机行为仿真，对建立的行为模型进行仿真，验证该行为模型的逻辑，其主要仿真形式是对所建立的行为模型（状态机图、序列图、活动图）的参数进行仿真。仿真结果主要体现为模型中的参数根据时序进行动态变化。

系统行为建模与仿真语言支持对系统行为描述，在第 4 章节重点介绍其抽象语法和具体语法。在用系统行为建模与仿真语言对系统行为进行描述时，主要包括对系统行为中一些仿真变量（连续变量、离散变量、常量、算数变量、方程）的定义，以及系统行为中的一些对象状态、函数、触发事件、时间等的描述。然后通过编写 JAVA 编译器，实现仿真文件的编译生成。最后通过将编译生成文件输入到仿真求解器中，进行系统行为仿真。得出仿真结果（系统行为中变量随时间的变化情况）后，对所描述的系统行为中的变量参数进行分析，验证其是否满足设计要求。

系统行为仿真的技术路径实现，首先在 MetaGraph 工具中通过系统行为建模与仿真语言对系统行为（状态机图、活动图以及序列图）进行描述。然后将混合自动机求解引擎集成到 MetaGraph 工具中。通过已定义的编译规则，将描述后的模型编译为仿真求解引擎所需要仿真文件，支持行为仿真求解器的仿真求解。最后将仿真结果（主要体现为系统行为中变量参数随时间的变化情况）显示在工具前端，从而实现系统行为仿真。

5.2 MetaGraph 工具介绍

MetaGraph (<http://www.zkhoneycomb.com/>) 是支持多架构统一建模语言 KARMA 的建模工具。KARMA 语言是一种可读文本式语言，多架构建模指在 MetaGraph 工具

中采用一种建模语言及技术实现多种架构描述及表达的方法。MetaGraph 工具启动页面如图 5.1 所示。



图 5.1 MetaGraph 工具启动页面

MetaGraph 工具的主要功能包括：

- 实现不同通用建模语言及框架的模型库开发及建模，如 UML，SysML，BPMN，Capella，UPDM 等。
- 实现特定域建模语言基于框架的模型库开发及建模，如 EAST-ADL 等。
- 可定制的架构驱动，即工具中架构模型之间的自动传递及转化^[64]。
- 可定制的代码生成，即工具中架构模型向他代码及数据的自动传递及转化。
- 实现基于 KARMA 语言的指标分析及验证。
- 实现基于 KARMA 语言的需求验证。
- 支持系统行为的动态仿真。
- 支持模型与本体的信息的相互转化^[65]。

MetaGraph 工具的核心理念是基于“文本可读的形式化语言”，即 KARMA 语言，对复杂装备的需求、功能、逻辑、架构等系统工程视点进行建模，并通过仿真分析及测试来验证指标的满足性及需求验证等。借助 KARMA 不仅可以在系统开发的早期阶段形式化其需求、功能、逻辑、架构等视点，还可以在系统方案初步确认前的概念设计阶段检查系统设计是否满足需求规范，从而极大地降低产品开发的成本与风险。

5.3 系统行为仿真的半自动化流程设计

5.3.1 系统行为仿真引擎介绍及工具集成

5.3.1.1 混合自动机求解引擎

混合自动机求解引擎是通过带有代数微分方程理论的混合自动机理论来模拟系统的计算和物理行为^[66]。采用混合自动机求解引擎可以实现指定离散事件、定时系统以及混合系统(包含离散事件和定时系统)行为仿真,并支持可伸缩向量图形(Scalable Vector Graphics, SVG)可视化显示。

5.3.1.2 混合自动机求解引擎集成到 MetaGraph 中

将混合自动机求解引擎集成到建模工具 MetaGraph 中,通过按钮“仿真执行”进行启动,弹出“行为仿真选项”配置框。用户只需要在配置框中进行仿真文件选择、仿真时间设定、仿真变量显示等操作,最后点击“确认”等待仿真结果显示。运行界面如图 5.2 所示。

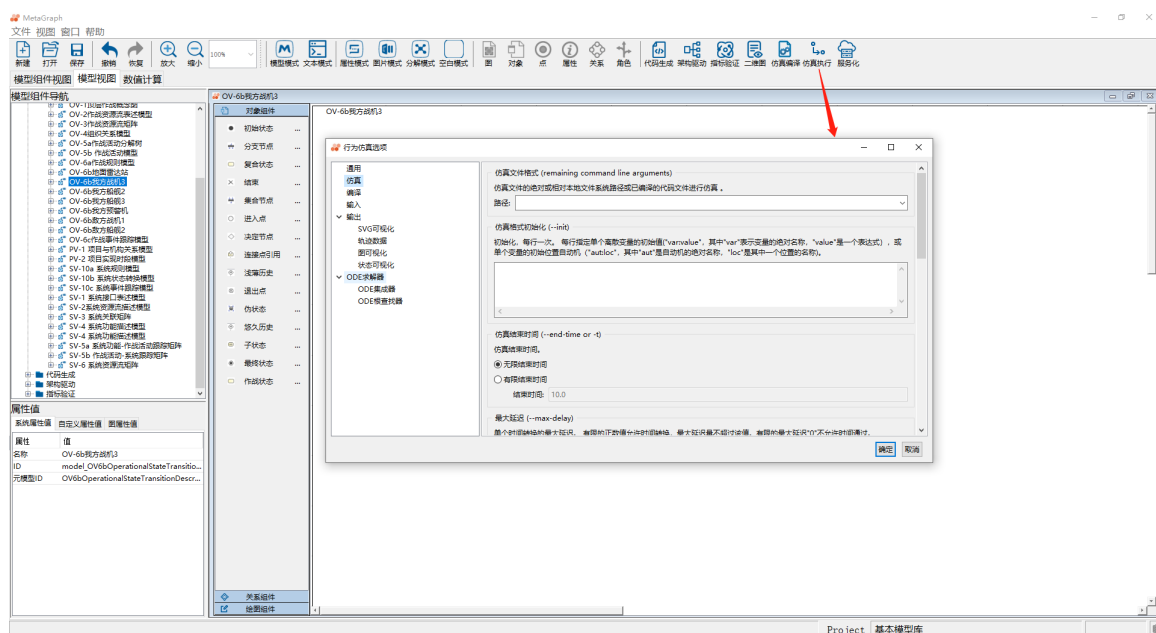


图 5.2 仿真执行启动界面

5.3.2 仿真文件编译生成

5.3.2.1 系统行为建模与仿真语言执行

ANTLR 是一款开源的语法分析器生成工具，可以根据语法规则文件生成对应的语法分析器。系统行为建模与仿真语言在实现时需要通过工具 ANTLR 构建语法解析器以及通过 JAVA 构建语法解释器，最终生成用于仿真的.sim 文件。

ANTLR 是一个开源模型的语法解析器，可以根据构造好的语法文件对输入的语言进行解析，自动生成抽象语法树并生成语法树遍历框架，提供针对语法树的操作接口。在具体应用中，ANTLR 对输入的系统行为建模与仿真语言进行读取，根据构造的系统行为建模与仿真语言语法规则，构建抽象语法树。通过 JAVA 以及系统行为建模与仿真语言与.sim 文件之间的转化规则，构建语法解释器，使得输入的系统行为建模与仿真语言最终转化为用于仿真的.sim 文件。具体的执行实现过程如图 5.3 所示，在 MetaGraph 工具中仿真编译界面如图 5.4 所示。

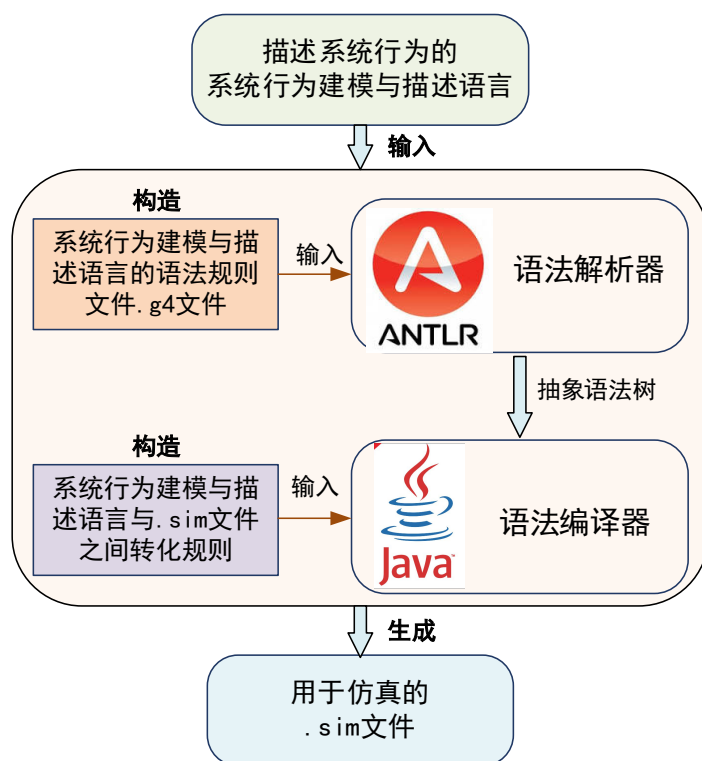


图 5.3 系统行为建模与仿真语言执行过程

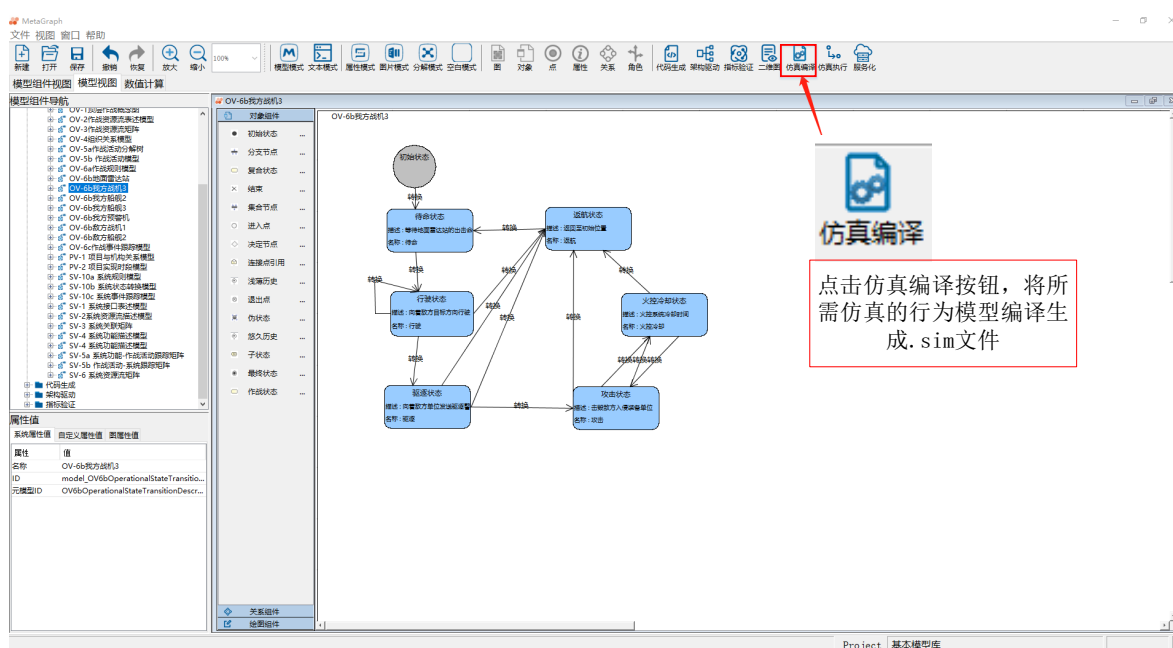


图 5.4 仿真文件编译界面

5.3.3 系统行为建模与仿真流程设计

如图 5.5 所示，系统行为建模与仿真流程设计主要包括 MetaGraph 工具前端与 MetaGraph 工具后端两部分内容。在工具前端，设计人员主要进行模型建立、系统行为建模与仿真语言描述系统行为以及对后续仿真结果进行分析。工具后端的主要工作包括仿真文件 .sim 文件的编译生成以及导入到混合自动机仿真求解引擎中进行仿真。

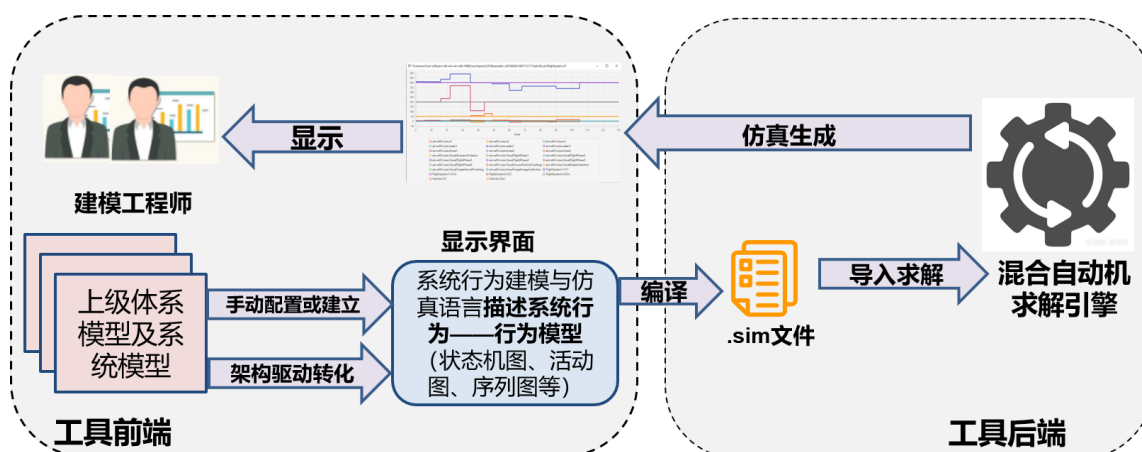


图 5.5 系统行为建模与仿真流程

5.4 仿真编译文件映射关系研究

5.4.1 多系统行为模型联合仿真

一方面，在进行复杂产品系统行为描述及仿真时，存在一种需要将多个系统行为模型联合仿真的场景。另一方面，由于混合自动机求解引擎每次执行仿真，只能输入一个.sim 文件。因此，在进行多个行为模型联合仿真时，需要将每幅图编译生成的.sim 文件进行合并，生成一个.sim 文件。在工具操作进行仿真文件编译时，需要设计人员将编译生成的多个.sim 文件进行手动选择合并，形成一个完整的.sim 文件，合并过程如图 5.6 所示。在 MetaGraph 工具中，联合仿真文件选择生成界面如图 5.7 所示

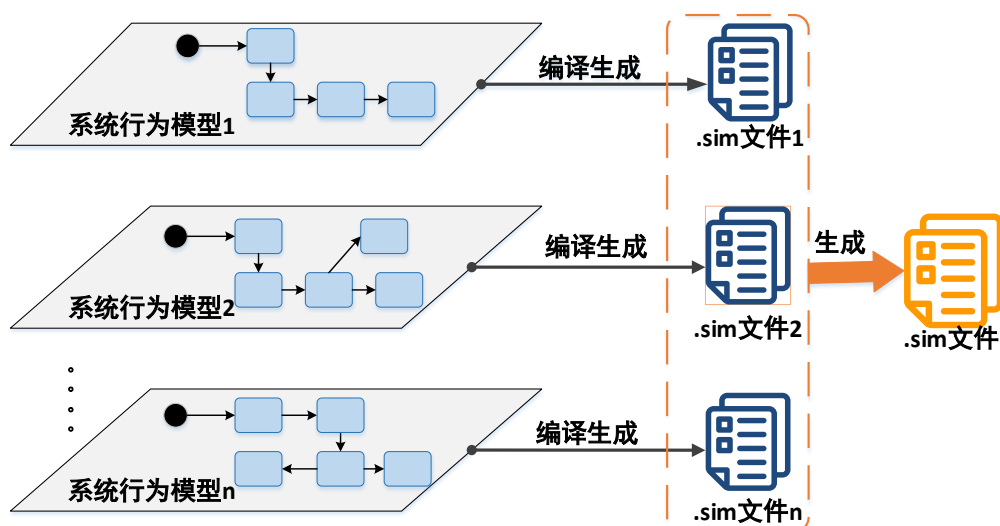


图 5.6 多个.sim 文件编译并合并

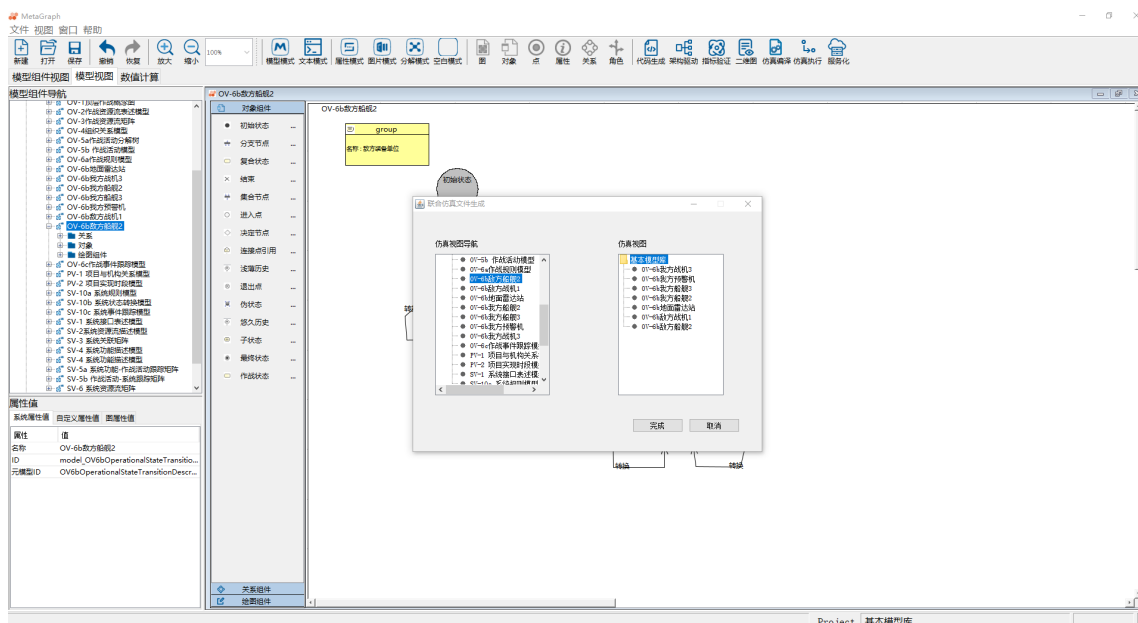


图 5.7 联合仿真文件生成选择界面

5.4.2 复合状态仿真

在进行状态机图建模过程中，存在一种状态为复合状态，如图 5.8 所示红色虚线部分。

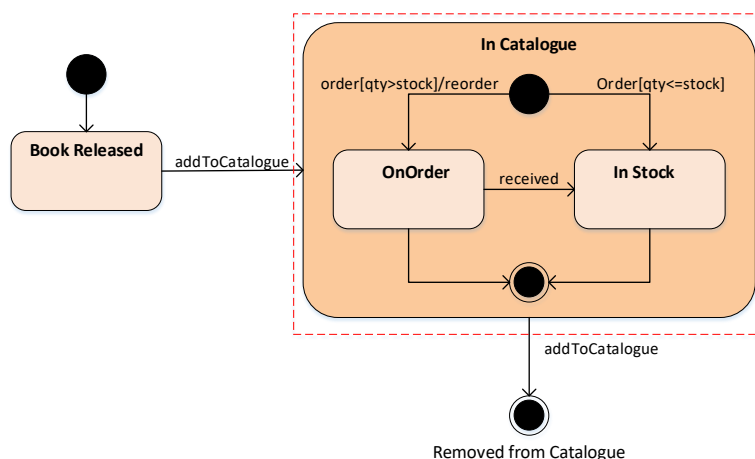


图 5.8 复合状态

该复合状态内部嵌套一个状态机图。针对这种场景的行为模型仿真，在仿真文件编译过程中需要考虑将嵌套的状态机图也编译生成一个仿真文件。解决方案如图 5.9 所示，将该复合状态通过建模工具 MetaGraph 和 KARMA 语言所支持的分解功能，将

该复合状态分解为一张“图”，即一个新的行为模型。首先将带有复合状态的状态机图简化为简单状态，可将该状态图编译生成一个.sim 文件。然后将简化后的复合状态进行分解为一个不含有复合状态的状态机图，同样编译生成一个.sim 文件。最后将两个编译生成的文件合并为一个仿真文件，从而实现带有复合状态的状态机图编译以及后续的仿真。

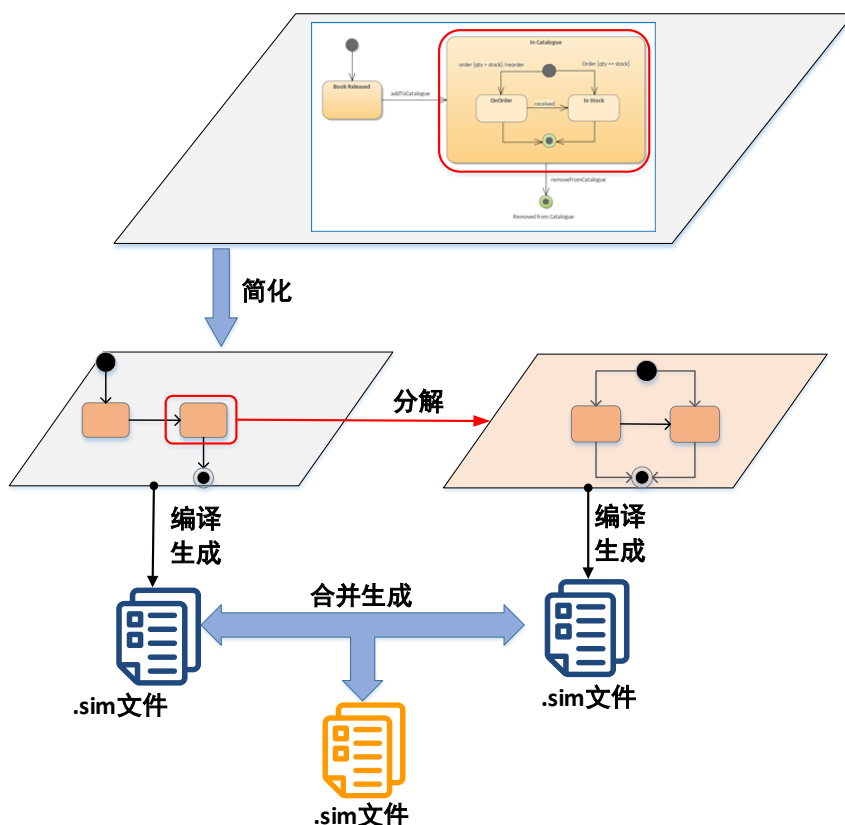


图 5.9 复合状态分解

5.5 本章小结

本章主要介绍了系统行为仿真的流程设计，实现系统行为建模与仿真语言的执行功能。首先介绍了 MetaGraph 工具，分析了采用混合自动机求解引擎的必要性，在此基础上研究设计系统行为仿真的流程。最后研究多系统行为模型联合仿真和复合状态仿真两种形式与仿真文件之间的映射关系，以满足不同场景的设计仿真需求。

第6章 案例实现及验证

6.1 引言

本文主要针对复杂产品系统进行行为建模与仿真语言设计，并对该语言的实现过程进行研究。因此，在案例实现及验证过程中，首先根据系统行为的三种描述方式分别进行了 SysML 状态机图仿真、活动图仿真、序列图仿真。这三个仿真案例描述的是三个复杂产品系统中的部分场景。然后进行了探测拦截多状态机图联合仿真案例实现，验证了复杂产品系统中多状态图联合仿真的可行性。最后，对目前仿真功能实现进行总结。

6.2 多场景（状态机图、活动图、序列图）仿真案例

6.2.1 无人车行驶状态机图仿真案例

案例背景：无人车具有行驶、路面检测、图像收集处理的功能，如图 6.1 所示。已知该无人车安装有导航系统、探测雷达、图像收集处理系统、以及巡航控制雷达。车辆行驶道路存在泥泞道路。该无人车可以自动探测道路情况，当探测到泥泞道路时，无人车减速行驶，确保车辆行驶安全。无人车在加速行驶过程中，会打开图像收集系统，并拍下加速行驶过程中的沿途照片，自动保存处理。无人车在加速过程中，加速度为 $3m/s^2$ 、功率为 $64kW$ ，且当速度高于 $30m/s$ 时，无人车停止加速；减速过程中加速度为 $-1m/s^2$ 、功率为 $36kW$ ，且当速度低于 $21m/s$ 时，无人车停止减速；匀速行驶过程中，功率为 $40kW$ 。无人车初始运行速度为 $0m/s$ 。

对无人车在该路段的行驶过程进行设计：当 $5s$ 时，导航系统开启，无人车的发动机启动。 $8s$ 车辆开始迅速行驶，且车上的巡航控制雷达开启。匀速行驶 $10s$ 后，车辆开始加速。并进行后续路面探测、图像收集处理，行驶至 $35s$ ，无人车导航系统和巡航控制雷达关系，车辆此刻处于待命状态。无人车在该路程中的行驶时间设置为 $35s$ ，短期行驶以及后续行为仿真的目的是测试无人车在该路段行驶过程中的性能，主要包括耗电量、行驶路程随时间变化情况。

仿真结果指标： $35s$ 后若无人车行驶路程大于等于 $150m$ ，且耗电量小于等于

$0.2kW \cdot h$ ，则认为该设计合理。



图 6.1 无人车检测、图像收集、行驶过程

(1) 模型建立：建立 SysML 无人车行驶状态机图，如图 6.2 所示。对无人车行驶过程中行为进行描述。

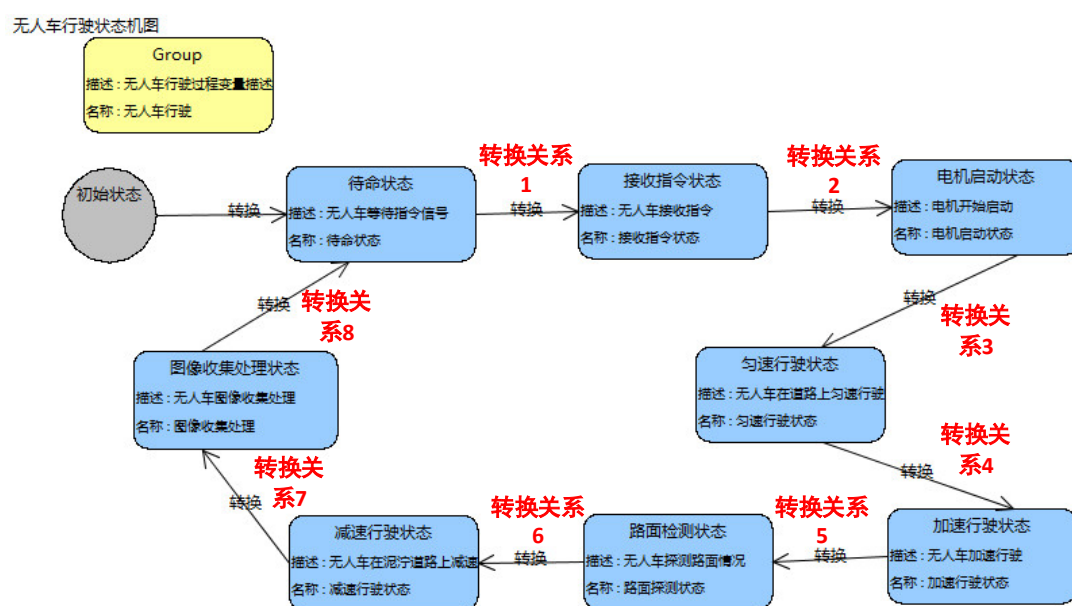


图 6.2 无人车行驶状态机图

(2) 系统行为建模与仿真语言描述系统行为：

1) Automaton 图语句：

```

1. discrete Boolean NavigationSystem=false; //导航系统初始为关闭状态
2. discrete Boolean DetectionRadar=false; //探测雷达初始为关闭状态
3. discrete Boolean accelerate=false; //加速初始为关闭状态
4. discrete Boolean decelerate=false; //减速初始为关闭状态
5. discrete Boolean ImageCollectionSystem=false; //图像收集系统初始为关闭状态
6. discrete Boolean CruiseControlRadar=false; //巡航控制雷达初始为关闭状态
7. event StartSignal; //开始信号事件
8. event muddy; //泥泞道路事件

```

2) Group 对象语句:

```

1. func Real AcceleratedSpeed(Boolean accelerate, Boolean decelerate){ //加速度 a 的
计算
2.     Real a = 0.0;
3.     if(accelerate)
4.         a=3.0;
5.     elseif(decelerate)
6.         a=-1.0;
7.     else
8.         a=0;
9.     end
10.    return a;
11. }
12. func Real Power(Boolean accelerate, Boolean decelerate){ //功率 P 的计算
13.     Real P = 0.0;
14.     if(accelerate)
15.         P=64;
16.     elseif(decelerate)
17.         P=36;
18.     else
19.         P=40;
20.     end
21.    return P;
22. }
23. continuous Real X=0.0; //连续变量-行驶距离
24. continuous Real V=0.0; //连续变量-行驶速度
25. continuous Real Q=0.0; //连续变量-耗电量
26. //加速度 a 的计算
27. algebraic Real
a=AcceleratedSpeed(UnmannedVehicle.accelerate, UnmannedVehicle.decelerate);
28. //功率 P 的计算

```

```

29.    algebraic Real
P=Power(UnmannedVehicle.accelerate,UnmannedVehicle.decelerate);
30.    equation V'=a; //速度 V 的导数为 a
31.    equation X'=V; //行驶路程 x 的导数为 V
32.    equation Q'=P; //耗电量 Q 的导数为 P

```

3) State 对象语句:

```

1.    initial; //初始状态设定, 为图 6.2 中的待命状态。

```

4) Transition 关系语句:

```

1. //转换关系 1
2. transition{
3.     event StartSignal; //启动事件触发, 车辆开始启动
4.     goto ReceiveInstruction; //进入下一状态-接收指令状态
5. };
6. //转换关系 2
7. transition{
8.     when (when time>=5); //运行 5 秒后
9.     do NavigationSystem=true; //导航系统开启
10.    goto MotorStart; //进入下一状态-电机启动状态
11. };
12. //转换关系 3
13. transition{
14.    when (time>=8); //运行 8 秒后
15.    do CruiseControlRadar=true; //巡航控制雷达开启
16.    goto ConstantSpeed1; //进入下一状态-匀速行驶状态
17. };
18. //转换关系 4
19. transition{
20.    when (when time>=18); //运行 18 秒后
21.    do accelerate=true; //车辆开始加速
22.    goto Accelerate; //进入下一状态-加速行驶状态
23. };
24. //转换关系 5
25. transition{
26.    when (UnmannedVehicleDriving.V>=30); //当运行速度大于 30 时
27.    do accelerate=false,DetectionRadar=true; //车辆停止加速, 探测雷达开启
28.    goto RoadTest; //进入下一状态-路面检测状态

```

```
29. };
30. //转换关系 6
31. transition{
32.     event muddy; //泥泞事件触发
33.     do decelerate=true; //车辆开始减速
34.     goto Decelerate; //进入下一状态-减速行驶状态
35. };
36. //转换关系 7
37. transition{
38.     when (UnmannedVehicleDriving.V<=21); //当运行速度小于 21 时
39.     do decelerate=false,ImageCollectionSystem=true; //车辆停止减速，图像收集系
    统开启
40.     goto ImageCollection; //进入下一状态-图像收集处理状态
41. };
42. //转换关系 8
43. transition{
44.     when (time>=35); //运行时间 35 后
45. //导航系统关闭，巡航控制雷达关闭
46.     do NavigationSystem:=false,CruiseControlRadar:=false;
47.     goto StandbyState; //进入下一状态-待命状态
48. };
```

(3) 仿真文件编译及仿真：

通过点击 MetaGraph 工具中“仿真编译”按钮，则自动生成仿真文件。后点击“行为仿真”，在弹出的仿真配置页面中，输入仿真文件，设置仿真时间为 35（秒）。点击“确认”后，会得到仿真结果，如图 6.3 所示。根据仿真结果可判断出在 35s 时，行驶路程 X 大于 300m；耗电量 Q 小于 $0.2kW \cdot h$ ，对应图中纵轴数值为 1800，满足仿真结果指标，判断出该设计过程合理。

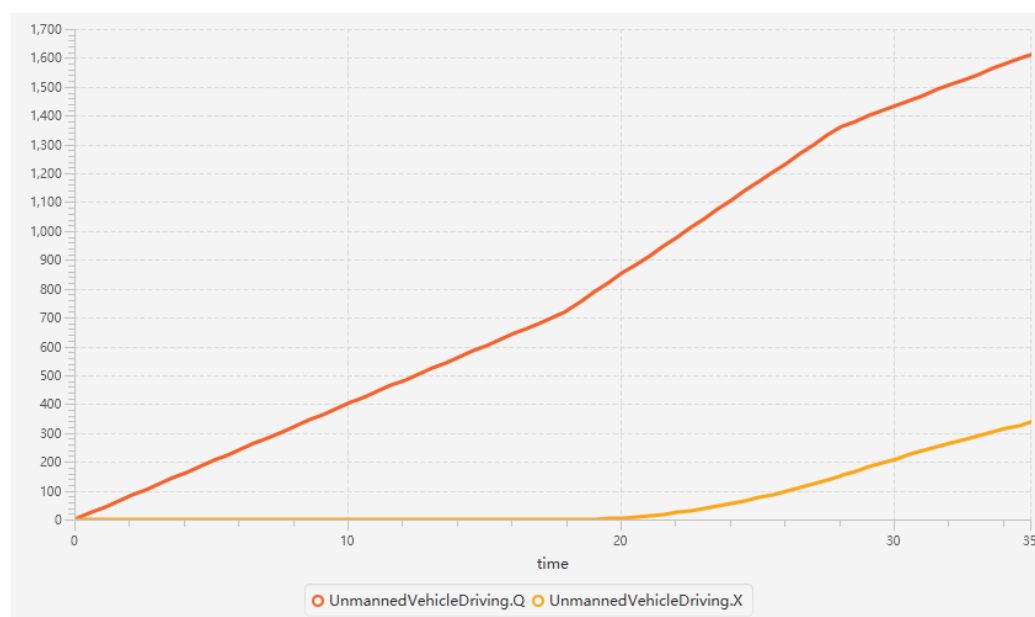


图 6.3 无人车状态机图仿真结果

6.2.2 无人车行驶活动图仿真案例

案例背景：无人车具有行驶、路面检测、避障、巡航功能，如图 6.4 所示。已知该无人车安装有导航系统、电池组、防御开关、探测雷达以及巡航控制雷达。车辆行驶道路存在泥泞道路、平坦道路、一般道路三种路况，该无人车可以自动探测道路情况。当探测到泥泞道路时，无人车减速行驶，确保车辆行驶安全；当探测到平坦道路时，无人车加速行驶；车辆行驶到一般道路时，无人车也处于加速行驶状态、此刻加速度小于在平坦道路上的加速度。无人车在行驶过程中，如果遇见障碍物，后退行驶 85m。无人车在平坦道路加速过程中，加速度为 $0.6m/s^2$ ，且当速度高于 $30m/s$ 时，无人车停止加速；减速过程中加速度为 $-0.4m/s^2$ ，且当速度低于 $21m/s$ 时，无人车停止减速；一般道路行驶过程中，加速度为 $0.1m/s^2$ 。无人车初始运行速度为 $0m/s$ 。

对无人车在该路段的行驶过程进行设计：无人车在该路程中的行驶时间设置为 85s，短期行驶以及后续行为仿真的目的是测试无人车在该路段行驶过程中的性能，包括加速度、速度、行驶路程的随时间变化情况。当车辆启动后，行驶 3s 后，会进入探测区域。10s 车辆车上的探测雷达开启，进行路面检测。行驶路途途中会探测路况进行加速、减速，遇见障碍物则后退 85m。行驶至 85s，无人车防御开关和巡航控制雷达关闭，车辆此刻处于路况探测状态。

仿真结果指标：85s后若无人车行驶路程大于等于1300m，则认为该设计合理。



图 6.4 无人车检测、避障、巡航行驶过程

(1) 模型建立：无人车行驶过程包括对行驶道路状况探测，遇见障碍物后退以及防御、巡航功能。对其行为进行描述，建立的 SysML 活动图如图 6.5 所示。

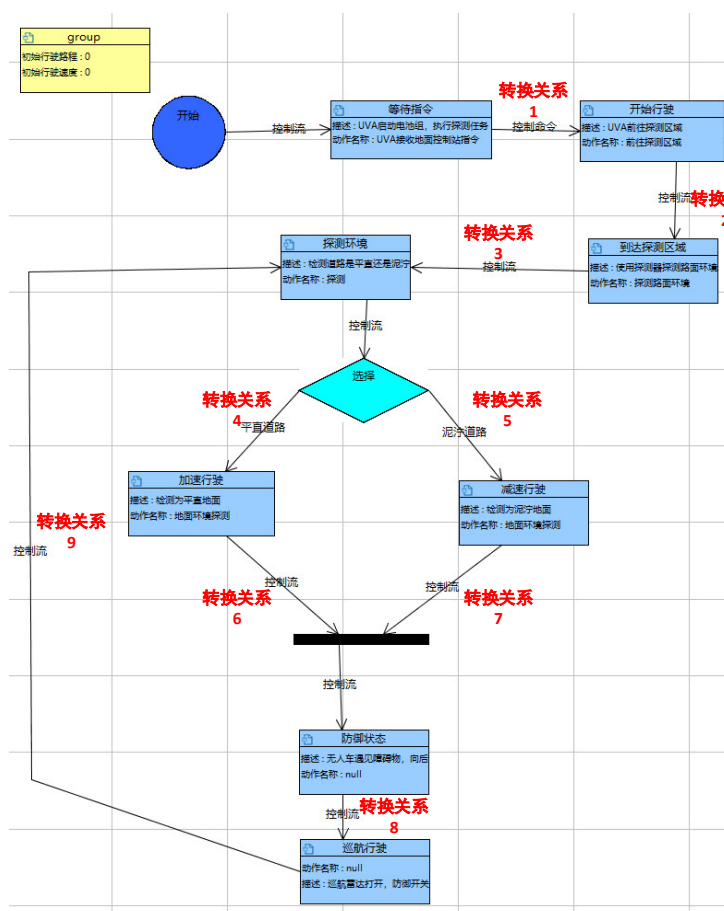


图 6.5 无人车探测过程活动图

(2) 系统行为建模与仿真语言描述系统行为:

1) Automaton 图语句:

```

1. discrete Int count=0; //统计值
2. discrete Boolean MuddyRoad=false; //泥泞道路状况设置
3. discrete Boolean FlatRoad=false; //平坦道路状况设置
4. discrete Boolean Obstacle=false; //障碍物设置
5. discrete Boolean NavigationSystem=false; //巡航系统
6. discrete Boolean DetectionRadar=false; //探测雷达
7. discrete Boolean BatteryPack=false; //电池组
8. discrete Boolean DefensiveSwitch=false; //防御开关
9. discrete Boolean CruiseRadar=false; //巡航雷达
10. event ControlInstruction; //控制指令事件
11. event muddy; //泥泞道路事件
12. event flat; //平坦道路事件
13. algebraic Real L=RegionalExploration.X+RegionalExploration.d; //总行程计算

```

2) Group 对象语句:

```

1. func Real UAV_a(Boolean MuddyRoad, Boolean FlatRoad){ //加速度 a 计算
2.     Real a = 0.0;
3.     if(MuddyRoad)
4.         a=-0.4;
5.     elseif(FlatRoad)
6.         a=0.6;
7.     else
8.         a=0.1;
9.     end
10.    return a;
11. }
12. func Real UAV_d(Boolean Obstacle){ //后退距离 d 计算
13.    Real d = 0.0;
14.    if(Obstacle=true)
15.        d=-85;
16.    else
17.        d=0.0;
18.    end
19.    return d;
20. }
21. continuous Real X=0.0; //连续变量-行驶距离

```

```

22.    continuous Real V=0.0; //连续变量-行驶速度
23.    algebraic Real a=UAV_a(UAV.MuddyRoad,UAV.FlatRoad); //加速度 a
24.    algebraic Real d=UAV_d(UAV.Obstacle); //后退距离 d
25.    equation V'=a; //速度 V 的导数为 a
26.    equation X'=V; //行驶距离 X 的导数为 V

```

3) State 对象语句:

```

1.    initial; //初始状态设定

```

4) Transition 关系语句:

```

1.    //转换关系 1
2.    transition{
3.        event ControlInstruction; //控制指令事件触发
4.        do BatteryPack=true,NavigationSystem=true; //无人车电池组开启, 导航系统开启
5.        goto StartExercising; //进入下一状态-开始行驶状态
6.    };
7.    //转换关系 2
8.    transition{
9.        when (time>=3); //3 秒后
10.       goto ReachDetectionArea; //进入下一状态-到达探测区域状态
11.    };
12.   //转换关系 3
13.   transition{
14.       when (time>=10); //10 秒后
15.       do DetectionRadar =true; //探测雷达开启
16.       goto ProbeEnvironment; //进入下一状态-探测环境状态
17.    };
18.   //转换关系 4
19.   transition{
20.       event muddy; //泥泞道路事件触发
21.       do MuddyRoad=true; //泥泞道路行驶
22.       goto DecelerationCondition; //进入下一状态-减速行驶状态
23.    }
24.   //转换关系 5
25.   transition{
26.       event flat; //平坦道路事件触发
27.       do FlatRoad=true; //平坦道路行驶
28.       goto AccelerationCondition; //进入下一状态-加速行驶状态

```

```

29. };
30. //转换关系 6
31. transition{
32.     when (RegionalExploration.V <=30); //无人车行驶速度小于 30
33.     do MuddyRoad=false,DefensiveSwitch=true; //行驶出泥泞道路, 防御开关开启
34.     goto DefensiveState; //进入下一状态-防御状态
35. };
36. //转换关系 7
37. transition{
38.     when (RegionalExploration.V >=25); //无人车行驶速度大于 25
39.     do FlatRoad=false,DefensiveSwitch=true; //行驶出能平坦道路, 防御开关开启
40.     goto DefensiveState; //进入下一状态-防御状态
41. };
42. //转换关系 8
43. transition{
44.     when (time>=60); //60 秒后
45.     do Obstacle=true; //遇见障碍物
46.     goto CruiseState; //进入下一状态-巡航行驶状态
47. }
48. //转换关系 9
49. transition{
50.     when (time>=85); //85 秒后
51.     do Obstacle=false,CruiseRadar=false,DefensiveSwitch=false; //离开障碍物, 巡航雷达关闭, 防御开关关闭
52.     goto ProbeEnvironment; //进入下一状态-探测环境状态
53. }

```

(3) 仿真文件编译及仿真:

系统行为建模与仿真语言对系统行为描述后, 通过点击 MetaGraph 工具中“仿真编译”按钮, 则自动生成仿真文件。后点击“行为仿真”, 在弹出的仿真配置页面中, 输入仿真文件, 设置仿真时间为 90s。点击“确认”后, 会得到仿真结果, 如图 6.6 所示。根据仿真结果可判断出在 85s 时, 行驶路程 X 大于 1300m。满足仿真结果指标, 可判断出该设计过程合理。

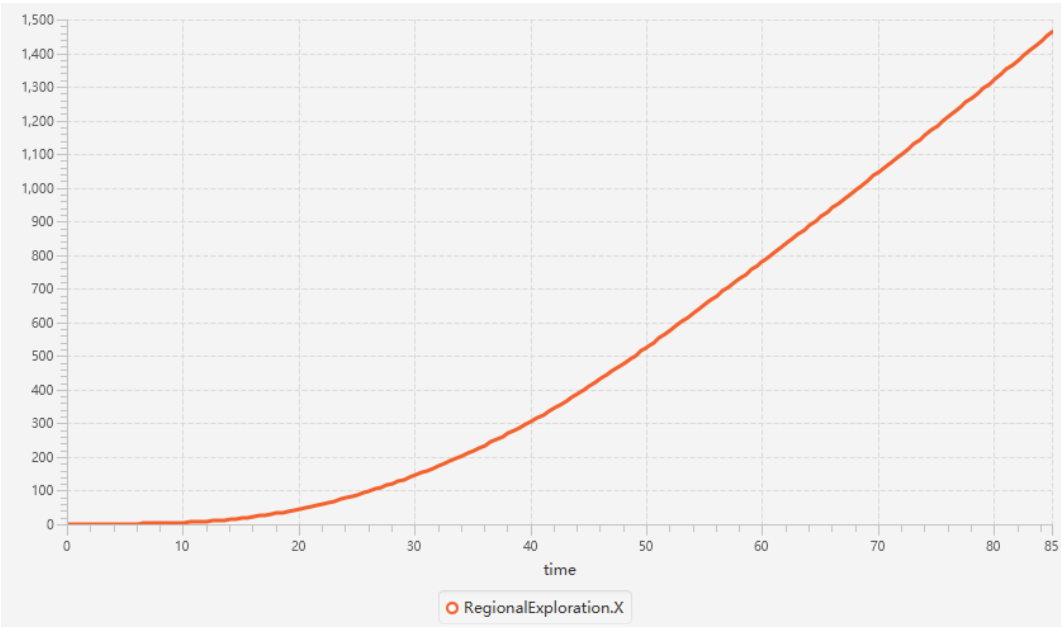


图 6.6 无人车探测活动图仿真结果

6.2.3 机舱影音播放序列图仿真案例

案例背景：某飞机机舱座位电视具有播放音乐和播放视频的功能。乘客、乘务员与影音播放系统之间的交互如图 6.7 所示。播放视频和音乐消耗电量和流量值如表 6.1 所示，总耗电量为 $E(W \cdot min)$ ，总消耗流量为 $Y(MB)$ 。

表 6.1 影音播放系统音乐、视频统计表

影音	流量 Q (MB)	功率 P (W)	播放时间 (min)
音乐	Q1=30	P1=20	time1=15
视频	Q2=50	P2=60	time2=20

对机舱座位影音系统播放过程进行设计：乘客首先点播音乐，发送给播放系统信号。播放系统会接收信号，并解压相应的数据包，将音频传送给乘客，播放 15min 音乐。中间会有乘务员向播放系统发送音频公告，乘客接收音乐停止，接收乘务员 1min 的广播。最后乘客会点击视频播放，播放系统处理后，播放 20min 视频。

仿真结果指标：50min 后，若机舱座位影音系统总消耗流量 Y 小于 3000MB 和总耗电量 E 小于 $2kW \cdot h$ ，则认为符合设计需求。



图 6.7 影音播放系统与乘客、乘务员之间的交互

通过序列图进行仿真的目的是对乘客、播放系统以及乘务员进行行为描述及验证是否具有预期的行为。由于序列图与状态机图都是根据系统行为进行建模，具有一定的共性。其中序列图中的角色功能和系统功能可以看成为状态机图中的状态，交互可以看成为状态机图中的状态转换。

(1) 模型建立：交互场景-音频视频娱乐方案序列图，如图 6.8 所示，描述了乘客、乘务员以及飞机客舱中影音播放系统之间的交互行为。

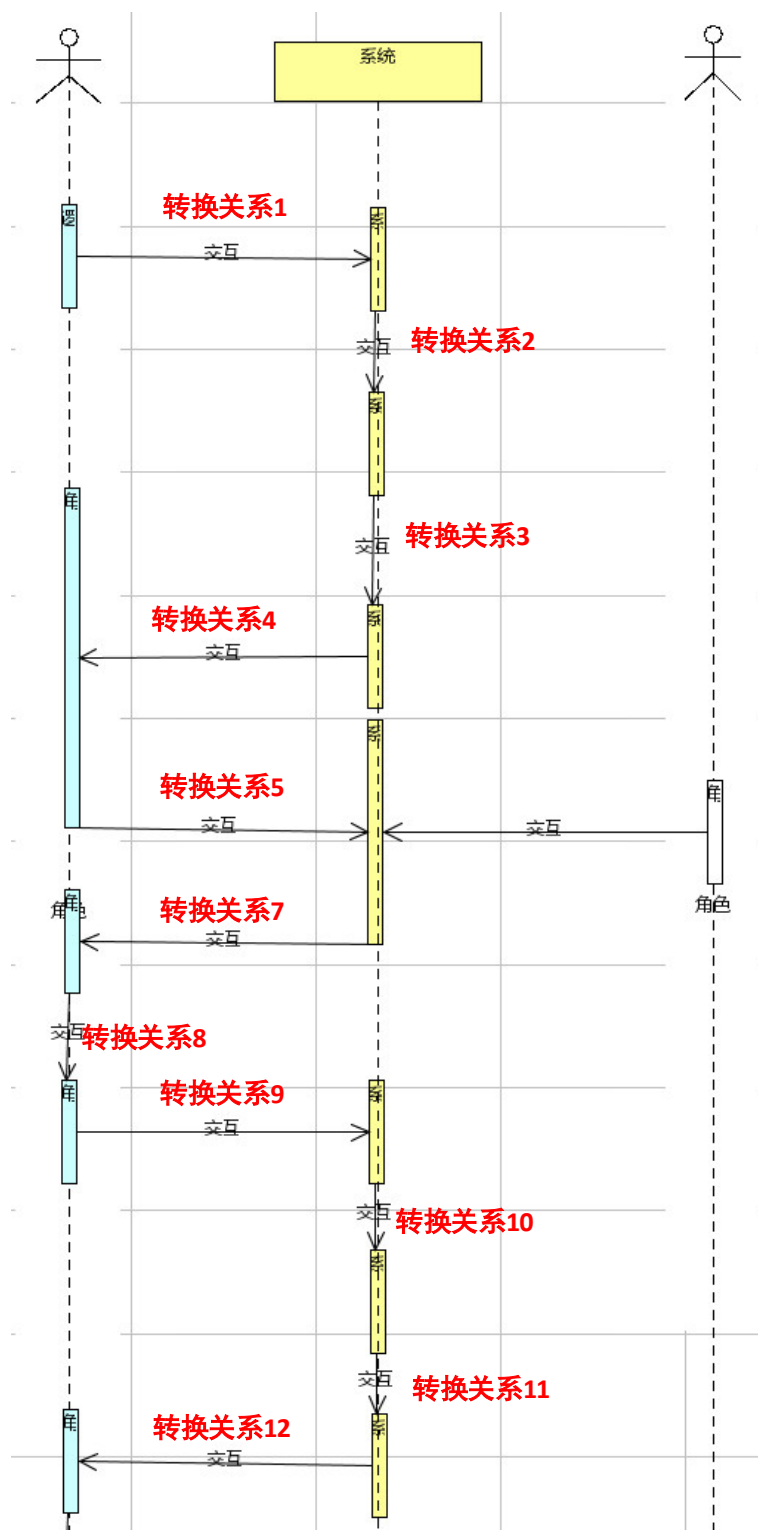


图 6.8 交互场景-音频视频娱乐方案序列图

(2) 系统行为建模与仿真语言描述系统行为:

1) Automaton 图语句:

```

1. algebraic Real time1; //播放音乐的时间
2. algebraic Real time2; //播放视频的时间
3. discrete Boolean AudioSystem=false; //音频系统刚开始为关闭状态
4. discrete Boolean VideoSystem=false; //视频系统刚开始为关闭状态
5. discrete Boolean SendAudioAnnouncement=false; //播放广播关闭

```

2) Group 对象语句:

```

1. algebraic Int Q1=30; //播放音频流量
2. algebraic Int Q2=50; //播放视频流量
3. algebraic Int P1=20; //播放音频功率
4. algebraic Int P2=60; //播放视频功率
5. continuous Y=0; //连续变量播放系统总流量计算
6. algebraic Real
Q=(EntertainmentSystem.time1*Q1+EntertainmentSystem.time2*Q2)*0.1;
7. continuous E=0; //连续变量播放系统总电量计算
8. algebraic Real
E=(EntertainmentSystem.time1*P1+EntertainmentSystem.time2*P2)*0.1;
9. equation Y'=Q;
10. equation E'=P;

```

3) State 对象语句:

```

1. //对象(OnDemandMusic)
2. initial;
3. equation time1=0;
4. equation time2=0;
5. //对象(ReceiptAudioSignal)
6. equation time1=0;
7. equation time2=0;
8. //对象(ExtractAudioPacket)
9. equation time1=0;
10. equation time2=0;
11. //对象(SendAudio)
12. equation time1=0;
13. equation time2=0;
14. //对象(ReceiptMusic)
15. equation time1=15;

```

```
16. equation time2=0;
17. //对象(CurrentServiceInterruption)
18. equation time1=0.5;
19. equation time2=0;
20. //对象(ReceiptAudioBroadcast)
21. equation time1=1;
22. equation time2=0;
23. //对象(OnDemandMovie)
24. equation time1=0;
25. equation time2=0;
26. //对象(ReceiptVideoPacket)
27. equation time1=0;
28. equation time2=0;
29. //对象(Sendvideo)
30. equation time1=0;
31. equation time2=0;
32. //对象(WatchMovie)
33. equation time1=1;
34. equation time2=20;
35. //对象(Stop)
36. equation time1=0;
37. equation time2=0;
```

4) Transition 关系语句:

```
1. //转换关系 1
2. transition{
3.     when (time>=0.5); //0.5 分钟后
4.     do AudioSystem=true; //音频系统开启
5.     goto ReceiptAudioSignal; //进入接收音频信号状态
6. };
7. //转换关系 2
8. transition{
9.     when (when time>=0.9); //0.9 分钟后
10.    goto ExtractAudioPacket; //进入音频压缩包解压状态
11. };
12. //转换关系 3
13. transition{
14.    when (time>=1.2); //1.2 分钟后
15.    goto SendAudio; //进入发送音频状态
16. };
```

```
17. //转换关系 4
18.transition{
19.     when (time>=1.5); //1.5 分钟后
20.     goto ReceiptMusic; //进入收听音频音乐状态
21. }
22. //转换关系 5
23.transition{
24.     when (time>=20); //20 分钟后
25.     do AudioSystem=false; //音频系统关闭
26.     goto CurrentServiceInterruption; //进入当前服务终止状态
27. };
28. //转换关系 6
29.transition{
30.     when (time>=21); //21 分钟后
31.     do SendAudioAnnouncement=true; //开始播放广播信号
32.     goto ReceiptAudioBroadcast; //进入接收广播音频状态
33. };
34. //转换关系 7
35.transition{
36.     when (time>=23); //23 分钟后
37.     do SendAudioAnnouncement=false; //停止发送广播信号
38.     goto OnDemandMovie; //进入点播视频状态
39. };
40. //转换关系 8
41.transition{
42.     when (time>=25); //25 分钟后
43.     do VideoSystem=true; //视频系统开启
44.     goto ReceiptVideoSignal; //进入接收信号状态
45. }
46. //转换关系 9
47.transition{
48.     when (time>=25.3); //25.3 分钟后
49.     goto ExtractVideoPacket; //进入视频压缩包解压状态
50. }
51. //转换关系 10
52.transition{
53.     when (time>=26); //26 分钟后
54.     goto Sendvideo; //进入发送视频状态
55. }
56. //转换关系 11
57.transition{
```

```
58.    when (time>=26.5); //26.5 分钟后
59.    goto WatchMovie; //进入观看视频状态
60. }
61. //转换关系 12
62. transition{
63.    when (time>=50); //50 分钟后
64.    do VideoSystem=false; //视频系统关闭
65.    goto Stop; //进入停止状态
66. }
```

(3) 仿真文件编译及仿真：

系统行为建模与仿真语言对该系统行为描述后，通过点击 MetaGraph 工具中“仿真编译”按钮，则自动生成仿真文件。后点击“行为仿真”，在弹出的仿真配置页面中，输入仿真文件，设置仿真时间为 50（分钟）。点击“确认”后，会得到仿真结果，如图 6.9 所示。根据仿真结果可判断出在 50min 时，总耗电量 E 对应纵轴数值小于 4000，即 $3kW \cdot h$ ；总消耗流量 Y 对应数值小于 3000MB。以上两个结果中总耗电量 E 不满足仿真结果指标，总消耗流量 Y 满足仿真结果指标，可判断出该设计过程不合理，需要对该设计过程进行完善。

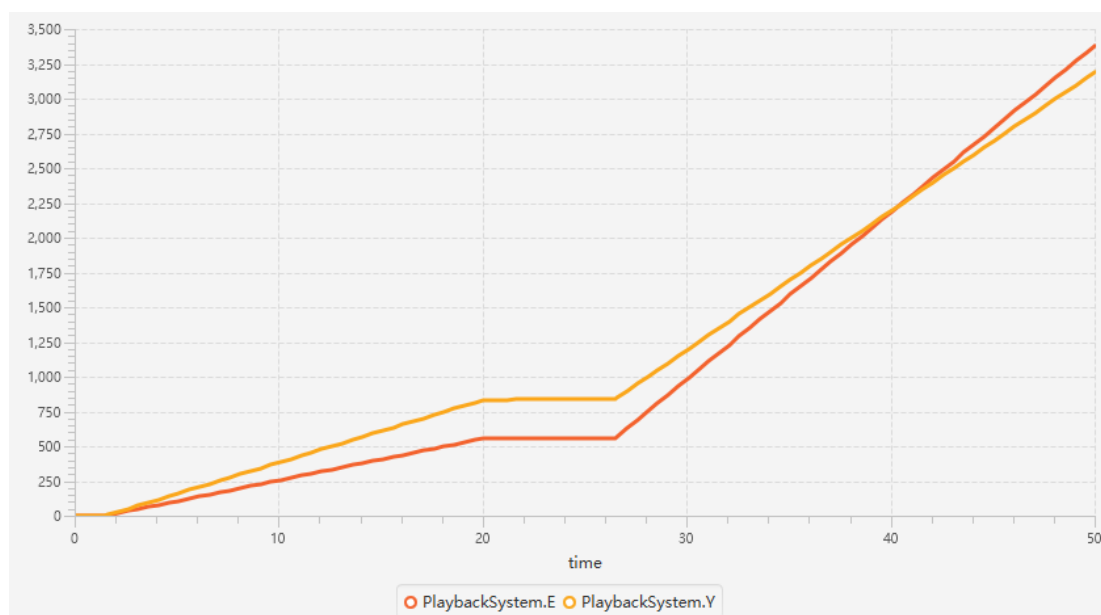


图 6.9 交互场景-音频视频娱乐方案序列图仿真结果

6.3 探测拦截-多状态机图仿真案例

6.3.1 案例背景

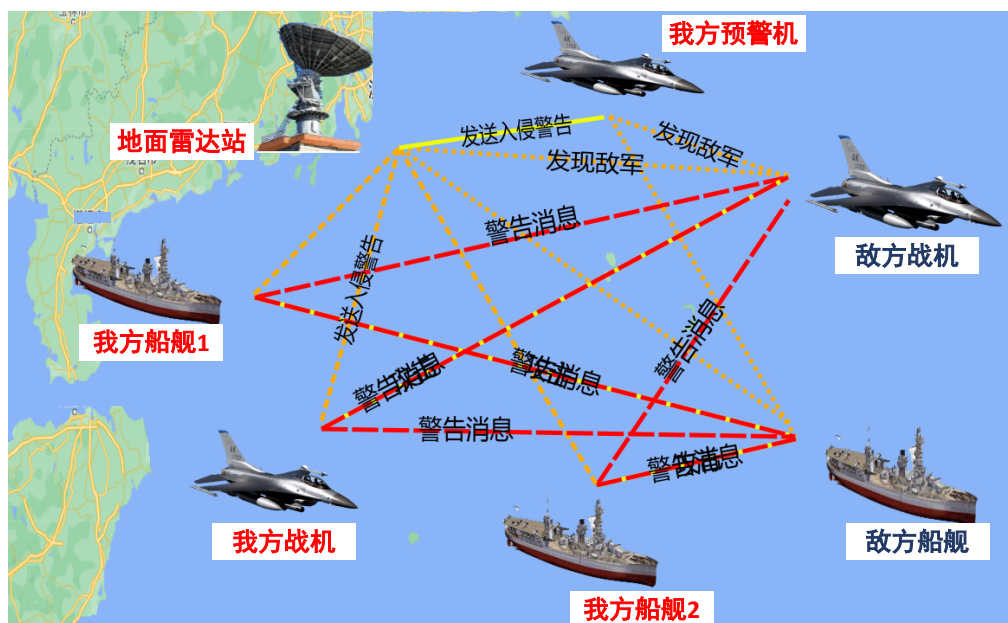


图 6.10 探测拦截示意图

案例示意图如图 6.10 所示，预警机巡逻我方领域，敌军入侵我方领域被我方地面雷达检测信号捕获后，雷达控制中心指挥战机与舰船拦截。首先战机与舰船寻找敌方，确认踪迹后发送驱逐警告信号，若敌方撤退则驱逐成功，若敌方对警告信号无反应且继续行驶，则构成入侵意图，控制中心指挥我方战机与舰船进行火力拦截，确保敌军被击毁，完成拦截任务后返航。

地面雷达站构建参考超视距雷达，设置探测距离为 2500km 。地面雷达站启动后进入探测状态，检测到敌人后进入预警状态，与敌人距离超过探测范围或敌人被击毁后重新进入探测状态。

预警机构建参考空警-2000，设置巡航速度为 850km/h ，挂载雷达探测距离为 470km 。预警机具有固定航线，并且巡航 3 次后返航。预警机启动后进入行驶状态，并按照设置的经纬点改变航线完成固定线路巡航，当检测到敌人后进入预警状态，并将加速度切换到与敌军一致，跟踪敌军。

舰艇参考 055 型导弹驱逐舰，设置行驶速度为 56km/h ，挂载雷达为 346 型雷达，探测距离为 450km ，火控系统的有效攻击范围为 200km 。

6.3.2 案例流程

探测拦截流程图如图 6.11 所示。

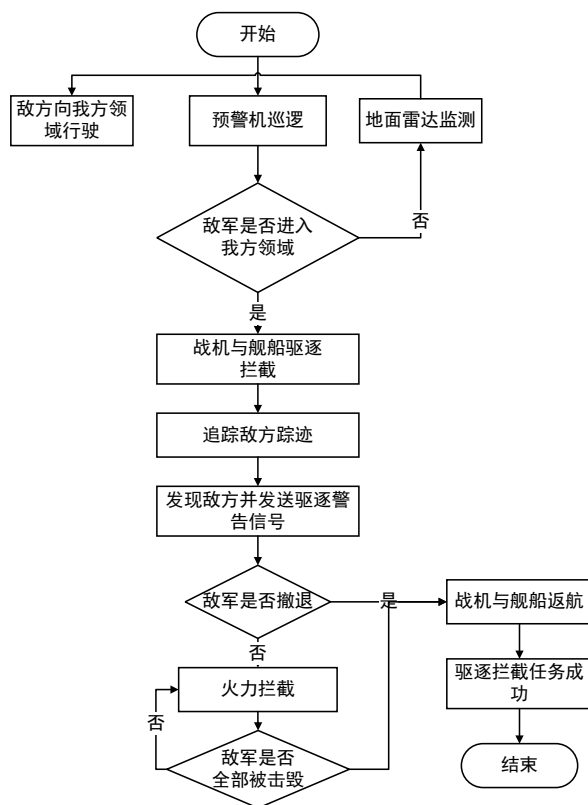


图 6.11 探测拦截流程图

(1) 我方预警机开始巡航任务，地面雷达站检测是否有异常装备单位进入我方领域，敌军向我方领域行驶。

(2) 我方预警机巡航任务结束。

(3) 敌军进入我方地面雷达站监测范围，地面雷达实时监控敌方行踪，判断是否进入我方领域。

(4) 敌军进入我方领域，雷达控制中心命令我方战机与舰船进行驱逐拦截。

(5) 地面雷达始终保持与我方装备信息同步，战机与舰船追踪敌方踪迹。

(6) 敌军进入我方战机与舰船雷达监测范围，对敌军发送驱逐警告信号。

(7) 敌军收到警告信号后，若选择撤退，则我方装备单位返航，完成驱逐拦截任务，地面雷达继续监测直到敌方驶出我方领域。

(8) 敌军收到警告信号后，若选择继续前进，则构成入侵威胁，我方装备进入迎敌状态。

(9) 敌军进入我方装备有效攻击范围后，对敌军进行火力驱逐直到全部敌军被击毁，我方装备返回，完成驱逐拦截任务。

当地面雷达站监测到敌方后，我方船舰进入行驶状态，并向敌方目标行驶。敌方进入雷达探测范围后向敌方发送警告信号，若敌方撤退则返航；若敌方继续行驶则启动火控系统准备击毁敌方装备单位。敌方进入有效攻击范围后进行火力拦截，直至击毁敌方单位后返航。敌方战舰在启动后进入行驶状态，并向我方领域行驶。收到我方发出的警告信息后，可选择继续前进或撤退。继续前进则进入入侵状态，若被敌方攻击两次则进入击毁状态；选择撤退则进入返航状态。

6.3.3 模型建立

根据案例背景和流程，建立 SysML 地面雷达站状态机图如图 6.12 所示；SysML 我方船舰、预警机、战机状态机图如图 6.13 所示；SysML 敌方船舰和敌方战机状态机图如图 6.14 所示。

地面雷达状态机图主要描述了雷达探测、侦测敌方意图、警告敌方三个状态，以及状态之间的转化。当探测到有异常装备时，地面雷达站处于“侦测敌方意图状态”。通过判断我方与敌方的距离来侦查敌方意图，如果敌方进入我方区域，则雷达发送警告信号；如果敌方返航，则雷达继续进入“探测状态”。

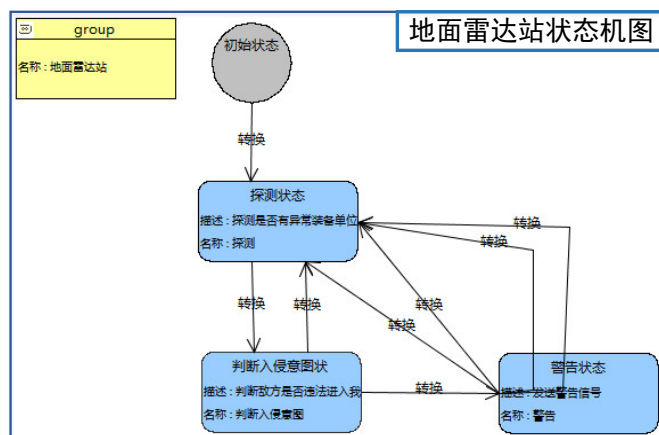


图 6.12 地面雷达站状态机图

我方舰船 1 状态机图和我方舰船 2 状态机图描述了我方战舰在拦截敌方战舰这个系统行为中具有“待命状态”、“行驶状态”、“驱逐状态”、“火控状态”、“攻击状态”、“返航”状态。

我方预警机状态机图中我方预警机在巡航一、二、三个阶段处于“巡航状态”，发现敌机采取警告，并追踪敌方。刚开始我方预警机处于“待命状态”。第三次巡航结束，会等待 2 分钟时间，随后进入到“待命状态”。

我方战机状态机图描述我方战机在收到出航命令后，进入“行驶状态”，并向着目标领域方向行驶。当接收到驱逐敌方警告信号，对敌方发出驱逐警告，通过判断与敌方的距离采取“返航”或“攻击敌方”。等待“火控冷却状态”结束后，进入到“返航状态”。

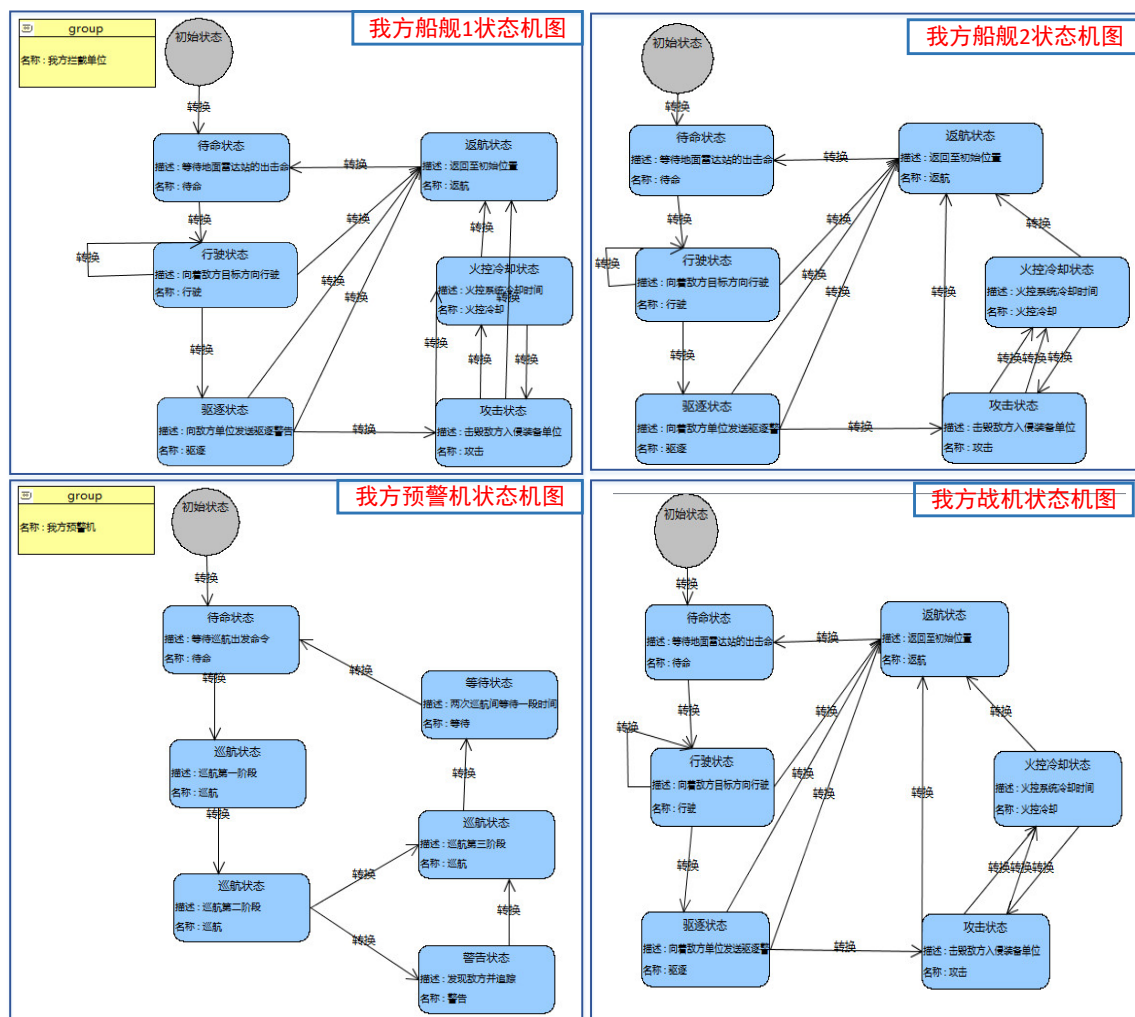


图 6.13 我方舰船、预警机、战机状态机图

敌方舰船状态机图描述敌方舰船在收到出航命令后,进入“行驶状态”,并向着目标领域方向行驶。当接收到“继续移动”信号,继续入侵目标领域。当接收到“返航”信号后进入“返航状态”,并继续等待出航命令。如果在“入侵状态”被击毁,则敌方舰船进入“被击毁状态”。

敌方战机状态机图描述敌方战机在收到出航命令后,进入“行驶状态”,并向着目标领域方向行驶。当接收到“继续移动”信号,继续入侵目标领域。当接收到“返航”信号后进入“返航状态”。如果在“入侵状态”被击毁,则敌方战机进入“被击毁状态”。

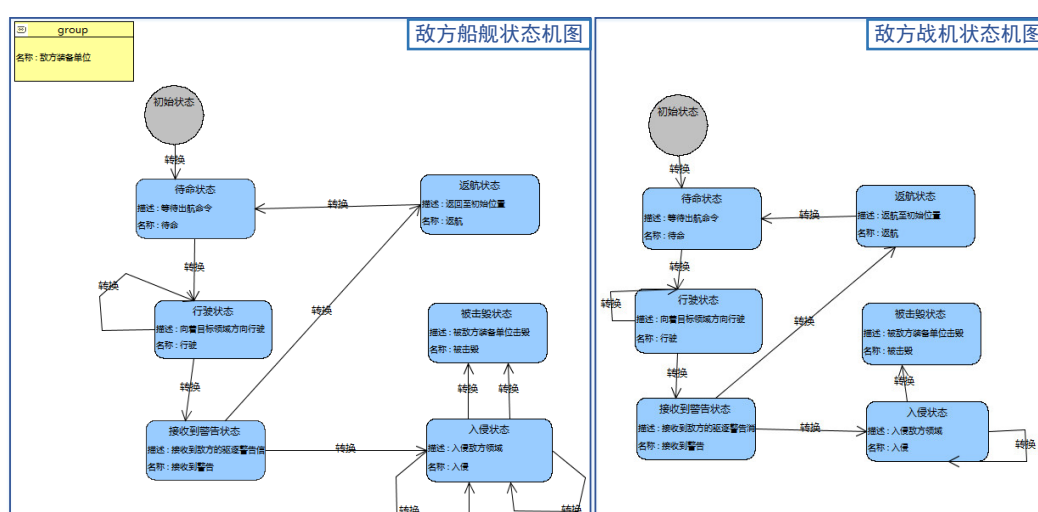


图 6.14 地方舰船和战机状态机图

6.3.4 案例系统行为仿真流程

由于上述七幅状态机图描述的分别是七个对象（地面雷达站、我方舰船 1、我方舰船 2、我方预警机、我方战机、敌方舰船、敌方战机）。由于这七个对象在探测拦截系统中都有参与,因此需要将这七个对象对应的行为模型编译生成的仿真文件存放在一个仿真文件中,进行多状态机图联合仿真。

在系统行为建模与仿真语言的基础上,分别对建立的每个状态机图进行系统行为描述,包括 Automaton 图语句、Group 对象语句、State 对象语句、Transition 关系语句。由于描述的代码量较大,详细内容见附录 B。然后对建立的七幅状态机图依次进行“仿真编译”,生成每个状态机图所对应的.sim 文件。最后通过仿真文件合并,生成一个总仿真文件。

混合自动机求解引擎支持可伸缩向量图形 SVG 可视化显示，因此需要在总仿真文件中添加对应的 SVG 代码。最后通过混合自动机求解引擎得出仿真结果，如图 6.16 所示。由于仿真结果曲线较多，用户可以在仿真配置选择需要单独显示的变量。SVG 可视化结果显示敌方战舰和敌方战机均已被成功驱逐，如图 6.17 所示。该案例的完整流程如图 6.15 所示。

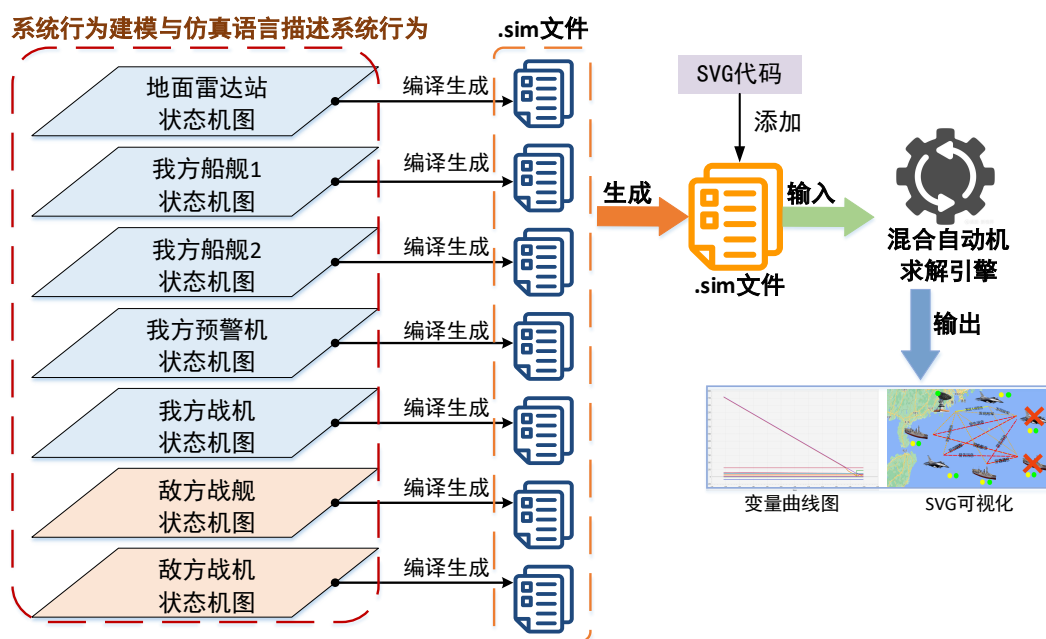


图 6.15 建模、仿真流程

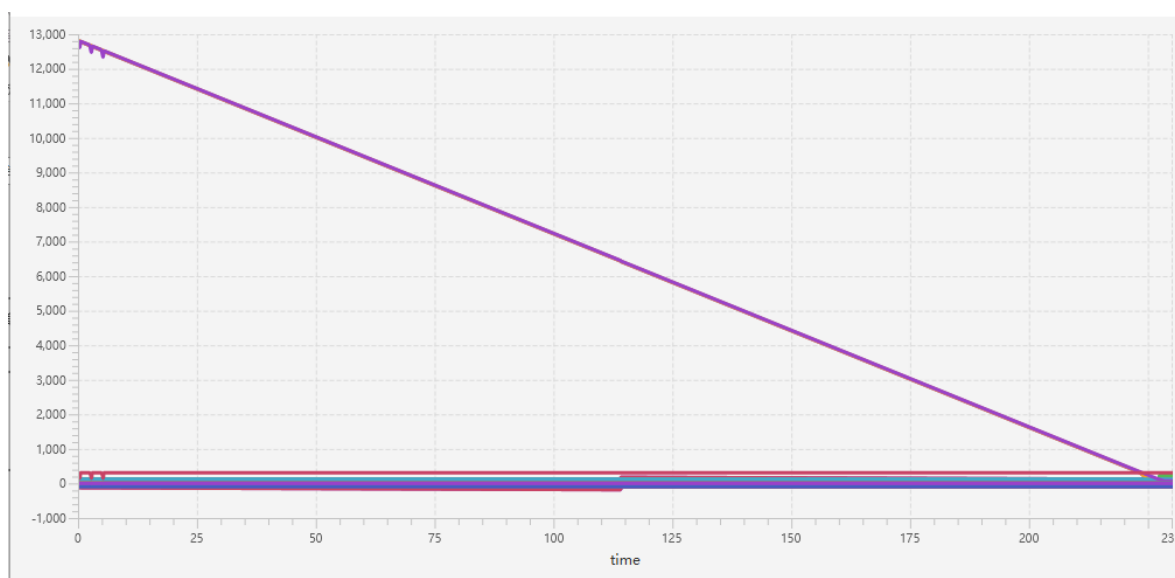


图 6.16 探测拦截系统系统行为仿真结果

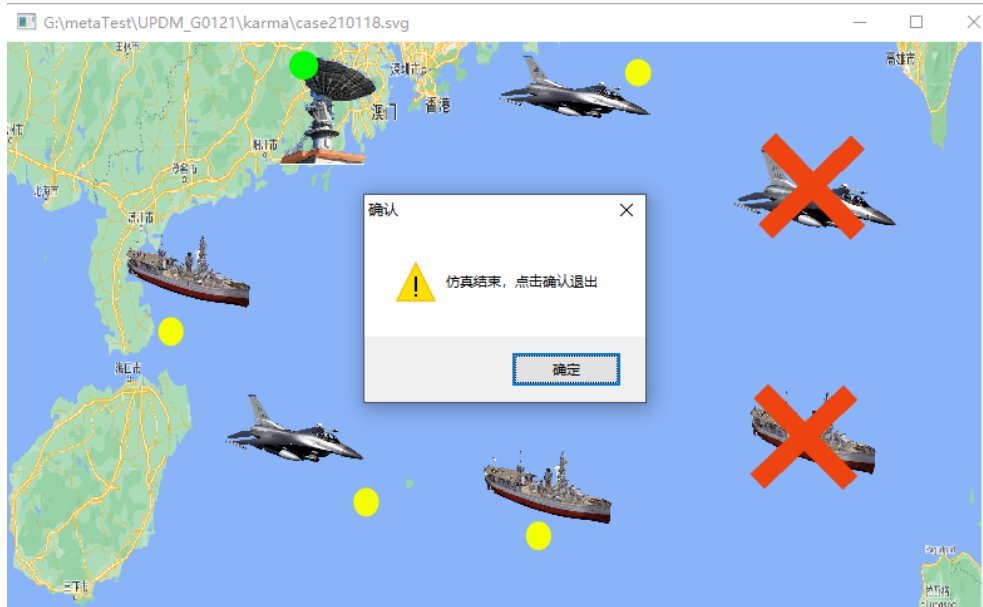


图 6.17 探测拦截系统系统行为 SVG 显示

6.4 本章小结

本章主要为验证系统行为建模与仿真语言的执行功能和可行性，分别建立复杂产品系统中的不同场景案例进行实现。最后对目前在工具 MetaGraph 中仿真功能的实现情况进行总结，如表 6.2 所示。

表 6.2 仿真功能实现总结

系统行为仿真类型	说明	是否支持
SysML 状态机图仿真		支持
SysML 活动图仿真		支持
SysML 序列图仿真		支持
UML 状态图仿真		支持
UML 活动图仿真		支持
UML 序列图仿真		支持
SysML 多状态机图联合仿真		支持
SysML 复合状态机图仿真		支持
SysML 多复合状态机图仿真	包含多个复合状态	不支持
SysML 多复合状态机图联合仿真	包含多个复合状态	不支持

第7章 总结与展望

7.1 工作总结

基于模型的系统工程是一种在系统开发过程中持续贯穿全生命周期的标准化建模方法，通过形式化的建模方式支持系统需求分析，系统分析与设计以及后续的仿真和验证。在系统的设计阶段，基于行为模型对系统进行仿真，能够验证系统行为逻辑的正确性和合理性，提前发现设计中存在的问题，从而有效避免后期系统实现中出现难以挽回的损失。目前在 MBSE 中进行系统行为仿真，主要存在两方面问题。一方面，用于系统行为仿真的工具平台依赖性强，与建模工具没有很好地结合。另一方面，由于复杂产品全生命周期研发过程往往是一个多学科交叉开发过程，进行系统行为描述及建模时，领域工程师采用 MBSE 方法需要建立各自特定域（UML, SysML, BPMN, UPDM 等）行为模型，缺少一种语言对这些建模语言的系统行为模型进行统一描述，以便后续进行仿真验证。

本文针对上述背景和存在问题，在 GOPPRR-E 元建模方法的基础上，结合系统行为参考模型以及可描述系统行为的元模型和模型，以及混合自动机系统行为描述的过程，扩充多架构建模语言 KARMA 语言语法和语义，设计系统行为建模与仿真语言，具备描述系统行为的能力。并在 MetaGraph 工具上设计开发系统行为仿真流程（即描述系统行为的 KARMA 语言实现过程），本文的主要工作内容总结为以下几个方面：

（1）本文针对在 MBSE 领域中，不同建模语言构建不同行为模型，缺少对系统行为模型统一描述构建的问题，采用 GOPPRR-E 元建模方法对总结分析出 SysML 和 UML 建模语言中可用于构建描述系统行为的模型进行统一描述构建。首先通过系统行为定义以及可描述系统行为的三个参考模型（数据流行为模型、控制流行为模型、状态机行为模型）为出发点，分析出在 SysML 和 UML 建模语言中可描述系统行为的模型。然后基于 GOPPRR-E 元建模方法，对上一步总结出的模型进行分析，尤其是描述系统行为的模型基本元素，可实现对系统行为模型统一描述和构建。

（2）本文针对描述混合系统的行为模型如何支持系统行为统一描述和构建问题，提出了支持混合自动机的系统行为描述方法，并结合 GOPPRR-E 元建模方法，实现混合系统行为模型的统一描述和表达。首先，结合目前复杂产品的复杂度不断提高，产品设计人员必须面对复杂的各类系统，其中包括混合系统。通过系统行为的机制和描

述方式，总结得出混合自动机可以更好的描述混合系统。然后通过详细介绍混合自动机理论、符号表达过程以及相关案例，为下一章节对可描述系统行为的 KARMA 语言设计作为参考依据。最后将混合自动机与 GOPRRR-E 元建模方法进行结合，解决混合自动机支持 SysML 和 UML 系统行为模型的统一描述和构建问题。

(3) 本文针对缺少一种语言对系统行为统一描述的问题，在多架构建模语言 KARMA 语言的基础上扩充 KARMA 的语法和语义，设计系统行为建模与仿真语言，具备描述系统行为的能力。KARMA 语言是一种文本式建模语言，支持描述各种系统架构视角。首先设计系统行为建模与仿真语言的抽象语法，主要包括四种类型语句模块，并展示出系统行为建模与仿真语言的数据类型。然后详细描述系统行为建模与仿真语言具体语法，主要包括描述系统行为的连续变量、离散变量、函数、转换关系等内容，方便用户对该语言的学习和理解。

(4) 本文针对可描述系统行为的系统行为建模与仿真语言实现问题，设计了系统行为仿真半自动化流程。本文首先介绍了混合自动机求解引擎，并将其集成在建模工具 MetaGraph 中。然后通过 ANTLR 语法解析器和构造语法编译器，通过编译规则，编译生成仿真所需文件。最后将仿真结果显示在工具前端，便于设计人员分析。

(5) 本文针对系统行为建模仿真中存在的多状态机图联合仿真、复合状态仿真两种状况，分别提出了多仿真文件合并和复合状态分解方法。多仿真文件合并方法通过用户手动选择需要合并的编译文件，自动生成最终仿真文件。复合状态分解方法通过将复合状态分解为一个行为模型的方式，实现复合状态的系统行为模型编译以及仿真。

通过以上总结，本文的创新点主要体现在以下两个方面：

(1) 设计可对系统行为模型进行统一描述的语言——系统行为建模与仿真语言。该语言结合 MBSE 中 GOPRRR-E 元建模方法论和混合自动机理论，对多架构建模语言 KARMA 语言进行语法和语义扩充，使其具备描述系统行为的能力。系统行为建模与仿真语言通过直观的四模块内容(Automaton 图、Group 对象、State 对象、Transition 关系)来描述系统行为，实现对 SysML 和 UML 中行为模型统一描述。

(2) 设计行为模型建立到仿真的半自动化流程。系统行为建模仿真过程方便、灵活，且容易理解。一方面，将混合自动机求解引擎与建模工具 MetaGraph 结合，建立模型建立到仿真的半自动化流程，增加设计模型到仿真模型的信息传递效率。另一方面，用户在使用系统行为建模与仿真语言进行系统行为描述过程时，只需要在元模型

层增加的四个模块中添加对应描述内容,使得建模仿真过程更为方便快捷。除此之外,在系统行为模型的半自动化仿真流程的基础上,提出的多仿真文件合并和状态分解方法,解决了多状态机联合仿真和复合状态仿真编译问题。

7.2 未来展望

本文以基于模型的系统工程中的系统行为仿真为背景,围绕可描述复杂产品系统行为模型,对支持系统行为描述的系统行为建模与仿真语言设计和实现过程进行了比较深入的研究。为了进一步完善系统行为建模与仿真编译过程中的便捷和优越性,未来还需要从以下两个方面进行完善:

(1) 本文目前对 SysML 和 UML 建模语言中的行为模型进行分析,未来还需要对 BPMN、UPDM 等多架构建模语言中行为模型的表达进行分析,以支持多种建模语言中系统行为模型的描述。

(2) 在进行更为复杂的系统行为描述过程中,未来还需要继续根据不同场景扩充系统行为建模与仿真语言的具体语法和抽象语法,并优化仿真文件编译过程,以支持多种复杂场景的系统行为描述和仿真。

参考文献

- [1] Hobday M, Rush H, Tidd J. Innovation in complex product and system[J]. Elsevier, 2000.
- [2] Ying L I, Sun K W, Yang J, et al. Model-based system configuration approach for Internetware[J]. Science China, 2013(08):1-20.
- [3] 张楚贤, 李世其, 蔡冽. 混合系统建模理论与混合系统模型[J]. 计算机仿真, 2007,024(012):81-84.
- [4] Alur, R., Courcoubetis, et al. The algorithmic analysis of hybrid systems[J]. THEORETICAL COMPUTER SCIENCE -AMSTERDAM-, 1995.
- [5] Authors S. A Survey of Model-Based Systems Engineering (MBSE) Methodologies[J]. 2013.
- [6] 韩凤宇, 林益明, 范海涛. 基于模型的系统工程在航天器研制中的研究与实践[J]. 航天器工程, 2014,23(003):119-125.
- [7] 刘玉生, 袁文强, 樊红日, 等. 基于 SysML 的模型驱动复杂产品设计的信息集成框架研究[J]. 中国机械工程, 2012(12):1438-1445.
- [8] 邵健. 基于 MBSE 的航天任务分析与设计方法[D]. 哈尔滨工业大学, 2015.
- [9] Engineering I. INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities[M]. INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities, 2015.
- [10] Lukei M, Dr B H, Dr R D, et al. Modular Inspection Equipment Design for Modular Structured Mechatronic Products – Model Based Systems Engineering Approach for an Integrative Product and Production System Development[J]. Procedia Technology, 2016,26:455-464.
- [11] McKelvin J M, Jimenez A. Specification and design of electrical flight system architectures with SysML[M]//2012:2534.
- [12] 王崑声, 袁建华, 陈红涛, 等. 国外基于模型的系统工程方法研究与实践[J]. 中国航天, 2012(11):52-57.
- [13] Conroy M, Mazzone R, Lin W. NASA Integrated Model-Centric Architecture (NIMA) model use and re-use: Aerospace Conference, 2012[C].
- [14] Schindel B, Dove R. Introduction to the Agile Systems Engineering Life Cycle MBSE Pattern: John Wiley & Sons, Ltd., 2016[C].

- [15] Andersson H, Herzog E, Johansson G, et al. Experience from introducing Unified Modeling Language/Systems Modeling Language at Saab Aerosystems[J]. Systems Engineering, 2010,13(4):369-380.
- [16] Ramos, A., Ferreira, et al. Model-Based Systems Engineering: An Emerging Approach for Modern Systems[J]. IEEE Transactions on Systems Man & Cybernetics Part C Applications & Reviews A Publication of the IEEE Systems Man & Cybernetics Society, 2012.
- [17] Wu Y M, Sun L, Wu L J. A SFMEA method based on modeling and simulation by Rhapsody, 2014[C]. Trans Tech Publ, 2014.
- [18] 葛立敏, 刘远恒, 王扬. 基于模型的系统工程在航电系统设计中的应用研究[J]. 航空制造技术, 2015,477(8):60-63.
- [19] 丁鼎. 基于模型的系统工程在民机领域的应用[J]. 沈阳航空航天大学学报, 2012,29(004):47-50, 54.
- [20] 曹雷, 薛平贞, CAOLei, 等. 基于模型驱动设计方法的雷达仿真系统实现[J]. 电子科技, 2013,26(10):56.
- [21] 黄昌彬. 基于 MBSE 的导弹测试流程设计与优化仿真[D]. 哈尔滨工业大学, 2019.
- [22] 尧伟文. 基于 MBSE 的舰载无人机飞控系统架构研究[D]. 南京航空航天大学, 2017.
- [23] AA A I. Modeling and Simulation of Ground Based Radar Surveillance Solutions for Unmanned Aircraft System Sense and Avoid, 2011[C].
- [24] 阚宝铎. 基于 SysML 的北斗接收机建模与仿真研究[D]. 电子科技大学, 2017.
- [25] Evans A, France R, Lano K, et al. The UML as a Formal Modeling Notation: International Conference on the Unified Modeling Language, 2003[C].
- [26] Burger E J. Flexible views for view-based model-driven development[J]. Components and architecture, 2013.
- [27] Johnson T, Kerzhner A, Paredis C, et al. Integrating Models and Simulations of Continuous Dynamics Into SysML[J]. Journal of Computing & Information Science in Engineering, 2012,38(1):122-129.
- [28] Chinosi M, Trombetta A. BPMN: An introduction to the standard[J]. Computer Standards & Interfaces, 2012,34(1):124-134.
- [29] Roques P. Systems architecture modeling with the arcadia method : a practical guide to capella[J]. 2017.

- [30] 孙岩, 王光伟, 何新华, 等. 基于 UPDM 的武器装备体系建模[J]. 装甲兵工程学院学报, 2013(02):70-73.
- [31] Hause M. The Unified Profile for DoDAF/MODAF (UPDM) enabling systems of systems on many levels: Systems Conference, 2010[C].
- [32] Feiler P H, Gluch D P. Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language[J]. Pearson Schweiz Ag, 2013.
- [33] IDEF0 Function Model of Rapid Prototyping Networked Manufacturing System[J]. Value Engineering, 2014.
- [34] 赵建军, 陈立平, 华中科技大学国家 CAD 支撑软件工程技术研究中心 武汉. Modelica 语言及其多领域统一建模与仿真机理: 中国系统仿真学会全国会员代表大会暨全国学术年会, 2006[C].
- [35] 曹悦. 基于 SysML 的多域复杂机电产品系统层建模与仿真集成研究[D]. 浙江大学, 2011.
- [36] Beek D, Fokkink W J, Hendriks D, et al. CIF 3 : model-based engineering of supervisory controllers: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, 2014[C].
- [37] Agut D, Beek D V, Rooda J E. Syntax and semantics of the compositional interchange format for hybrid systems[J]. Ifac Proceedings Volumes, 2013,82(1):1-52.
- [38] Theunissen R, Beek D, Rooda J E. Improving evolvability of a patient communication control system using state-based supervisory control synthesis[J]. Advanced Engineering Informatics, 2012,26(3):502-515.
- [39] Kamphuis R, Bax I. Design and real-time implementation of a supervisory controller for baggage handling at Veghel Airport[J].
- [40] Dijk D V. Supervisory control of a Philips MRI-scanner / D. van Dijk; supervisor W.J. Fokkink; advisors D.A. van Beek, F. van Liere[J]. 2013.
- [41] Johnson T, Kerzhner A, Paredis C, et al. Integrating Models and Simulations of Continuous Dynamics Into SysML[J]. Journal of Computing & Information Science in Engineering, 2012,38(1):122-129.
- [42] Schamai W, Fritzson P, Paredis C, et al. Towards Unified System Modeling and Simulation with ModelicaML: Modeling of Executable Behavior Using Graphical Notations: In Proceedings of the 7th International Modelica Conference (Modelica'2009), Como, Italy, September.20-22, 2009., 2009[C].
- [43] Cao Y, Liu Y, Fa N H, et al. SysML-based uniform behavior modeling and automated mapping of design and simulation model for complex mechatronics[J]. Computer-Aided Design, 2013,45(3):764-776.

- [44] 李新光, 刘继红. 基于 SysML 的系统设计-仿真模型可视化转换[J]. 计算机辅助设计与图形学学报, 2016,28(011):1973-1981.
- [45] Schamai W, Fritzson P, Paredis C, et al. Towards Unified System Modeling and Simulation with ModelicaML: Modeling of Executable Behavior Using Graphical Notations: In Proceedings of the 7th International Modelica Conference (Modelica'2009), Como, Italy, September.20-22, 2009., 2009[C].
- [46] 张政. 基于 SysML 和 Modelica 的多领域设计和仿真建模集成[D]. 浙江大学, 2017.
- [47] Dandan W, Hao L, JianWen C. Modeling and Simulation of Top-level Design Based on MBSE, 2020[C]. IOP Publishing, 2020.
- [48] Kindler E. Model-Based Software Engineering and Process-Aware Information Systems[J]. Springer-Verlag, 2009.
- [49] Gude V. Predicting complex system behavior using hybrid modeling and computational intelligence[J]. 2020.
- [50] Wenjing, Zhang, Yakun, et al. Data-driven direct automatic tuning scheme for fixed-structure digital controllers of hybrid systems[J]. IET Control Theory Applications, 2019.
- [51] 陈倩, 余维, 叶阳东. 一种基于 TF-HBPN 的复杂系统行为分析方法[J]. 计算机科学, 2015(07):62-67.
- [52] Huang E, Mcginnis L F, Mitchell S W. Verifying SysML activity diagrams using formal transformation to Petri nets[J]. Systems Engineering, 2020,23(1).
- [53] 薛乐, 魏晨, 陈宗基. 混合系统的建模设计与仿真综述[J]. 计算机仿真, 2006(06):6-10.
- [54] Yuan S, Wan Y, Zhang L, et al. Control for hybrid systems: Applications and methods for adaptation and optimality[J]. Optimal Control Applications and Methods, 2020,41(6):1811-1812.
- [55] Alur, R., Courcoubetis, et al. The algorithmic analysis of hybrid systems[J]. THEORETICAL COMPUTER SCIENCE -AMSTERDAM-, 1995.
- [56] Alawida M, Teh J S, Samsudin A, et al. An image encryption scheme based on hybridizing digital chaos and finite state machine[J]. Signal Processing, 2019,164:249-266.
- [57] Kone Ny M, Taha W, Barthä F, et al. Enclosing the behavior of a hybrid automaton up to and beyond a Zeno point[J]. Nonlinear Analysis Hybrid Systems, 2016,20:1-20.
- [58] Deterministic finite automaton[J]. 2015.
- [59] Si Me U-Abazi Z, Gascard E. Fault diagnosis method for timed discrete-event systems: Application

to autonomous electric vehicle: 2019 International Conference on Control, Automation and Diagnosis (ICCAD), 2020[C].

[60] Vodencarevic A, Buning H K, Niggemann O, et al. Using behavior models for anomaly detection in hybrid systems[J]. IEEE, 2011.

[61] Frehse G. An Introduction to Hybrid Automata, Numerical Simulation and Reachability Analysis[M]. Formal Modeling and Verification of Cyber-Physical Systems, 2015.

[62] Lu J, Wang G, Wang H, et al. General Modeling Language for Supporting Model-based Systems Engineering Formalisms (Part 1): 30th Annual INCOSE International Symposium, 2020[C].

[63] Graignic P, Vosgien T, Jankovic M, et al. Complex System Simulation: Proposition of a MBSE Framework for Design-Analysis Integration[J]. Procedia Computer Science, 2013,16(1):59-68.

[64] Guo J, Wang G, Lu J, et al. General Modeling Language Supporting Model Transformations of MBSE (Part 2): INCOSE International Symposium, 2020[C].

[65] Wang H, Wang G, Lu J, et al. Ontology Supporting Model-Based Systems Engineering Based on a GOPPRR Approach[M]. New Knowledge in Information Systems and Technologies, 2019.

[66] Sharma A, Reniers M. Integrated simulation of CIF3 and Simulink models[M]//2016:33-37.

攻读学位期间发表论文与研究成果清单

参与科研项目

主要参研国家重点研发计划

发表论文

Model-based Systems Engineering Supporting Integrated Modeling and Optimization of Radar Cabin Layout（第一作者，International Conference Advances in Production Management Systems（APMS），2021）

学科竞赛

1. 2019 年 5 月，北京理工大学研究生科技创新项目校级重点团队
2. 2020 年 9 月，北京理工大学第十七届“世纪杯”学生创业竞赛银奖
3. 2020 年 9 月，第六届中国国际“互联网+”大学生创新创业大赛（北京赛区复赛）一等奖

附录 A

```

modelClause
:
(
    propertyClause
    | optAutDeclsNew //自动机属性-图
    | objectInstance
    | relationshipInstance
    | connectClause
    | concreteSyntaxClause
    | sVGPartClause
    | externalFunctionDeclareClause
)*
;

objectInstance
:
(
    'Group'
    | 'State'
)? 'Object' IDENT
(
    'refer'
    | 'clone'
    | 'copy'
) IDENT
(
    propertyClause
    | cifAutDeclLocaDecl
    | pointInstance
    | explosionClause
    | decompositionClause
    | concreteSyntaxClause
)* 'end' IDENT ';';

cifAutDeclLocaDecl
:
    continuousClause
    | constantClause
    | algebraicClause
    | behaviorfunctionClause
    | equationClause
    | initialStateClause
    | equationClause
;

relationshipInstance
:
    'Relationship' IDENT 'refer' IDENT
    (
        'extends' IDENT
    )?
    (
        propertyClause
        | transitionPartClause //行为仿真关系
        | explosionClause
        | roleInstance
        | decompositionClause
        | concreteSyntaxClause
    ) * 'end' IDENT ';';

/*=====Behavior Simulation
Clause define===== */
//The beginning of Behavior Simulation
optAutDeclsNew
:
    discreteClause
    | algebraicClause
    | equationClause
    | eventClause
    | continuousClause
    | constantClause
    | enumClause
;

enumClause
:
    fieldDeclaration
;

edgeElementsNew
:
    transitionPartClause
;

continuousClause
:
    'continuous' type contDecl ';';
;

contDecl
:
    identifier optDerivative
    | identifier unitCif? '=' values
;

values
:
    expression optDerivative
;

optDerivative
:
    (
        'der' expression
    )?
;

constantClause
:

```

'constant' assingmentDeclarement ';' ;	
algebraicClause	optElseifExprs
: 'algebraic' assingmentDeclarement ';' ;	: ('else if' expression ':' expression)*
behaviorfunctionClause	switchBody
: operationDeclaration ;	: switchCases switchCases 'else' expression 'else' expression ;
funcExpression	switchCases
: funcExpressionOne funcExpressionTwo ;	: ('case' expression ':' expression)+ ;
funcExpressionOne	dictPairs
: expressionFactor stdLibFunction '(' ')' stdLibFunction '(' expressions ')' ;	: (expression ':' expression) (';' (expression ':' expression))*
funcExpressionTwo	optExpression
: ('[' expression ']' '[' optExpression ':' optExpression ']' '(' ')' '(' expressions ')')*	: expression? ;
expressionFactor	nonEmptySetExpression
: 'true' 'false' REAL 'Real' stringToken 'time' '[' ']' '[' expressions ']' '{' '}' nonEmptySetExpression '{' dictPairs '}' '(' expression ',' expressions ')' 'if' expressions ':' expression 'switch' expression ':' switchBody 'end' '<' type '>' expressionFactor '(' expression ')' name name ',' '?' 'self'	: '{' expressions '}' ;
optElseifExprs 'else' expression 'end'	name
'switch' expression ':' switchBody 'end'	: identifier relativeName absoluteName rootName ;
'<' type '>' expressionFactor	rootName
'(' expression ')'	: '\^' (
name	
name ','	
'?'	
'self'	

identifier relativeName) ; absoluteName : ' ' (identifier relativeName) ; relativeName : identifier (' ' identifier)+ ; equationClause : 'equation' equations ';' ; equations : equation (' ' equation)* ; equation : identifier \" '=' expression identifier '=' expression ; discreteClause : 'discrete' fieldDeclaration ; eventClause : 'event' identifier ';' ; 'event' 'void' identifier ';' ; ; initialStateClause : 'initial' ' ' ; transitionPartClause	: 'transition' '{' transitionBody '}' ';' ; ; transitionBody : transitionBodyEvent transitionBodyWhen transitionBodyDo transitionBodyGoto ; transitionBodyEvent : ('event' edgeEvents ';')? ; transitionBodyWhen : ('when' '(' expressions ')' ';')? ; ; expressions : expression (' ' expression)* ; updates : update (' ' update)* ; update : addressable '=' expression selectionStatement ; addressables : addressable (' ' addressable)* ; addressable : identifier identifier projections
--	---

'(' addressable ',' addressables ')'	'Boolean'
;	'Real'
projections	<u>'Int'</u>
:	'Double'
projection+	'Cell'
;	'Set'
projection	'List'
:	'Dictionary'
[' expression ']	'Distribution'
;	'Map'
;	;
edgeEvents	stdLibFunction
:	:
edgeEvent	<u>'acosh'</u>
(	<u>'acos'</u>
',' edgeEvent	<u>'asinh'</u>
)*	<u>'asin'</u>
;	<u>'atanh'</u>
edgeEvent	<u>'atan'</u>
:	<u>'cosh'</u>
'tau'	<u>'cos'</u>
name	<u>'sinh'</u>
name '!'	<u>'sin'</u>
name '!' expression	<u>'tanh'</u>
name '?'	<u>'tan'</u>
;	<u>'abs'</u>
transitionBodyDo	<u>'cbrt'</u>
:	<u>'ceil'</u>
(	<u>'del'</u>
'do' updates ';'	'empty'
)?	<u>'exp'</u>
;	'floor'
transitionBodyGoto	<u>'fmt'</u>
:	<u>'ln'</u>
(	'log'
'goto' identifier ';'	'max'
)?	<u>'min'</u>
;	'pop'
stringToken	<u>'pow'</u>
:	'round'
'String'	'scale'
;	'sign'
identifier	'size'
:	<u>'sqrt'</u>
IDENT	<u>'bernoulli'</u>
;	'beta'
type	;
:	sVGPartClause
'String'	:
'Number'	'SVGPart' '{' optloDecls '}'
'Text'	;
	externalFunctionDeclareClause
	:
	externalFunctionInput
	;
	//The end of Behavior Simulation

附录 B

1.地面雷达站

(1) Group 对象语句:

```

constant Real SCOUNTRANGE = 2200; //地面雷达站扫描范围
constant Real TERRITORIAL_WATER = 392.624; //海域范围
constant Real PI = 3.14159265358979328;
constant Real EARTH_RADIUS = 6378137; //地球半径
func Real rad(real d){ //将经纬度转为弧度
    return d * PI / 180.0;}
//根据两点经纬度计算两点距离, 单位 km
// lon1,lat1 为一点经纬度, lon2,lat2 为另一点经纬度
func Real LatitudeLongitudeDist(Real lon1, Real lat1, Real lon2, Real lat2){
    //Convert latitude and longitude to radians
    Real radLat1 = rad(lat1);
    Real radLat2 = rad(lat2);
    Real radLon1 = rad(lon1);
    Real radLon2 = rad(lon2);
    Real x1,y1,z1,x2,y2,z2,d,theta,disten;
    x1=EARTH_RADIUS * cos(radLon1) * sin(radLat1);
    y1=EARTH_RADIUS * sin(radLon1) * sin(radLat1);
    z1=EARTH_RADIUS * cos(radLat1);
    x2 = EARTH_RADIUS * cos(radLon2) * sin(radLat2);
    y2 = EARTH_RADIUS * sin(radLon2) * sin(radLat2);
    z2 = EARTH_RADIUS * cos(radLat2);
    d = sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2) + (z1 - z2) * (z1 - z2));
    //Azimuth and distance
    theta = acos((EARTH_RADIUS * EARTH_RADIUS + EARTH_RADIUS *
EARTH_RADIUS - d * d) / (2 * EARTH_RADIUS * EARTH_RADIUS));
    disten = theta * EARTH_RADIUS;
    return (disten+25000)/1000;}

```

(2) Automaton 图语句:

```

constant Real x = 30.931224; //经度
constant Real y = 121.920634; //维度
discrete Boolean start = true; //启动标识符
discrete Boolean warn_inf = false; //警告信号

```

(3) State 对象语句:

```
initial;
```

(4) Transition 关系语句:

```
transition{
```

```

when (start);
do start =false;
goto detect;};

transition{
  when (OurWariningAircraft1.warn_inf = true || ((!Enemiesbattleship1.dead &&
CoastalRadar.LantitudeLongitudeDist(x,y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y)
CoastalRadar.SCOUNTRANGE) || (!Enemiesbattleship2.dead &&
CoastalRadar.LantitudeLongitudeDist(x,y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y)
CoastalRadar.SCOUNTRANGE)));
  goto judge;};

transition{
  when (((! Enemiesbattleship1.dead && CoastalRadar.LantitudeLongitudeDist(x,y,
  Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y) < CoastalRadar.TERRITORIAL_WATER) || (!
Enemiesbattleship2.dead && CoastalRadar.LantitudeLongitudeDist(x,y,Enemiesbattleship2.bs_x,
  Enemiesbattleship2.bs_y) < CoastalRadar.TERRITORIAL_WATER)));
  do warn_inf = true;
  goto warning;};

transition{
  when(CoastalRadar.LantitudeLongitudeDist(x,y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y
) > CoastalRadar.SCOUNTRANGE && CoastalRadar.LantitudeLongitudeDist(x,y,
Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y) > CoastalRadar.SCOUNTRANGE);
  do warn_inf=false;
  goto detect;};

transition{
  when (Enemiesbattleship1.dead && Enemiesbattleship2.dead);
  do warn_inf=false;
  goto detect;};

transition{
  when(CoastalRadar.LantitudeLongitudeDist(x,y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs
_y) > CoastalRadar.SCOUNTRANGE and CoastalRadar.LantitudeLongitudeDist
(x,y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y) > CoastalRadar.SCOUNTRANGE);
  do warn_inf=false;
  goto detect;};

transition{
  when (Enemiesbattleship1.dead && CoastalRadar.LantitudeLongitudeDist

```

```

(x,y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y) > CoastalRadar.SCOUNTRANGE);
do warn_inf=false;
goto detect;}

transition{
when (Enemiesbattleship2.dead && CoastalRadar.LantitudeLongitudeDist
(x,y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y) > CoastalRadar.SCOUNTRANGE);
do warn_inf=false;
goto detect;}

```

2. 预警机

(1) Group 对象语句:

```

constant Real SCOUNTRANGE = 470; //挂载雷达扫描范围
constant Real PI = 3.14159265358;
constant Real EARTH_RADIUS = 6378137; //地球半径
constant Real speed = 850; //预警机巡航速度
//计算转弯后的 xy 方向上的加速度
//speed 是该装备速度, ang 是转弯角度
func List(Real) turn(Real a_x, Real a_y, Real speed Real,ang){
Real a_x_trans = (speed * sin(PI*ang/180) - a_x); //a_x 是原始 x 方向加速度
Real a_y_trans = (speed * cos(PI*ang/180) - a_y); //a_y 是原始 y 方向加速度
List(Real) A = {a_x_trans,a_y_trans};
return A;}
//根据初始位置、目标位置以及速度计算 xy 方向加速度
//current_x 是初始位置经度, current_y 是初始位置维度
//aim_x 是目标位置经度, aim_y 是目标位置维度
//speed 是该装备速度
func List(Real) aim(Real current_x, Real current_y, Real aim_x, Real aim_y,speed){
Real x_trans = aim_x - current_x;
Real y_trans = aim_y - current_y;
Real distance,spend,a_x,a_y;
List(Real) A;
if(aim_y < 0)
y_trans = 180-current_y+180+aim_y;
end
distance = LantitudeLongitudeDist(current_x,current_y,aim_x,aim_y);
spend = distance / speed;
a_x = x_trans / spend;
a_y = y_trans / spend;
A = [a_x,a_y];
return A;}
//将经纬度转为弧度

```

```

func Real rad(Real d){
    return d * PI / 180.0;}
//根据两点经纬度计算两点距离，单位 km
// lon1,lat1 为一点经纬度，lon2,lat2 为另一点经纬度
func Real LantitudeLongitudeDist(Real lon1, Real lat1, Real lon2, Real lat2){
    Real radLat1 = rad(lat1);
    Real radLat2 = rad(lat2);
    Real radLon1 = rad(lon1);
    Real radLon2 = rad(lon2);
    Real x1,y1,z1,x2,y2,z2,d,theta,disten;
    x1 = EARTH_RADIUS * cos(radLon1) * sin(radLat1);
    y1 = EARTH_RADIUS * sin(radLon1) * sin(radLat1);
    z1 = EARTH_RADIUS * cos(radLat1);
    x2 = EARTH_RADIUS * cos(radLon2) * sin(radLat2);
    y2 = EARTH_RADIUS * sin(radLon2) * sin(radLat2);
    z2 = EARTH_RADIUS * cos(radLat2);
    d = sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2) + (z1 - z2) * (z1 - z2));
    theta = acos((EARTH_RADIUS * EARTH_RADIUS + EARTH_RADIUS *
EARTH_RADIUS - d * d) / (2 * EARTH_RADIUS * EARTH_RADIUS));
    disten = theta * EARTH_RADIUS;
    return (disten+25000)/1000;}

```

(2) Automaton 图语句:

```

constant Real x = 30.003969;
constant Real y = 122.393492;
continuous Real bs_x = x;
continuous Real bs_y = y;
discrete Boolean start = true;
continuous Real trans_x = 0;
continuous Real trans_y = 0;
continuous Real temp1 = 0 der 0;
continuous Real temp2 = 0 der 0;
discrete Real angle = 90;
algebraic Real r_speed = sqrt(pow(bs_x',2)+pow(bs_y',2));
continuous Real live = 1 der 0;
discrete Boolean warn_inf = false;
discrete Real patrolNums = 3;
discrete Real waitTime = 0;
algebraic Real disten = OurWariningAircraft.LantitudeLongitudeDist
(bs_x,bs_y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y);
algebraic Real distence = OurWariningAircraft.LantitudeLongitudeDist
(bs_x,bs_y,29.223781,124.888571);

```

(3) State 对象语句:

//对象 idle initial; equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 move equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 move2 equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;
//对象 warning equation bs_x'=Enemiestbattleship1.trans_x; equation bs_y'=Enemiestbattleship1.trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 returnBack equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 wait equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;

(4) Transition 关系语句:

```

transition{
    when (patrolNums != 0 do trans_x = OurWariningAircraft.aim
(x,y,31.965655,124.576915,OurWariningAircraft.speed)[0], trans_y = OurWariningAircraft.aim(x,y,
31.965655,124.576915,OurWariningAircraft.speed)[1]);
    goto move;};

transition{
    when (abs(bs_x-31.965655) < 1 and abs(bs_y-124.576915) < 1 do trans_x =
OurWariningAircraft.aim(bs_x,bs_y,29.223781,124.888571,OurWariningAircraft.speed)[0], trans_y =
OurWariningAircraft.aim(bs_x,bs_y,29.223781,124.888571,OurWariningAircraft.speed)[1]);
    goto move2;};

transition{
    when (!Enemiestbattleship1.dead && !Enemiestbattleship2.dead &&
(OurWariningAircraft.LantitudeLongitudeDist(bs_x,bs_y,Enemiestbattleship1.bs_x,Enemiestbattleship1.b
s_y) < OurWariningAircraft.SCOUNTRANGE || OurWariningAircraft.LantitudeLongitudeDist(bs_x,
bs_y,Enemiestbattleship2.bs_x,Enemiestbattleship2.bs_y) < OurWariningAircraft.SCOUNTRANGE));
    do warn_inf = true;};
    goto warning;};

transition{
    when (abs(bs_x-29.223781) < 1 && abs(bs_y-124.888571) < 1);
    do trans_x = OurWariningAircraft.aim(bs_x,bs_y,x,y,OurWariningAircraft.speed)[0], trans_y
= OurWariningAircraft.aim(bs_x,bs_y,x,y,OurWariningAircraft.speed)[1]);
    goto returnBack;};

transition{
    when (!Enemiestbattleship1.dead && !Enemiestbattleship2.dead && (OurWariningAircraft.
LantitudeLongitudeDist(bs_x,bs_y,Enemiestbattleship1.bs_x,Enemiestbattleship1.bs_y) <

```

```

OurWaringAircraft.SCOUNTRANGE || OurWaringAircraft.LantitudeLongitudeDist(bs_x,bs_y,
Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y) < OurWaringAircraft.SCOUNTRANGE));
    do warn_inf = true;
    goto warning;};

transition{
    when (Enemiesbattleship1.dead);
    do warn_inf = false, trans_x = OurWaringAircraft.aim(bs_x,bs_y,x,y,OurWaring
Aircraft.speed)[0], trans_y = OurWaringAircraft.aim(bs_x,bs_y,x,y,OurWaringAircraft.speed)[1];
    goto returnBack;};

transition{
    when ((abs(bs_x-x) <= 1) && (abs(bs_y-y) <= 1));
    do patrolNums=patrolNums-1,waitTime=time,bs_x= x,bs_y= y;
    goto wait;};

transition{
    when (!Enemiesbattleship1.dead && !Enemiesbattleship2.dead &&(OurWaringAircraft.
LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y) <
OurWaringAircraft.SCOUNTRANGE || OurWaringAircraft.LantitudeLongitudeDist(bs_x,bs_y,
Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y) < OurWaringAircraft.SCOUNTRANGE));
    do warn_inf = true;
    goto warning;};

transition{
    when (time - waitTime > 2);
    goto idel;};

```

3.我方舰船 1

(1) Group 对象语句:

```

constant Real SCOUNTRANGE = 200; //挂载雷达扫描范围
constant Real PI = 3.141592653;
constant Real EARTH_RADIUS = 6378137;
constant Real speed = 56;
constant Real FIRE_SCOUNTRANGE = 30; //有效攻击范围
func List(Real) turn(real a_x,a_y,speed,ang){
    real a_x_trans = (speed * sin(PI*ang/180) - a_x)/10;
    real a_y_trans = (speed * cos(PI*ang/180) - a_y)/10;
    List(Real) A = {a_x_trans,a_y_trans};
    return A;};
func List(Real) aim(Real current_x, Real current_y, Real aim_x, Real aim_y, Real speed){
    Real x_trans = aim_x - current_x;

```

```

Real y_trans = aim_y - current_y;
Real distance, spend, a_x, a_y;
List(Real) A;
if(aim_y < 0)
    y_trans = 180 - current_y + 180 + aim_y;
end
distance = LantitudeLongitudeDist(current_x, current_y, aim_x, aim_y);
spend = distance / speed;
a_x = x_trans / spend;
a_y = y_trans / spend;
A = {a_x, a_y};
return A;
}

func Real rad(real d){
    return d * PI / 180.0;
}

func Real LantitudeLongitudeDist(Real lon1, Real lat1, Real lon2, Real lat2){
    Real radLat1 = rad(lat1);
    Real radLat2 = rad(lat2);
    Real radLon1 = rad(lon1);
    Real radLon2 = rad(lon2);
    Real x1, y1, z1, x2, y2, z2, d, theta, disten;
    x1 = EARTH_RADIUS * cos(radLon1) * sin(radLat1);
    y1 = EARTH_RADIUS * sin(radLon1) * sin(radLat1);
    z1 = EARTH_RADIUS * cos(radLat1);
    x2 = EARTH_RADIUS * cos(radLon2) * sin(radLat2);
    y2 = EARTH_RADIUS * sin(radLon2) * sin(radLat2);
    z2 = EARTH_RADIUS * cos(radLat2);
    d = sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2) + (z1 - z2) * (z1 - z2));
    theta = acos((EARTH_RADIUS * EARTH_RADIUS + EARTH_RADIUS *
EARTH_RADIUS - d * d) / (2 * EARTH_RADIUS * EARTH_RADIUS));
    disten = theta * EARTH_RADIUS;
    return (disten + 25000) / 1000;
}

```

(2) Automaton 图语句:

```

event aimed;
event unaimed;
constant Real x = 30.382812;
constant Real y = 122.499132;
continuous Real bs_x = x;
continuous Real bs_y = y;
discrete Boolean start = true;

```

```

continuous Real trans_x = 0;
continuous Real trans_y = 0;
continuous Real temp1 = 0 der 0;
continuous Real temp2 = 0 der 0;
discrete Real angle = -90;
algebraic Real r_speed = sqrt(pow(bs_x',2)+pow(bs_y',2));
continuous Real live = 1 der 0;
discrete Real fireTime = 0;
discrete Boolean warn_inf = false;
algebraic Real disten =

```

```
Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y);
```

(3) State 对象语句:

//对象 idle initial; equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 move equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 warning equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;
//对象 fire equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 firing equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 returnBack equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;

(4) Transition 关系语句:

```

transition{
    when (start && CoastalRadar1.warn_inf=true);
    do start=false , trans_x = Ourbattleship.aim(x,y,Enemiesbattleship2.bs_x,Enemiesbattleship2.
bs_y,Ourbattleship.speed)[0], trans_y = Ourbattleship.aim(x,y,Enemiesbattleship2.bs_x,
Enemiesbattleship2.bs_y,Ourbattleship.speed)[1];
    goto move;};

transition{
    when (
        Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y)
< Ourbattleship.SCOUNTRANGE);
    do warn_inf = true;
    goto warning;};

transition{
    event Enemiesbattleship2.returning;
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

```

```
transition{
    when (abs(bs_y - 180) < 0.1);
    do bs_y = -180;
    goto move;};

transition{
    when (
        Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y)
        < Ourbattleship.FIRE_SCOUNTRANGE);
    goto fire;};

transition{
    event Enemiesbattleship2.returning;
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when (Enemiesbattleship2.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    event unaimed;
    when (Enemiesbattleship2.live != 0);
    do fireTime = time;
    goto firing;};

transition{
    event aimed;
    when (Enemiesbattleship2.live != 0);
    do fireTime = time;
    goto firing;};

transition{
    when (Enemiesbattleship2.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};
```

```

transition{
    when (time - fireTime > 1 && Enemiesbattleship2.live != 0);
    goto fire;};

transition{
    when (Enemiesbattleship2.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when ((abs(bs_x-x) <= 0.1) && (abs(bs_y-y) <= 0.1));
    do bs_x= x,bs_y= y;
    goto idle;};

```

4.我方舰船 2

(1) Automaton 图语句:

```

event aimed;
event unaimed;
constant Real x = 30.282812;
constant Real y = 122.509132;
continuous Real bs_x = x;
continuous Real bs_y = y;
discrete Boolean start = true;
continuous Real trans_x = 0;
continuous Real trans_y = 0;
continuous Real temp1 = 0 der 0;
continuous Real temp2 = 0 der 0;
discrete Real angle = -90;
algebraic Real r_speed = sqrt(pow(bs_x',2)+pow(bs_y',2));
continuous Real live = 1 der 0;
discrete Real fireTime = 0;
discrete Boolean warn_inf = false;
algebraic Real disten = Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,
Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y);

```

(2) State 对象语句:

//对象 idle initial; equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 move equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 warning equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;
//对象 fire equation bs_x'=0;	//对象 firing equation bs_x'=0;	//对象 returnBack equation bs_x'=trans_x;

equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;
---	---	---

(3) Transition 关系语句:

```

transition{
    when (start && CoastalRadar1.warn_inf=true);
    do start=false , trans_x = Ourbattleship.aim(x,y,Enemiesbattleship2.bs_x,Enemiesbattleship2.
bs_y,Ourbattleship.speed)[0], trans_y = Ourbattleship.aim(x,y,Enemiesbattleship2.bs_x,
Enemiesbattleship2.bs_y,Ourbattleship.speed)[1];
    goto move;};

transition{
    when (
    Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y)
< Ourbattleship.SCOUNTRANGE);
    do warn_inf = true;
    goto warning;};

transition{
    event Enemiesbattleship2.returning;
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when (abs(bs_y - 180) < 0.1);
    do bs_y = -180;
    goto move;};

transition{
    when (
    Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship2.bs_x,Enemiesbattleship2.bs_y)
< Ourbattleship.FIRE_SCOUNTRANGE);
    goto fire;};

transition{
    event Enemiesbattleship2.returning;
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{

```

```

    when (Enemiesbattleship2.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    event unaimed;
    when (Enemiesbattleship2.live != 0);
    do fireTime = time;
    goto firing;};

transition{
    event aimed;
    when (Enemiesbattleship2.live != 0);
    do fireTime = time;
    goto firing;};

transition{
    when (Enemiesbattleship2.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when (time - fireTime > 1 && Enemiesbattleship2.live != 0);
    goto fire;};

transition{
    when (Enemiesbattleship2.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when ((abs(bs_x-x) <= 0.1) && (abs(bs_y-y) <= 0.1));
    do bs_x= x,bs_y= y;
    goto idle;};

```

5.我方战机

(1) Automaton 图语句:

```

event aimed;
event unaimed;

```

```

constant Real x = 30.472812;
constant Real y = 122.469132;
continuous Real bs_x = x;
continuous Real bs_y = y;
discrete Boolean start = true;
continuous Real trans_x = 0;
continuous Real trans_y = 0;
continuous Real temp1 = 0 der 0;
continuous Real temp2 = 0 der 0;
discrete Real angle = -90;
algebraic Real r_speed = sqrt(pow(bs_x',2)+pow(bs_y',2));
continuous Real live = 1 der 0;
discrete Real fireTime = 0;
discrete Boolean warn_inf = false;
algebraic Real disten = Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,
Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y);

```

(2) State 对象语句:

//对象 idle initial; equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 move equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 warning equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;
//对象 fire equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 firing equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 returnBack equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;

(3) Transition 关系语句

```

transition{
    when (start && CoastalRadar1.warn_inf=true);
    do start=false , trans_x = Ourbattleship.aim(x,y,Enemiesbattleship1.bs_x,Enemiesbattleship1.
bs_y,Ourbattleship.speed)[0], trans_y = Ourbattleship.aim(x,y,Enemiesbattleship1.bs_x,
Enemiesbattleship1.bs_y,Ourbattleship.speed)[1];
    goto move;};

transition{
    when (
    Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y)
< Ourbattleship.SCOUNTRANGE);
    do warn_inf = true;
    goto warning;};

transition{

```

```

    event Enemiesbattleship1.returning;
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when (abs(bs_y - 180) < 0.1);
    do bs_y = -180;
    goto move;};

transition{
    when (
    Ourbattleship.LantitudeLongitudeDist(bs_x,bs_y,Enemiesbattleship1.bs_x,Enemiesbattleship1.bs_y)
    < Ourbattleship.FIRE_SCOUNTRANGE);
    goto fire;};

transition{
    event Enemiesbattleship1.returning;
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when (Enemiesbattleship1.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    event unaimed;
    when (Enemiesbattleship1.live != 0);
    do fireTime = time;
    goto firing;};

transition{
    event aimed;
    when (Enemiesbattleship1.live != 0);
    do fireTime = time;
    goto firing;};

transition{

```

```

    when (Enemiesbattleship1.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when (time - fireTime > 1 && Enemiesbattleship1.live != 0);
    goto fire;};

transition{
    when (Enemiesbattleship1.live = 0);
    do warn_inf = false,trans_x = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[0],
trans_y = Ourbattleship.aim(bs_x,bs_y,x,y,Ourbattleship.speed)[1];
    goto returnBack;};

transition{
    when ((abs(bs_x-x) <= 0.1) && (abs(bs_y-y) <= 0.1));
    do bs_x= x,bs_y= y;
    goto idle;};

```

6. 敌方船舰

(1) Group 对象语句:

```

constant Real SCOUNTRANGE = 200;
constant Real PI = 3.14159265358;
constant Real EARTH_RADIUS = 6378137;
constant Real speed = 56;
constant Real FIRE_SCOUNTRANGE = 30;
func List(Real) turn(real a_x,a_y,speed,ang){
    real a_x_trans = (speed * sin(PI*ang/180) - a_x)/10;
    real a_y_trans = (speed * cos(PI*ang/180) - a_y)/10;
    List(Real) A = {a_x_trans,a_y_trans};
    return A;};

func List(Real) aim(Real current_x,Real current_y,Real aim_x,Real aim_y,Real speed){
    Real x_trans = aim_x - current_x;
    Real y_trans = aim_y - current_y;
    Real distance,spend,a_x,a_y;
    List(Real) A;
    if(current_y < 0)
        y_trans = -180-current_y-180+aim_y;
    end
    distance = LantitudeLongitudeDist(current_x,current_y,aim_x,aim_y);

```

```

    spend = distance / speed;
    a_x = x_trans / spend;
    a_y = y_trans / spend;
    A = {a_x, a_y};
    return A;}

func Real rad(real d){
    return d * PI / 180.0;}

func Real LantitudeLongitudeDist(Real lon1, Real lat1, Real lon2, Real lat2){
    Real radLat1 = rad(lat1);
    Real radLat2 = rad(lat2);
    Real radLon1 = rad(lon1);
    Real radLon2 = rad(lon2);
    Real x1, y1, z1, x2, y2, z2, d, theta, disten;
    x1 = EARTH_RADIUS * cos(radLon1) * sin(radLat1);
    y1 = EARTH_RADIUS * sin(radLon1) * sin(radLat1);
    z1 = EARTH_RADIUS * cos(radLat1);
    x2 = EARTH_RADIUS * cos(radLon2) * sin(radLat2);
    y2 = EARTH_RADIUS * sin(radLon2) * sin(radLat2);
    z2 = EARTH_RADIUS * cos(radLat2);
    d = sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2) + (z1 - z2) * (z1 - z2));
    theta = acos((EARTH_RADIUS * EARTH_RADIUS + EARTH_RADIUS *
EARTH_RADIUS - d * d) / (2 * EARTH_RADIUS * EARTH_RADIUS));
    disten = theta * EARTH_RADIUS;
    return (disten + 25000) / 1000;}

```

(2) Automaton 图语句:

```

event moving;
event returning;
constant Real x = 33.191406;
constant Real y = -122.583744;
continuous Real bs_x = x;
continuous Real bs_y = y;
discrete Boolean start = true;
continuous Real trans_x = 0;
continuous Real trans_y = 0;
continuous Real temp1 = 0 der 0;
continuous Real temp2 = 0 der 0;
discrete Real angle = -90;
algebraic Real r_speed = sqrt(pow(bs_x', 2) + pow(bs_y', 2));
continuous Real live = 2 der 0;

```

(3) State 对象语句:

//对象 idel initial; equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0	//对象 move equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 getWarning equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;
//对象 invade equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 dead equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 returnBack equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;

(4) Transition 关系语句:

```

transition{
    when (start);
    do start=false , trans_x =Enemiesbattleship.aim(x,y,Ourbattleship1.bs_x,Ourbattleship1.bs_y,
Enemiesbattleship.speed)[0], trans_y = Enemiesbattleship.aim(x,y,Ourbattleship1.bs_x,
Ourbattleship1.bs_y,Enemiesbattleship.speed)[1];
    goto move;};

transition{
    when (Ourbattleship1.warn_inf = true || Ourbattleship2.warn_inf = true ||
Ourbattleship3.warn_inf = true);
    goto getWarning; };

transition{
    when (abs(bs_y + 180) < 0.1);
    do bs_y = 180;
    goto move; };

transition{
    event moving;
    goto invade; };

transition{
    event returning;
    do trans_x = Enemiesbattleship.aim(bs_x,bs_y,x,y,Enemiesbattleship.speed)[0], trans_y =
Enemiesbattleship.aim(bs_x,bs_y,x,y,Enemiesbattleship.speed)[1];
    goto returnBack; };

transition{
    event Ourbattleship1.aimed;
    when(live = 1)
    do live = live-1;

```

```
goto dead; };
```

```
transition{
    event Ourbattleship2.aimed;
    when(live = 1)
    do live = live-1;
    goto dead; };
```

```
transition{
    event Ourbattleship1.aimed;
    when(live = 1)
    do live = live-1;
    goto invade; };
```

```
transition{
    event Ourbattleship2.aimed;
    when(live != 1)
    do live = live-1;
    goto invade; };
```

```
transition{
    when((abs(bs_x-x) <= 0.1) && (abs(bs_y-y) <= 0.1))
    do bs_x= x,bs_y= y;
    goto idle; };
```

7.敌方战机

(1) Automaton 图语句:

```
event moving;
event returning;
constant Real x = 33.291406;
constant Real y = -122.553744;
continuous Real bs_x = x;
continuous Real bs_y = y;
discrete Boolean start = true;
continuous Real trans_x = 0;
continuous Real trans_y = 0;
continuous Real temp1 = 0 der 0;
continuous Real temp2 = 0 der 0;
discrete Real angle = -90;
algebraic Real r_speed = sqrt(pow(bs_x',2)+pow(bs_y',2));
continuous Real live = 2 der 0;
```

(2) State 对象语句:

//对象 idel initial; equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0	//对象 move equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 getWarning equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;
//对象 invade equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;	//对象 dead equation bs_x'=0; equation bs_y'=0; equation trans_y' = 0; equation trans_x' = 0;	//对象 returnBack equation bs_x'=trans_x; equation bs_y'=trans_y; equation trans_y' = 0; equation trans_x' = 0;

(3) Transition 关系语句:

```

transition{
    when (start);
    do start=false , trans_x =Enemiesbattleship.aim(x,y,Ourbattleship3.bs_x,Ourbattleship3.bs_y,
Enemiesbattleship.speed)[0], trans_y = Enemiesbattleship.aim(x,y,Ourbattleship3.bs_x,
Ourbattleship3.bs_y,Enemiesbattleship.speed)[1];
    goto move;};

transition{
    when (Ourbattleship1.warn_inf = true || Ourbattleship2.warn_inf = true ||
Ourbattleship3.warn_inf = true);
    goto getWarning; };

transition{
    when (abs(bs_y + 180) < 0.1);
    do bs_y = 180;
    goto move; };

transition{
    event moving;
    goto invade; };

transition{
    event returning;
    do trans_x = Enemiesbattleship.aim(bs_x,bs_y,x,y,Enemiesbattleship.speed)[0], trans_y =
Enemiesbattleship.aim(bs_x,bs_y,x,y,Enemiesbattleship.speed)[1];
    goto returnBack; };

transition{
    event Ourbattleship3.aimed;
    when(live = 1)
    do live = live-1;
    goto dead; };

```

```

transition{
    event Ourbattleship3.aimed;
    when(live != 1)
    do live = live-1;
    goto invade; };

transition{
    when((abs(bs_x-x) <= 0.1) && (abs(bs_y-y) <= 0.1))
    do bs_x= x,bs_y= y;
    goto idle; };
//SVG 可视化代码
svgfile "case210118.svg";
svgout id "comm_os1_es2" attr "visibility"
    value switch Ourbattleship1:
        case idle: "hidden"
        case move: "hidden"
        case warning: "hidden"
        case fire: "visible"
        case firing: "visible"
        case returnBack: "hidden"
    end;
svgout id "font_os1_es2" attr "visibility"
    value switch Ourbattleship1:
        case idle: "hidden"
        case move: "hidden"
        case warning: "hidden"
        case fire: "visible"
        case firing: "visible"
        case returnBack: "hidden"
    end;
svgout id "comm_os1_es2_r" attr "visibility"
    value switch Ourbattleship1:
        case idle: "hidden"
        case move: "hidden"
        case warning: "visible"
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
svgout id "font_os1_es2_r" attr "visibility"

```

```
value switch Ourbattleship1:
  case idle: "hidden"
  case move: "hidden"
  case warning: "visible"
  case fire: "hidden"
  case firing: "hidden"
  case returnBack: "hidden"
end;
svgout id "comm_os2_es2" attr "visibility"
value switch Ourbattleship2:
  case idle: "hidden"
  case move: "hidden"
  case warning: "hidden"
  case fire: "visible"
  case firing: "visible"
  case returnBack: "hidden"
end;
svgout id "font_os2_es2" attr "visibility"
value switch Ourbattleship2:
  case idle: "hidden"
  case move: "hidden"
  case warning: "hidden"
  case fire: "visible"
  case firing: "visible"
  case returnBack: "hidden"
end;
svgout id "comm_os2_es2_r" attr "visibility"
value switch Ourbattleship2:
  case idle: "hidden"
  case move: "hidden"
  case warning: "visible"
  case fire: "hidden"
  case firing: "hidden"
  case returnBack: "hidden"
end;
svgout id "font_os2_es2_r" attr "visibility"
value switch Ourbattleship2:
  case idle: "hidden"
  case move: "hidden"
  case warning: "visible"
  case fire: "hidden"
```

```
        case firing: "hidden"
        case returnBack: "hidden"
    end;
    svgout id "comm_os3_es1" attr "visibility"
    value switch Ourbattleship3:
        case idle: "hidden"
        case move: "hidden"
        case warning: "hidden"
        case fire: "visible"
        case firing: "visible"
        case returnBack: "hidden"
    end;
    svgout id "font_os3_es1" attr "visibility"
    value switch Ourbattleship3:
        case idle: "hidden"
        case move: "hidden"
        case warning: "hidden"
        case fire: "visible"
        case firing: "visible"
        case returnBack: "hidden"
    end;
    svgout id "comm_os3_es1_r" attr "visibility"
    value switch Ourbattleship3:
        case idle: "hidden"
        case move: "hidden"
        case warning: "visible"
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
    svgout id "font_os3_es1_r" attr "visibility"
    value switch Ourbattleship3:
        case idle: "hidden"
        case move: "hidden"
        case warning: "visible"
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
    svgout id "comm_os1_es1" attr "visibility"
    value switch Ourbattleship1:
```

```
        case idle: "hidden"
        case move: "hidden"
        case warning: if Enemiesbattleship1.dead: "hidden" else "visible" end
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
svgout id "font_os1_es1" attr "visibility"
    value switch Ourbattleship1:
        case idle: "hidden"
        case move: "hidden"
        case warning: if Enemiesbattleship1.dead: "hidden" else "visible" end
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
svgout id "comm_os2_es1" attr "visibility"
    value switch Ourbattleship2:
        case idle: "hidden"
        case move: "hidden"
        case warning: if Enemiesbattleship1.dead: "hidden" else "visible" end
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
svgout id "font_os2_es1" attr "visibility"
    value switch Ourbattleship2:
        case idle: "hidden"
        case move: "hidden"
        case warning: if Enemiesbattleship1.dead: "hidden" else "visible" end
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "hidden"
    end;
svgout id "comm_os3_es2" attr "visibility"
    value switch Ourbattleship3:
        case idle: "hidden"
        case move: "hidden"
        case warning: if Enemiesbattleship2.dead: "hidden" else "visible" end
        case fire: "hidden"
        case firing: "hidden"
```

```

        case returnBack: "hidden"
    end;
    svgout id "font_os3_es2" attr "visibility"
        value switch Ourbattleship3:
            case idle: "hidden"
            case move: "hidden"
            case warning: if Enemiesbattleship2.dead: "hidden" else "visible" end
            case fire: "hidden"
            case firing: "hidden"
            case returnBack: "hidden"
        end;
    svgout id "comm_os1_cr1" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "comm_os2_cr1" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "comm_os3_cr1" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "comm_cr1_es1" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "comm_cr1_es2" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "font_os3_cr1" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "font_cr1_es1" attr "visibility" value if CoastalRadar1.warning: "visible" else "hidden" end;
    svgout id "comm_wa1_cr1" attr "visibility" value if OurWariningAircraft1.warning: "visible" else
    "hidden" end;
    svgout id "comm_wa1_es1" attr "visibility" value if OurWariningAircraft1.warning: "visible" else
    "hidden" end;
    svgout id "comm_wa1_es2" attr "visibility" value if OurWariningAircraft1.warning: "visible" else
    "hidden" end;
    svgout id "font_wa1_cr1" attr "visibility" value if OurWariningAircraft1.warning: "visible" else "hidden"
    end;
    svgout id "font_wa1_es1" attr "visibility" value if OurWariningAircraft1.warning: "visible" else "hidden"
    end;
    svgout id "eship1_dead" attr "visibility" value if Enemiesbattleship1.dead: "visible" else "hidden" end;
    svgout id "eship2_dead" attr "visibility" value if Enemiesbattleship2.dead: "visible" else "hidden" end;

    svgout id "os1_y" attr "visibility" value if Ourbattleship1.idle: "visible" else "hidden" end;
    svgout id "os2_y" attr "visibility" value if Ourbattleship2.idle: "visible" else "hidden" end;
    svgout id "os3_y" attr "visibility" value if Ourbattleship3.idle: "visible" else "hidden" end;
    svgout id "es1_y" attr "visibility" value if Enemiesbattleship1.idle: "visible" else "hidden" end;
    svgout id "es2_y" attr "visibility" value if Enemiesbattleship2.idle: "visible" else "hidden" end;
    svgout id "wa1_y" attr "visibility" value if OurWariningAircraft1.idle: "visible" else "hidden" end;

    svgout id "os1_g" attr "visibility"
        value switch Ourbattleship1:
            case idle: "hidden"

```

```
        case move: "visible"
        case warning: "hidden"
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "visible"
    end;
    svgout id "os2_g" attr "visibility"
    value switch Ourbattleship2:
        case idle: "hidden"
        case move: "visible"
        case warning: "hidden"
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "visible"
    end;
    svgout id "os3_g" attr "visibility"
    value switch Ourbattleship3:
        case idle: "hidden"
        case move: "visible"
        case warning: "hidden"
        case fire: "hidden"
        case firing: "hidden"
        case returnBack: "visible"
    end;
    svgout id "es1_g" attr "visibility"
    value switch Enemiesbattleship1:
        case idle: "hidden"
        case move: "visible"
        case getWarning: "hidden"
        case invade: "hidden"
        case dead: "hidden"
        case returnBack: "visible"
    end;
    svgout id "es2_g" attr "visibility"
    value switch Enemiesbattleship2:
        case idle: "hidden"
        case move: "visible"
        case getWarning: "hidden"
        case invade: "hidden"
        case dead: "hidden"
        case returnBack: "visible"
```

```
end;
svgout id "wa1_g" attr "visibility"
  value switch OurWarningAircraft1:
    case idle: "hidden"
    case move: "visible"
    case move2: "visible"
    case warning: "hidden"
    case returnBack: "visible"
    case wait: "hidden"
  end;

svgout id "cr1_g" attr "visibility" value if CoastalRadar1.detect: "visible" else "hidden" end;
end
```

致谢

论文写到这里，意味着三年的研究生生涯即将结束，此刻心中感慨万千。内心不仅有对北理工美好的回忆，也有对未来的向往。但此刻更多的是体会到“珍惜”二字的意义以及表达对老师、同学、家人和朋友的深深谢意！

首先，感谢我的导师，王国新老师。在三年时间里，尽管老师的科研、教学工作任务繁重，但总是在百忙之中对我的课题研究过程中遇到的困难给予耐心的指导和帮助。王老师一丝不苟的治学态度、与人和善的处事风格都使我受益良多。生活中给予的关心和帮助更让我铭记于心，在此谨向王老师表达我真挚的感谢！

感谢瑞士联邦理工大学的鲁金直博士，对我的科研悉心指导。虽然鲁博士在国外，但每次线上会议指导都让我有很大的收获。鲁博士对工作、科研的热情，细致认真的态度也深深影响着我。

感谢我的前导师，段晓峰老师。感谢段老师给予我生活和学习中的关心和鼓励，使我顺利适应研究生的科研生活。永远怀念您！

感谢课题组的阎艳老师、郝佳老师、姚丽亚老师、明振军老师。阎老师出色的领导能力、渊博的学识、亲切随和的做事风格，为我们树立了奋斗的榜样。每次听取阎艳老师在活动中的讲话，总让我备受鼓舞和感动。郝老师在我刚入学，科研不顺利的时候，给予我多次鼓励和帮助。姚老师和明老师在我刚入学时，为我的科研方向提出很多宝贵建议。

感谢工业与智能系统工程研究所的胡耀光老师、薛庆老师、宫琳老师、张发平老师、敬石开老师、闻敬谦老师、刘敏霞老师。实验室的每一位老师都在我的学习生活中留下了深刻的印象。

感谢系统工程小组的每位同学：马君达师兄、郭江敏、陈婧琦、栗寒、丁洁、吴绶玄、李子航。小组内融洽的氛围以及努力的学习态度，带给了我无限的快乐。

感谢知识工程组的每位师兄和同学：王儒师兄、黄思瀚师兄、贾良跃师兄、牛红伟师兄、叶文斌师兄、周梦莹、靳益利、曾梦梦、陈雪健和各位学弟学妹。课题组是一个温暖的大家庭，曾经一起出差、聚餐的场景历历在目。

感谢我亲爱的舍友：郭江敏、万斯来、霍阳阳。一次次的帮助，我都铭记于心。读研期间，能认识三位挚友，也是我的荣幸。

感谢父母对我一如既往的关心、鼓励和支持。多年来的默默付出，是我奋斗的动力。感谢家人们一直以来对我的关心和照顾。

本论文的顺利完成，也参考了许多前辈辛苦的科研成果。在此，向本论文中引用的文献作者以及科研工作者致以深深的敬意。

最后，衷心感谢各位老师和专家在百忙之中评审本论文，并提出宝贵的意见。向各位辛苦的老师致敬！

二零二一年五月
于北京理工大学 1 号教学楼